

PyCuDAL

VALIDATION PROTOCOL

PyCuDAL — Content Uniformity and Dissolution Acceptance Limits (Python Implementation)

Approval

Role	Name (printed)	Signature	Date
Performed By	Moaz El-Essawey		
Reviewed By			
Approved By			

TABLE OF CONTENTS

- 1.0 Purpose
- 2.0 Scope
- 3.0 References
- 4.0 Definitions and Abbreviations
- 5.0 Responsibilities
- 6.0 System Description
- 7.0 Validation Approach
- 8.0 Acceptance Criteria
- 9.0 Deviation Handling
- 10.0 Documentation and Retention

Appendices

- Appendix A — Program Listing
- Appendix B — Primary Windows
- Appendix C — Default Output for Analysis (R Code)
- Appendix D — Navigation and Error Checks
- Appendix E — Math / Lower Bound Calculations
- Appendix F — Program Description
- Appendix G — Program Test Data Sets
- Appendix H — Curriculum Vitae's

Forms

- Form 1 — Load and Run Program
- Form 2 — Primary Window Navigation
- Form 3 — Program Strategy / Python Code Verification
- Form 4 — Test Data Agreement
- Form 5 — Problem / Request Report

1.0 PURPOSE

This Validation Protocol defines the approach, responsibilities, and acceptance criteria for validating the PyCuDAL (Content Uniformity and Dissolution Acceptance Limits) Python application. The purpose of this validation is to provide documented evidence that the Program consistently produces accurate, reliable, and reproducible results when used as intended, in accordance with applicable USP <905> Content Uniformity and USP <711> Dissolution requirements.

2.0 SCOPE

This protocol applies to the PyCuDAL Python application, including the Content Uniformity (Sampling Plan 1 and Sampling Plan 2) and Dissolution (Sampling Plan 1 and Sampling Plan 2) acceptance-limit calculation modules described in Appendix F.

Validation activities under this protocol cover: installation and startup verification, primary window/function navigation, review of the program's calculation strategy against its Python code implementation, and agreement between program output and predefined test data sets.

This protocol does not cover the underlying USP <905>/<711> statistical methodology itself, network or IT infrastructure qualification, or end-user training, which are addressed by separate documentation.

3.0 REFERENCES

- USP General Chapter <905> — Uniformity of Dosage Units
- USP General Chapter <711> — Dissolution
- CUSP2 Acceptance-Limit Discrepancy Investigation (Technical Note)
- Program source code listing (Appendix A)

4.0 DEFINITIONS AND ABBREVIATIONS

Term	Definition
PyCuDAL	Python – Content Uniformity and Dissolution Acceptance Limits (the Program)
USP	United States Pharmacopeia
CU	Content Uniformity (USP <905>)
AV	Acceptance Value, as defined in USP <905>
SE	Within-location standard deviation (CUSP2 / DISP2)
SM	Between-location standard deviation (CUSP2 / DISP2)
MEANL / MEANU	Lower / upper acceptable bound on the sample mean
LBOUND	Required lower bound on the probability of passing the USP test
CI	Confidence level / confidence interval

5.0 RESPONSIBILITIES

Role	Responsibility
Validation Lead / Author	Prepares this protocol, coordinates execution, and compiles the validation summary report.
Performer / Tester	Executes the test forms in this protocol and records results, evidence, and any deviations.
Reviewer	Reviews completed test forms and supporting documentation for completeness, accuracy, and compliance.
Approver	Approves this protocol prior to execution and approves the final validation summary upon completion.
System Owner / SME	Provides subject-matter support on program functionality and resolves technical deviations.

6.0 SYSTEM DESCRIPTION

PyCuDAL is a Python application that calculates parametric (tolerance-interval) acceptance limits for USP <905> Content Uniformity and USP <711> Dissolution testing. It is a translation of a legacy SAS/AF application of the same name. A complete description of the Program's modules, functions, and intended use is provided in Appendix F.

7.0 VALIDATION APPROACH

Validation will be performed using a combination of: installation and startup verification (Form 1); navigation checks across the Program's primary windows/functions (Form 2); review of the Program's calculation strategy against its Python code implementation (Form 3); and comparison of Program output against the predefined test data sets in Appendix G (Form 4). Any discrepancy identified during execution will be documented and evaluated per Section 9 using Form 5.

8.0 ACCEPTANCE CRITERIA

Validation is considered successful when all of the following are met:

- The Program loads and runs without error (Form 1).
- All primary windows/functions are accessible and navigate as intended (Form 2).
- The Program's calculation strategy is confirmed to be correctly reflected in the Python code implementation (Form 3).
- Program output agrees with the predefined test data sets in Appendix G, within the acceptance tolerance defined for each test case (Form 4).
- All test forms are completed, reviewed, and approved, and any deviations are closed prior to final approval of this protocol.

9.0 DEVIATION HANDLING

Any unexpected result, discrepancy, or issue identified during execution of this protocol shall be documented on Form 5 (Problem/Request Report). Each deviation shall be assessed for impact on validation status, assigned a severity, and tracked to resolution. Deviations must be closed and approved prior to final protocol closure.

10.0 DOCUMENTATION AND RETENTION

All completed forms, supporting data, and this protocol shall be retained. The completed protocol package, together with its appendices and forms, constitutes the validation record for the Program.

APPENDIX A — PROGRAM LISTING

pycudal/cudal/core.py

```

1 """
2 cudal.core
3 =====
4
5 Shared statistical building blocks for the CuDAL (Content Uniformity and
6 Dissolution Acceptance Limits) system, translated from the original SAS
7 macros (CuDAL.sas, Cusp1.sas, Cusp2.sas, Displ.sas, Disp2.sas).
8
9 SAS -> Python function mapping
10 -----
11     PROBNORM(x)          -> norm.cdf(x)
12     PROBIT(p)           -> norm.ppf(p)
13     PROBCHI(x, df)      -> chi2.cdf(x, df)
14     CINV(p, df)         -> chi2.ppf(p, df)
15
16 Two probability "cores" appear repeatedly in the SAS code, always with the
17 exact same formulas (only the values fed into them differ):
18
19 * ``c1calc`` (cusp1.sas) / ``cullu`` + ``cuulu`` (Cusp2.sas)
20   -> :func:`content_uniformity_bound` here.
21 * ``COMPUTE`` (Displ.sas / Disp2.sas -- byte-for-byte identical formula
22   in both files)
23   -> :func:`dissolution_bound` here.
24
25 Both are implemented as numpy ufunc-style, broadcastable functions:
26 ``mu``/``sigma`` (or ``llu``/``sigma``) may be scalars, or arrays of any
27 mutually-broadcastable shape, and the result has the broadcast shape.
28 This lets every acceptance-limit search evaluate an entire grid (e.g. all
29 300 rows of a MEAN table, or an 80x80 SE/SM grid) in a handful of numpy
30 calls instead of one Python-level call per grid point.
31
32 Performance notes
33 -----
34 This module deliberately avoids two patterns that make the equivalent
35 SAS code (and a naive line-by-line port of it) slow:
36
37 1. Per-point root finding. The SAS code steps a candidate value
38    (a standard deviation, or a mean) in tiny fixed increments, one
39    value at a time, until a probability threshold is crossed. Doing
40    this with a Python ``for`` loop -- even calling a fast vectorized
41    probability function inside it -- pays the interpreter-loop
42    overhead once per grid point. :func:`batched_root_find` instead
43    solves *all* grid points' roots simultaneously with a vectorized
44    bisection: every iteration is a single numpy expression over the
45    whole grid, not a Python loop over the grid.
46
47 2. Scalar broadcasting. :func:`content_uniformity_bound` and
48    :func:`dissolution_bound` accept arrays directly, so a table with
49    ``M`` mean values, or a grid with ``G = len(SE) * len(SM)`` cells,
50    is evaluated as one ``M``- or ``G``-length numpy computation rather
51    than ``M`` (or ``G``) separate scalar calls.
52
53 Together, building e.g. the CUSP2 acceptance-limit table for an 80x80
54 SE/SM grid drops from tens of thousands of scalar probability
55 evaluations to a few dozen vectorized ones.
56 """
57

```

```

58 from __future__ import annotations
59
60 from typing import Callable, Tuple
61
62 import numpy as np
63 from scipy.special import gammainc, gammaincinv, ndtr, ndtri
64
65 __all__ = [
66     "probnorm",
67     "probit",
68     "probchi",
69     "cinv",
70     "sas_do_range",
71     "content_uniformity_bound",
72     "dissolution_bound",
73     "batched_root_find",
74     "batched_two_sided_bounds",
75 ]
76
77
78 # -----
79 # Direct SAS function equivalents (already numpy ufuncs -> broadcast for free)
80 # -----
81
82
83 def probnorm(x):
84     """
85     SAS PROBNORM(x): standard normal CDF.
86
87     Implemented directly via ``scipy.special.ndtr`` rather than
88     ``scipy.stats.norm.cdf`` -- numerically identical, but avoids the
89     ~5-7x overhead of the generic frozen-distribution machinery, which
90     matters a lot here since this is called on arrays with millions of
91     elements during the acceptance-limit table searches.
92     """
93     return ndtr(x)
94
95
96 def probit(p):
97     """SAS PROBIT(p): standard normal inverse CDF (quantile function)."""
98     return ndtri(p)
99
100
101 def probchi(x, df):
102     """
103     SAS PROBCHI(x, df): chi-square CDF with `df` degrees of freedom.
104
105     Implemented via the regularized lower incomplete gamma function
106     (``chi2.cdf(x, df) == gammainc(df/2, x/2)``) rather than
107     ``scipy.stats.chi2.cdf`` -- numerically identical, ~2.5x faster.
108     """
109     return gammainc(df / 2.0, x / 2.0)
110
111
112 def cinv(p, df):
113     """
114     SAS CINV(p, df): chi-square inverse CDF (quantile function).
115
116     ``chi2.ppf(p, df) == 2 * gammaincinv(df/2, p)``.
117     """
118     return 2.0 * gammaincinv(df / 2.0, p)
119
120
121 def sas_do_range(start: float, stop: float, step: float) -> np.ndarray:

```

```

122     """
123     Reproduce the index values visited by a SAS ``DO start TO stop BY step;``
124     loop (inclusive of `stop`, subject to floating point rounding the same
125     way SAS handles it in practice for these fixed step sizes).
126     """
127     if step == 0:
128         raise ValueError("step must be non-zero")
129     n = int(round((stop - start) / step)) + 1
130     if n <= 0:
131         return np.array([])
132     return start + step * np.arange(n)
133
134
135# -----
136# Content uniformity core (clcalc / cullu / cuulu in the SAS source)
137# -----
138
139
140def _cu_stage_prob(mu: np.ndarray, sigma: np.ndarray, E, n: int, k: float) -
> np.ndarray:
141     """
142     One "stage" of the content-uniformity probability calculation
143     (n=10, k=2.4 for stage 1; n=30, k=2.0 for stage 2), reproducing the
144     int1/int2/int3 (or iint1/iint2/iint3) block of the SAS macro exactly.
145
146     ``mu``, ``sigma`` and ``E`` may be scalars or arrays of a common
147     broadcast shape; the result has that broadcast shape.
148     """
149     h = 0.05
150     L1 = 15.0
151
152     mu = np.asarray(mu, dtype=float)
153     sigma = np.asarray(sigma, dtype=float)
154     E = np.asarray(E, dtype=float)
155
156     z1 = (E - mu) * np.sqrt(n) / sigma
157     z2 = (98.5 - mu) * np.sqrt(n) / sigma
158     chi_a = probchi((n - 1) * L1**2 / (k * sigma) ** 2, n - 1)
159     int1 = (probnorm(z1) - probnorm(z2)) * chi_a
160
161     # Broadcast the fixed integration grid against mu/sigma/E by adding a
162     # trailing axis to the (broadcast) parameter arrays and summing over it.
163     mu_e, sigma_e, E_e = np.broadcast_arrays(mu, sigma, E)
164     mu_e = mu_e[..., None]
165     sigma_e = sigma_e[..., None]
166     E_e = E_e[..., None]
167
168     # int2 : do x = E to (E + 15 - h) by h; (E varies per element, so we
169     # build the grid relative to E_e directly rather than via sas_do_range)
170     n_steps = int(round(15 / h)) # number of h-steps spanning the 15-unit window
171     j = np.arange(n_steps) # 0 .. n_steps-1
172     xs = E_e + j * h
173     x1 = (xs - mu_e) * np.sqrt(n) / sigma_e
174     x2 = (xs + h - mu_e) * np.sqrt(n) / sigma_e
175     chi_args = (n - 1) * (E_e + 15 - xs - h / 2) ** 2 / (k * sigma_e) ** 2
176     int2 = np.sum((probnorm(x2) - probnorm(x1)) * probchi(chi_args, n - 1), axis=
-1)
177
178     # int3 : do x = (98.5 - 15) to (98.5 - h) by h; (fixed grid, no E dependenc
e)
179     xs3 = sas_do_range(98.5 - 15, 98.5 - h, h)
180     x1b = (xs3 - mu_e) * np.sqrt(n) / sigma_e
181     x2b = (xs3 + h - mu_e) * np.sqrt(n) / sigma_e
182     chi_args3 = (n - 1) * (15 - 98.5 + xs3 + h / 2) ** 2 / (k * sigma_e) ** 2

```



```

183     int3 = np.sum((probnorm(x2b) - probnorm(x1b)) * probchi(chi_args3, n - 1), ax
is=-1)
184
185     return int1 + int2 + int3
186
187
188 def content_uniformity_bound(mu, sigma, target):
189     """
190     Replicates the ``c1calc`` / ``cullu`` / ``cuulu`` SAS macro.
191
192     Estimated probability that a batch with true (population) mean ``mu``
193     and true standard deviation ``sigma`` passes the USP <905> Content
194     Uniformity test, for a label claim / target of ``target`` (%).
195
196     Fully broadcastable: ``mu``, ``sigma``, ``target`` may be Python
197     scalars or numpy arrays of a common broadcast shape (e.g. evaluate an
198     entire table of candidate means in one call). The return value has
199     that broadcast shape (a plain float is returned for scalar inputs).
200
201     Parameters
202     -----
203     mu : float or array_like
204         True population mean (% of label claim), e.g. LLU or ULU.
205     sigma : float or array_like
206         True population standard deviation (% of label claim).
207     target : float or array_like
208         Target / label claim value used by the USP test (usually 100).
209
210     Returns
211     -----
212     float or numpy.ndarray
213         The "OVERBD" value: max(P1, P2), where P1 is the stage-1 (n=10)
214         pass probability and P2 combines stage-2 (n=30) with an
215         additional 30-unit compound tail term.
216     """
217     mu = np.asarray(mu, dtype=float)
218     sigma = np.asarray(sigma, dtype=float)
219     target = np.asarray(target, dtype=float)
220     E = np.where(target <= 101.5, 101.5, target)
221
222     # ---- stage 1 (n=10, k=2.4) -> P1 -----
223     P1 = _cu_stage_prob(mu, sigma, E, n=10, k=2.4)
224
225     # ---- stage 2 (n=30, k=2.0) -> P2a + tail term P2b -----
226     P2a = _cu_stage_prob(mu, sigma, E, n=30, k=2.0)
227
228     zzz1 = (123.125 - mu) / sigma
229     zzz2 = np.where(target <= 101.5, (101.5 - 24.625 - mu) / sigma, (target - 24.
625 - mu) / sigma)
230     P2b = (probnorm(zzz1) - probnorm(zzz2)) ** 30
231     P2 = np.maximum(0.0, P2a + P2b - 1)
232
233     result = np.maximum(P1, P2)
234     return result if result.ndim else result.item()
235
236
237 # -----
238 # Dissolution core (COMPUTE macro in Displ.sas / Disp2.sas -- identical)
239 # -----
240
241
242 def dissolution_bound(llu, sigma):
243     """
244     Replicates the ``COMPUTE`` SAS macro (identical in Displ.sas and

```

```

245 Disp2.sas).
246
247 Estimated probability of passing the USP <711> Dissolution test
248 (stage 1: n=6, stage 2: n=12, stage 3: n=24), given a lower adjusted
249 bound on the population mean (`llu`) and population standard
250 deviation `sigma`. Fully broadcastable, like
251 :func:`content_uniformity_bound`.
252
253 Returns
254 -----
255 float or numpy.ndarray
256     The "OVERBD" value: max(F1, F2, F3) across the three USP <711>
257     stages.
258 """
259 llu = np.asarray(llu, dtype=float)
260 sigma = np.asarray(sigma, dtype=float)
261
262 F1 = (1 - probnorm((5 - llu) / sigma)) ** 6
263
264 sn2 = np.sqrt(12)
265 pm2 = probnorm(sn2 * (-llu) / sigma)
266 pb2 = 1 - probnorm((-15 - llu) / sigma)
267 F2 = pb2**12 - pm2
268
269 sn3 = np.sqrt(24)
270 pm3 = probnorm(sn3 * (-llu) / sigma)
271 p2 = probnorm((-15 - llu) / sigma) - probnorm((-25 - llu) / sigma)
272 p3 = 1 - probnorm((-15 - llu) / sigma)
273 F3 = p3**24 + 24 * p2 * p3**23 + 276 * p2**2 * p3**22 - pm3
274
275 result = np.maximum(np.maximum(F1, F2), F3)
276 return result if result.ndim else result.item()
277
278
279# -----
280# Vectorized batch root finding -- replaces per-point brentq/GOTO search
281# -----
282
283
284def batched_root_find(
285    func: Callable[[np.ndarray], np.ndarray],
286    lo,
287    hi,
288    scan_points: int = 12,
289    bisect_iters: int = 22,
290    which: str = "first",
291) -> Tuple[np.ndarray, np.ndarray]:
292    """
293    Solve ``func(x) == 0`` independently for every element of a grid, in
294    one vectorized pass, instead of one root-find per grid point.
295
296    ``func`` must accept an array of shape ``(S, 1)`` (or any shape that
297    broadcasts against the grid's own parameter arrays baked into the
298    closure) and return an array of shape ``(S, G)`` , and must also
299    accept a shape-``(G,)`` array (during bisection refinement) and
300    return shape ``(G,)``. This is automatic for anything built from
301    :func:`content_uniformity_bound` / :func:`dissolution_bound` plus
302    elementwise arithmetic -- see cudal.cusp1/cusp2/disp1/disp2 for the
303    closures passed in.
304
305    Parameters
306    -----
307    func : callable
308        Elementwise function; ``func(x)`` broadcasts ``x`` against the

```

```

309     grid parameters captured in the closure.
310     lo, hi : float
311         Scalar bounds of the search interval (the same interval is
312         scanned for every grid point -- pass a wide-enough range).
313     scan_points : int
314         Number of points used to bracket the root before refining. The
315         function is assumed to have exactly one sign crossing of
316         interest between ``lo`` and ``hi`` at this resolution (this
317         mirrors the same monotonicity assumption the original SAS
318         stepped search relies on).
319     bisect_iters : int
320         Number of vectorized bisection halving steps used to refine each
321         bracket. The SAS code this replaces used a fixed step of
322         0.001-0.2 depending on macro; matching that resolution needs on
323         the order of  $\log_2((hi-lo)/step) \approx 13-20$  iterations, so the
324         default of 22 already has comfortable headroom while staying
325         cheap (each extra iteration re-evaluates the full probability
326         integral for the whole grid).
327     which : {"first", "last"}
328         If the scanned curve has more than one sign crossing (e.g. the
329         content-uniformity bound is bump-shaped in the mean), "first"
330         finds the lowest-x crossing and "last" finds the highest-x one.
331         This mirrors the SAS code's two opposite-direction searches
332         (scan up from the low end vs. scan down from the high end).
333
334     Returns
335     -----
336     (root, found) : (numpy.ndarray, numpy.ndarray[bool])
337         ``root`` has shape ``(G,)`` -- the last scanned/bisected value is
338         returned even where no sign change was found (caller decides how
339         to interpret that; ``found`` flags which entries actually
340         bracketed a root).
341     """
342     xs = np.linspace(lo, hi, scan_points) # shape (S,)
343     xs_col = xs[:, None] # (S, 1) -- broadcasts against the grid inside func
344     vals = np.asarray(func(xs_col)) # shape (S, G)
345
346     sign = np.sign(vals)
347     changes = np.diff(sign, axis=0) != 0 # (S-1, G)
348     found = changes.any(axis=0)
349
350     if which == "first":
351         idx = np.argmax(changes, axis=0)
352     else:
353         idx = changes.shape[0] - 1 - np.argmax(changes[::-1, :], axis=0)
354
355     g = vals.shape[-1]
356     cols = np.arange(g)
357     lo_x = np.where(found, xs[idx], lo)
358     hi_x = np.where(found, xs[idx + 1], hi)
359     lo_val = np.where(found, vals[idx, cols], vals[0, cols])
360
361     # Vectorized bisection refinement of every bracket simultaneously.
362     cur_lo, cur_hi, cur_lo_val = lo_x.copy(), hi_x.copy(), lo_val.copy()
363     for _ in range(bisect_iters):
364         mid = (cur_lo + cur_hi) / 2.0
365         f_mid = np.asarray(func(mid))
366         same_sign_as_lo = np.sign(f_mid) == np.sign(cur_lo_val)
367         cur_lo = np.where(same_sign_as_lo, mid, cur_lo)
368         cur_hi = np.where(same_sign_as_lo, cur_hi, mid)
369         cur_lo_val = np.where(same_sign_as_lo, f_mid, cur_lo_val)
370
371     root = (cur_lo + cur_hi) / 2.0
372     return root, found

```

```

373
374
375def batched_two_sided_bounds(
376    func_lower: Callable[[np.ndarray], np.ndarray],
377    func_upper: Callable[[np.ndarray], np.ndarray],
378    lo,
379    hi,
380    scan_points: int = 24,
381    bisect_iters: int = 22,
382):
383    """
384    Convenience wrapper for the CUSP2-style pattern of two searches over
385    the same grid: the lower boundary found scanning low->high (first
386    crossing) and the upper boundary found scanning high->low (last
387    crossing). See :func:`batched_root_find`.
388
389    Returns
390    -----
391    (lower, lower_found, upper, upper_found)
392    """
393    lower, lower_found = batched_root_find(
394        func_lower, lo, hi, scan_points, bisect_iters, which="first"
395    )
396    upper, upper_found = batched_root_find(
397        func_upper, lo, hi, scan_points, bisect_iters, which="last"
398    )
399    return lower, lower_found, upper, upper_found
400

```

pycudal/cudal/cusp1.py

```

1 """
2 cudal.cusp1
3 =====
4
5 Translation of ``cusp1.sas`` -- Content Uniformity, Sampling Plan 1
6 (single location / composite sample, USP <905>).
7
8 Three analyses, matching the three SAS entry points:
9
10 * :func:`acceptance_limit_table` (``CALCUSP1`` / ``PRTCUSP1``, A1CUSP1=Y)
11 * :func:`probability_of_passing` (``EVCUSP1``, A2CUSP1=Y)
12 * :func:`sample_probability` (``SMPCUSP1``, A3CUSP1=Y)
13
14 Performance
15 -----
16 ``acceptance_limit_table`` finds, for every candidate MEAN in the grid,
17 the boundary standard deviation where the pass-probability crosses the
18 required bound. Rather than looping over means one at a time (each doing
19 its own root search), all means are solved *simultaneously* with
20 :func:`cudal.core.batched_root_find` -- a handful of vectorized numpy
21 calls covering the whole table instead of hundreds of individual ones.
22
23 ``probability_of_passing`` vectorizes across the full (U, CV) grid at
24 once using broadcasting, rather than a Python loop over each combination.
25 """
26
27 from __future__ import annotations
28
29 import numpy as np
30 import pandas as pd
31
32 from .core import batched_root_find, cinv, content_uniformity_bound, probchi, prob
33 it, probnorm
34
35 def acceptance_limit_table(
36     number: int,
37     target: float,
38     lbound: float,
39     cilevel: float,
40     mean_low: float = 85.1,
41     mean_high: float = 114.9,
42     mean_step: float = 0.1,
43 ) -> pd.DataFrame:
44     """
45     SAS: ``%CALCUSP1`` (called when A1CUSP1=Y or A2CUSP1=Y).
46
47     For each candidate sample MEAN, finds the largest sample standard
48     deviation (expressed as CV%) for which the probability of passing
49     USP <905> is still at least ``lbound`` (given as a percentage, e.g.
50     95 for 95%) with ``cilevel`` confidence, for ``number`` (N) units
51     and a label claim ``target``.
52
53     The whole MEAN grid is solved in one vectorized batch (see module
54     docstring) rather than one root-search per mean.
55
56     Returns
57     -----
58     pandas.DataFrame with columns ["MEAN", "CV"].
59     A CV of 0.0 marks a mean where even a (near) zero standard
60     deviation batch would not meet the bound -- SAS's "CV = 0" branch.
61     """

```

```

62 z = probit((1 + np.sqrt(cilevel / 100)) / 2)
63 n = number
64 chi = cinv(1 - np.sqrt(cilevel / 100), n - 1)
65 target_prob = lbound / 100
66
67 means = np.round(np.arange(mean_low, mean_high + mean_step / 2, mean_step), 6)
68
69 def overbd_of_sd(sampsd):
70     # sampsd broadcasts against `means` (row vector (S,1) x (M,) -> (S,M),
71     # or shape (M,) during the bisection refinement stage).
72     sigma = np.sqrt((n - 1) * sampsd**2 / chi)
73     ll_u = means - z * sigma / np.sqrt(n)
74     ul_u = means + z * sigma / np.sqrt(n)
75     overlbd = content_uniformity_bound(ll_u, sigma, target)
76     overubd = content_uniformity_bound(ul_u, sigma, target)
77     return np.minimum(overlbd, overubd) - target_prob
78
79 sd_lo, sd_hi = 0.01, 7.8
80 root, found = batched_root_find(overbd_of_sd, sd_lo, sd_hi)
81
82 floor_fail = overbd_of_sd(np.full_like(means, sd_lo)) < 0 # even minimal SD f
ails -> CV = 0
83 sd = np.where(floor_fail, sd_lo, np.where(found, root, sd_hi))
84 cv = np.where(floor_fail, 0.0, 100 * sd / means)
85
86 return pd.DataFrame({"MEAN": means, "CV": cv})
87
88
89 def probability_of_passing(
90     table: pd.DataFrame,
91     number: int,
92     u_values,
93     cv_values,
94 ) -> pd.DataFrame:
95     """
96     SAS: ``%EVCUSP1`` / ``%SIGCUSP1`` (A2CUSP1=Y).
97
98     Given the acceptance-limit ``table`` from :func:`acceptance_limit_table`
99     (columns MEAN, CV), evaluate -- for every combination of assumed true
100     population mean ``U`` and true population CV -- the probability that a
101     sample of size ``number`` drawn from that population lands inside the
102     acceptance region traced out by the table (a trapezoid-style sum over
103     each pair of adjacent MEAN/CV/STD table entries, exactly reproducing
104     the running ``PTRAP + PT`` accumulation in ``%SIGCUSP1``).
105
106     The full (row, U, CV) computation is done as one broadcasted numpy
107     array rather than a Python loop over (U, CV) pairs.
108
109     Returns
110     -----
111     pandas.DataFrame with columns ["U", "CV", "PTRAP"] -- PTRAP is the
112     estimated probability of passing.
113     """
114     t = table.sort_values("MEAN").reset_index(drop=True)
115     x = t["MEAN"].to_numpy()
116     std = (t["MEAN"] * t["CV"] / 100).to_numpy()
117     n = number
118
119     u = np.asarray(u_values, dtype=float)[None, :, None] # (1, U, 1)
120     cv = np.asarray(cv_values, dtype=float)[None, None, :] # (1, 1, CV)
121     sigma = u * cv / 100 # (1, U, CV)
122
123     x_hi = x[1:, None, None] # (R-1, 1, 1)
124     x_lo = x[:-1, None, None]

```

```

125     std_hi = std[1:, None, None]
126     std_lo = std[:-1, None, None]
127
128     pmean = probnorm((x_hi - u) * np.sqrt(n) / sigma) - probnorm((x_lo - u) * np.
sqrt(n) / sigma)
129     aveht = (std_hi + std_lo) / 2
130     pstd = probchi((n - 1) * aveht**2 / sigma**2, n - 1)
131     ptrap = np.sum(pmean * pstd, axis=0) # (U, CV)
132
133     u_flat = np.asarray(u_values, dtype=float)
134     cv_flat = np.asarray(cv_values, dtype=float)
135     uu, cc = np.meshgrid(u_flat, cv_flat, indexing="ij")
136     return pd.DataFrame({"U": uu.ravel(), "CV": cc.ravel(), "PTRAP": ptrap.ravel(
)})
137
138
139 def sample_probability(
140     mean: float,
141     cv: float,
142     number: int,
143     target: float,
144     lbound: float,
145     cilevel: float,
146 ) -> dict:
147     """
148     SAS: ``%SMPCUSP1`` (A3CUSP1=Y).
149
150     Given an observed sample ``mean`` (%) claim) and ``cv`` (%), directly
151     computes the sample standard deviation and the probability
152     ("OVERBD") that future samples from that population will pass
153     USP <905>, for ``number`` units, label claim ``target``, at
154     ``cilevel``% confidence with the requested ``lbound``% lower bound
155     (lbound/cilevel are only used to report context; OVERBD itself does
156     not depend on lbound, mirroring the SAS code).
157
158     Returns
159     -----
160     dict with keys: MEAN, CV, SAMPSD, OVERBD
161     """
162     z = probit((1 + np.sqrt(cilevel / 100)) / 2)
163     n = number
164     chi = cinv(1 - np.sqrt(cilevel / 100), n - 1)
165
166     sampsd = mean * cv / 100
167     sigma = np.sqrt((n - 1) * sampsd**2 / chi)
168     ll_u = mean - z * sigma / np.sqrt(n)
169     ul_u = mean + z * sigma / np.sqrt(n)
170
171     overlbd = content_uniformity_bound(ll_u, sigma, target)
172     overubd = content_uniformity_bound(ul_u, sigma, target)
173     overbd = min(overlbd, overubd)
174
175     return {"MEAN": mean, "CV": cv, "SAMPSD": sampsd, "OVERBD": overbd}
176

```

pycudal/cudal/cusp2.py

```

1 """
2 cudal.cusp2
3 =====
4
5 Translation of ``Cusp2.sas`` -- Content Uniformity, Sampling Plan 2
6 (multiple locations, each with its own composite sample -- a
7 within-location / between-location variance-components model, USP <905>).
8
9 Three analyses, matching the three SAS entry points:
10
11 * :func:`acceptance_limit_table`  (``CALCUSP2``, A1CUSP2=Y)
12 * :func:`probability_of_passing`  (``EVCUSP2`` / ``SIGCUSP2``, A2CUSP2=Y)
13 * :func:`sample_probability`      (``SMPCUSP2``, A3CUSP2=Y)
14
15 Variance-component notation (mirrors the SAS variable names)
16 -----
17 NN  -- number of units assayed per location (SAS &NUM)
18 L   -- number of locations (SAS &LOC)
19 N   = NN * L
20 SE  -- pooled within-location standard deviation
21 SM  -- between-location standard deviation
22 VAR -- total variance used as sigma^2 in the content-uniformity core
23 MVAR -- upper confidence bound on the between-location variance
24        component, used (via sqrt(MVAR/N)) as the standard error of
25        the overall mean for the MEANL/MEANU confidence bound
26
27 Performance
28 -----
29 The original SAS code (and a naive Python port) loops over every (SE, SM)
30 grid cell and, for each one, scans/steps a candidate MEAN up from below
31 (for MEANL) and down from above (for MEANU) one value at a time. Here the
32 *entire* SE/SM grid is flattened to a single array of length
33 ``G = len(SE) * len(SM)`` , and both boundaries for all G cells are solved
34 in one vectorized pass with :func:`cudal.core.batched_two_sided_bounds`
35 -- turning what would be thousands of scalar root searches (each costing
36 hundreds of probability evaluations) into a couple dozen array-wide
37 evaluations.
38 """
39
40 from __future__ import annotations
41
42 import numpy as np
43 import pandas as pd
44
45 from .core import (
46     batched_two_sided_bounds,
47     cinv,
48     content_uniformity_bound,
49     probchi,
50     probit,
51     probnorm,
52 )
53
54
55 def _variance_components(se, sm, nn: int, l: int, chierr: float, chiloc: float):
56     """
57     Shared VAR / MVAR calculation used by all three CUSP2 analyses.
58     Works elementwise on scalars or numpy arrays of any shape.
59     """
60     se2 = se * se
61     h2 = 1 * (nn - 1) / chierr - 1
62     sec = ((1 - 1 / nn) * h2 * se2) ** 2

```



```

63
64     sl2 = sm * sm * nn
65     sl2ub = (1 - 1) * sl2 / chiloc
66     h1 = (1 - 1) / chiloc - 1
67     first = ((1 / nn) * h1 * sl2) ** 2
68
69     ptest = (1 / nn) * sl2 + (1 - 1 / nn) * se2
70     var = ptest + np.sqrt(first + sec)
71     mvar = sl2ub
72     return var, mvar
73
74
75 def acceptance_limit_table(
76     num: int,
77     loc: int,
78     target: float,
79     lbound: float,
80     cilevel: float,
81     se_values,
82     sm_values,
83     mean_search_range=(80.0, 120.0),
84 ) -> pd.DataFrame:
85     """
86     SAS: ``%CALCUSP2`` (A1CUSP2=Y).
87
88     Parameters
89     -----
90     num : int
91         Units per location (SAS &NUM).
92     loc : int
93         Number of locations (SAS &LOC).
94     target, lbound, cilevel : float
95         Label claim / target, required lower bound (%), confidence
96         level (%) -- same meaning as in :mod:`cudal.cusp1`.
97     se_values, sm_values : iterable of float
98         Grids of within-location (SE) and between-location (SM) standard
99         deviations to evaluate (SAS steps these by D1=0.10).
100     mean_search_range : (float, float)
101         Range to scan for the MEANL/MEANU roots.
102
103     Returns
104     -----
105     pandas.DataFrame with columns SE, SM, MEANL, MEANU (NaN where no
106     acceptable mean range exists for that SE/SM combination -- i.e. the
107     combined variability is too large to ever pass).
108     """
109     nn, l = num, loc
110     n = nn * l
111     z = probit((1 + np.sqrt(cilevel / 100)) / 2)
112     chierr = cinv(1 - np.sqrt(cilevel / 100), 1 * (nn - 1))
113     chiloc = cinv(1 - np.sqrt(cilevel / 100), 1 - 1)
114     target_prob = lbound / 100
115     lo, hi = mean_search_range
116
117     se_grid, sm_grid = np.meshgrid(
118         np.asarray(se_values, dtype=float), np.asarray(sm_values, dtype=float), i
119         ndexing="ij"
120     )
121     se_flat = se_grid.ravel()
122     sm_flat = sm_grid.ravel()
123
124     var, mvar = _variance_components(se_flat, sm_flat, nn, l, chierr, chiloc)
125     sigma = np.sqrt(var) # shape (G,)
126     se_of_mean = np.sqrt(mvar / n) # shape (G,)

```

```

126
127     def func_lower(mean):
128         ll_u = mean - z * se_of_mean
129         return content_uniformity_bound(ll_u, sigma, target) - target_prob
130
131     def func_upper(mean):
132         ul_u = mean + z * se_of_mean
133         return content_uniformity_bound(ul_u, sigma, target) - target_prob
134
135     # Bump scan_points/bisect_iters to avoid crossing of overbd_of_sd(sd)
136     meanl, meanl_found, meanu, meanu_found = batched_two_sided_bounds(
137         func_lower,
138         func_upper,
139         lo,
140         hi,
141         scan_points=200,
142         bisect_iters=30,
143     )
144
145     ok = meanl_found & meanu_found & (meanu > meanl)
146     meanl_out = np.where(ok, meanl, np.nan)
147     meanu_out = np.where(ok, meanu, np.nan)
148
149     return pd.DataFrame({"SE": se_flat, "SM": sm_flat, "MEANL": meanl_out, "MEANU": meanu_out})
150
151
152 def probability_of_passing(
153     table: pd.DataFrame,
154     num: int,
155     loc: int,
156     dl: float,
157     u_values,
158     sigse_values,
159     sigsm_values,
160 ) -> pd.DataFrame:
161     """
162     SAS: ``%EVCUSP2`` / ``%SIGCUSP2`` (A2CUSP2=Y).
163
164     Given the acceptance-limit ``table`` (columns SE, SM, MEANL, MEANU),
165     sums, over every (SE, SM) grid cell, the probability that a batch with
166     assumed true mean ``U`` and true within-/between-location standard
167     deviations (``sigse``, ``sigsm``) both (a) lands its location means in
168     [MEANL, MEANU], (b) has a within-location SD landing in the SE bin,
169     and (c) has a between-location SD landing in the SM bin.
170
171     The full (table row x U x SIGSE x SIGSM) computation is done as one
172     broadcasted numpy array rather than nested Python loops.
173
174     Returns
175     -----
176     pandas.DataFrame with columns ["U", "SIGSE", "SIGSM", "PSUM"].
177     """
178     t = table.dropna(subset=["MEANL", "MEANU"])
179     nn, l = num, loc
180     n = nn * l
181
182     se = t["SE"].to_numpy()[ :, None, None, None]
183     sm = t["SM"].to_numpy()[ :, None, None, None]
184     meanl = t["MEANL"].to_numpy()[ :, None, None, None]
185     meanu = t["MEANU"].to_numpy()[ :, None, None, None]
186
187     u = np.asarray(u_values, dtype=float)[None, :, None, None]
188     sigse = np.asarray(sigse_values, dtype=float)[None, None, :, None]

```

```

189 sigsm = np.asarray(sigsm_values, dtype=float)[None, None, None, :]
190
191 expse2 = sigse**2
192 expsm2 = expse2 + nn * sigsm**2
193
194 pmean = probnorm((meanu - u) * np.sqrt(n / expsm2)) - probnorm(
195     (meanl - u) * np.sqrt(n / expsm2)
196 )
197 pse = probchi(1 * (nn - 1) * se**2 / expse2, 1 * (nn - 1)) - probchi(
198     1 * (nn - 1) * (se - dl) ** 2 / expse2, 1 * (nn - 1)
199 )
200 psm = probchi((1 - 1) * nn * sm**2 / expsm2, 1 - 1) - probchi(
201     (1 - 1) * nn * (sm - dl) ** 2 / expsm2, 1 - 1
202 )
203
204 psum = np.sum(pmean * pse * psm, axis=0) # (U, SIGSE, SIGSM)
205
206 uu, ee, mm = np.meshgrid(
207     np.asarray(u_values, dtype=float),
208     np.asarray(sigse_values, dtype=float),
209     np.asarray(sigsm_values, dtype=float),
210     indexing="ij",
211 )
212 return pd.DataFrame(
213     {
214         "U": uu.ravel(),
215         "SIGSE": ee.ravel(),
216         "SIGSM": mm.ravel(),
217         "PSUM": psum.ravel(),
218     }
219 )
220
221
222 def sample_probability(
223     mean: float,
224     se: float,
225     sm: float,
226     num: int,
227     loc: int,
228     target: float,
229     cilevel: float,
230 ) -> dict:
231     """
232     SAS: ``%SMPCUSP2`` (A3CUSP2=Y).
233
234     Given an observed sample MEAN, within-location SD (SE) and
235     between-location SD (SM), computes the probability ("OVERBD") that
236     future samples from that population pass USP <905>.
237     """
238     nn, l = num, loc
239     n = nn * l
240     z = probit((1 + np.sqrt(cilevel / 100)) / 2)
241     chierr = cinv(1 - np.sqrt(cilevel / 100), 1 * (nn - 1))
242     chiloc = cinv(1 - np.sqrt(cilevel / 100), 1 - 1)
243
244     var, mvar = _variance_components(se, sm, nn, l, chierr, chiloc)
245     sigma = np.sqrt(var)
246     se_of_mean = np.sqrt(mvar / n)
247
248     llm = mean - z * se_of_mean
249     ulm = mean + z * se_of_mean
250
251     overbdl = content_uniformity_bound(llm, sigma, target)
252     overbdu = content_uniformity_bound(ulm, sigma, target)

```

```
253     overbd = min(overbdl, overbdu)
254
255     return {
256         "MEAN": mean,
257         "SE": se,
258         "SM": sm,
259         "VAR": var,
260         "MVAR": mvar,
261         "SIGMA": sigma,
262         "OVERBDL": overbdl,
263         "OVERBDU": overbdu,
264         "OVERBD": overbd,
265     }
```

pycudal/cudal/disp1.py

```

1 """
2 cudal.disp1
3 =====
4
5 Translation of ``Disp1.sas`` -- Dissolution, Sampling Plan 1
6 (single location, USP <711>).
7
8 Three analyses, matching the three SAS entry points:
9
10 * :func:`acceptance_limit_table`  (``CALDISP1`` / ``PRTDISP1``, A1DISP1=Y)
11 * :func:`probability_of_passing`  (``EVDISP1`` / ``SIGDISP1``, A2DISP1=Y)
12 * :func:`sample_probability`      (``SMPDISP1``, A3DISP1=Y)
13
14 Note the dissolution confidence multiplier is one-sided:
15 ``Z = PROBIT(SQRT(CILEVEL/100))``, unlike the two-sided
16 ``PROBIT((1+SQRT(CILEVEL/100))/2)`` used for content uniformity.
17
18 Performance: see :mod:`cudal.cusp1` -- the MEAN grid's boundary standard
19 deviations are all solved in one vectorized batch via
20 :func:`cudal.core.batched_root_find`, and ``probability_of_passing``
21 evaluates the full (U, CV) grid as one broadcasted array.
22 """
23
24 from __future__ import annotations
25
26 import numpy as np
27 import pandas as pd
28
29 from .core import batched_root_find, cinv, dissolution_bound, probchi, probit, pro
30 bnorm
31
32 def acceptance_limit_table(
33     number: int,
34     q: float,
35     lbound: float,
36     cilevel: float,
37     meanadj_step: float = 0.2,
38 ) -> pd.DataFrame:
39     """
40     SAS: ``%CALDISP1`` (A1DISP1=Y or A2DISP1=Y).
41
42     Parameters
43     -----
44     number : int
45         Sample size N (SAS &NUMBER).
46     q : float
47         USP <711> Q value (% dissolved), e.g. 80.
48     lbound, cilevel : float
49         Required lower bound (%) and confidence level (%).
50     meanadj_step : float
51         Step size for the internal MEANADJ = MEAN - Q grid (SAS D = 0.2).
52
53     Returns
54     -----
55     pandas.DataFrame with columns ["MEAN", "CV"] where MEAN ranges from
56     just above Q up to 100 (Q + LIM). CV = 0.0 marks a mean adjustment
57     where even a (near) zero SD fails to meet the bound.
58     """
59     lim = 100 - q
60     n = number
61     z = probit(np.sqrt(cilevel / 100))

```

```

62 chi = cinv(1 - np.sqrt(cilevel / 100), n - 1)
63 target_prob = lbound / 100
64
65 meanadjs = np.round(np.arange(meanadj_step, lim + meanadj_step / 2, meanadj_step), 6)
66
67 def overbd_of_sd(sampsd):
68     sigma = np.sqrt((n - 1) * sampsd**2 / chi)
69     ll_u = meanadjs - z * sigma / np.sqrt(n)
70     return dissolution_bound(ll_u, sigma) - target_prob
71
72 sd_lo, sd_hi = 0.002, 60.0
73 root, found = batched_root_find(overbd_of_sd, sd_lo, sd_hi)
74
75 floor_fail = overbd_of_sd(np.full_like(meanadjs, sd_lo)) < 0
76 sd = np.where(floor_fail, sd_lo, np.where(found, root, sd_hi))
77 mean = meanadjs + q
78 cv = np.where(floor_fail, 0.0, 100 * sd / mean)
79
80 return pd.DataFrame({"MEAN": mean, "CV": cv})
81
82
83 def probability_of_passing(
84     table: pd.DataFrame,
85     number: int,
86     u_values,
87     cv_values,
88 ) -> pd.DataFrame:
89     """
90     SAS: ``%EVDISP1`` / ``%SIGDISP1`` (A2DISP1=Y).
91
92     Same trapezoidal-style accumulation as
93     :func:`cudal.cuspl.probability_of_passing`, plus the SAS code's extra
94     one-sided upper-tail correction applied to every table row whose mean
95     exceeds 99.9 (% claim), using that row's own standard deviation rather
96     than an average with its neighbour. The full (row, U, CV) grid is
97     evaluated as one broadcasted array.
98
99     Returns
100     -----
101     pandas.DataFrame with columns ["U", "CV", "PTRAP"].
102     """
103     t = table.sort_values("MEAN").reset_index(drop=True)
104     x = t["MEAN"].to_numpy()
105     std = (t["MEAN"] * t["CV"] / 100).to_numpy()
106     n = number
107
108     u = np.asarray(u_values, dtype=float)[None, :, None] # (1, U, 1)
109     cv = np.asarray(cv_values, dtype=float)[None, None, :] # (1, 1, CV)
110     sigma = u * cv / 100 # (1, U, CV)
111
112     # main pairwise (trapezoid-style) term, rows 2..end
113     x_hi, x_lo = x[1:, None, None], x[:-1, None, None]
114     std_hi, std_lo = std[1:, None, None], std[:-1, None, None]
115     pmean = probnorm((x_hi - u) * np.sqrt(n) / sigma) - probnorm((x_lo - u) * np.
sqrt(n) / sigma)
116     aveht = (std_hi + std_lo) / 2
117     pstd = probchi((n - 1) * aveht**2 / sigma**2, n - 1)
118     ptrap = np.sum(pmean * pstd, axis=0) # (U, CV)
119
120     # extra upper-tail correction for every row with X > 99.9
121     tail_mask = x > 99.9
122     if np.any(tail_mask):
123         xt = x[tail_mask][:, None, None]

```

```

124     stdt = std[tail_mask][:, None, None]
125     pmean_t = 1 - probnorm((xt - u) * np.sqrt(n) / sigma)
126     pstd_t = probchi((n - 1) * stdt**2 / sigma**2, n - 1)
127     ptrap = ptrap + np.sum(pmean_t * pstd_t, axis=0)
128
129     u_flat = np.asarray(u_values, dtype=float)
130     cv_flat = np.asarray(cv_values, dtype=float)
131     uu, cc = np.meshgrid(u_flat, cv_flat, indexing="ij")
132     return pd.DataFrame({"U": uu.ravel(), "CV": cc.ravel(), "PTRAP": ptrap.ravel()
133 })
134
135 def sample_probability(
136     mean: float,
137     cv: float,
138     number: int,
139     q: float,
140     cilevel: float,
141 ) -> dict:
142     """
143     SAS: ``%SMPDISP1`` (A3DISP1=Y).
144
145     Given an observed sample mean (% dissolved) and CV (%), computes the
146     probability ("OVERBD") that future samples pass USP <711>.
147     """
148     n = number
149     z = probit(np.sqrt(cilevel / 100))
150     chi = cinv(1 - np.sqrt(cilevel / 100), n - 1)
151
152     meanadj = mean - q
153     sampsd = mean * cv / 100
154     sigma = np.sqrt((n - 1) * sampsd**2 / chi)
155     ll_u = meanadj - z * sigma / np.sqrt(n)
156     overbd = dissolution_bound(ll_u, sigma)
157
158     return {"MEAN": mean, "CV": cv, "SAMPD": sampsd, "OVERBD": overbd}

```

pycudal/cudal/disp2.py

```

1 """
2 cudal.disp2
3 =====
4
5 Translation of ``Disp2.sas`` -- Dissolution, Sampling Plan 2
6 (multiple locations, variance-components model, USP <711>).
7
8 Unlike the content-uniformity Sampling Plan 2 (:mod:`cudal.cusp2`), the
9 dissolution test only has a *lower* bound (Q value), so there is a single
10 ``MEANL`` boundary per (SE, SM) combination, not a [MEANL, MEANU] pair.
11
12 Three analyses:
13
14 * :func:`acceptance_limit_table`  (``CALDISP2```, A1DISP2=Y)
15 * :func:`probability_of_passing`  (``EVDISP2``` / ``SIGDISP2```, A2DISP2=Y)
16 * :func:`sample_probability`      (``SMPDISP2```, A3DISP2=Y)
17
18 Performance: as in :mod:`cudal.cusp2`, the whole (SE, SM) grid is
19 flattened and solved for MEANL in one vectorized batch via
20 :func:`cudal.core.batched_root_find`, instead of a per-cell scalar search.
21 """
22
23 from __future__ import annotations
24
25 import numpy as np
26 import pandas as pd
27
28 from .core import batched_root_find, cinv, dissolution_bound, probchi, probit, pro
29 from .cusp2 import _variance_components
30
31
32 def acceptance_limit_table(
33     num: int,
34     loc: int,
35     q: float,
36     lbound: float,
37     cilevel: float,
38     se_values,
39     sm_values,
40     meanadj_search_range=(-20.0, 100.0),
41 ) -> pd.DataFrame:
42     """
43     SAS: ``%CALDISP2`` (A1DISP2=Y).
44
45     For each (SE, SM) combination, finds the smallest acceptable sample
46     mean (MEAN = MEANL + Q) such that the probability of passing USP
47     <711> is still at least ``lbound``%. The whole SE/SM grid is solved
48     in one vectorized batch (see module docstring).
49
50     Returns
51     -----
52     pandas.DataFrame with columns ["SE", "SM", "MEAN"] (NaN MEAN where no
53     acceptable mean exists in the search range for that SE/SM).
54     """
55     nn, l = num, loc
56     n = nn * l
57     lim = 100 - q
58     z = probit(np.sqrt(cilevel / 100))
59     chierr = cinv(1 - np.sqrt(cilevel / 100), 1 * (nn - 1))
60     chiloc = cinv(1 - np.sqrt(cilevel / 100), 1 - 1)
61     target_prob = lbound / 100

```



```

62     lo, hi = meanadj_search_range
63     hi = min(hi, lim)
64
65     se_grid, sm_grid = np.meshgrid(
66         np.asarray(se_values, dtype=float), np.asarray(sm_values, dtype=float), in
dexing="ij"
67     )
68     se_flat = se_grid.ravel()
69     sm_flat = sm_grid.ravel()
70
71     var, mvar = _variance_components(se_flat, sm_flat, nn, l, chierr, chiloc)
72     sigma = np.sqrt(var)
73     se_of_mean = np.sqrt(mvar / n)
74
75     def func(meanadj):
76         ll_u = meanadj - z * se_of_mean
77         return dissolution_bound(ll_u, sigma) - target_prob
78
79     meanl, found = batched_root_find(func, lo, hi, which="first")
80     mean = np.where(found, meanl + q, np.nan)
81
82     return pd.DataFrame({"SE": se_flat, "SM": sm_flat, "MEAN": mean})
83
84
85 def probability_of_passing(
86     table: pd.DataFrame,
87     num: int,
88     loc: int,
89     dse: float,
90     dsm: float,
91     u_values,
92     sigse_values,
93     sigsm_values,
94 ) -> pd.DataFrame:
95     """
96     SAS: ``%EVDISP2`` / ``%SIGDISP2`` (A2DISP2=Y).
97
98     One-sided analogue of :func:`cudal.cusp2.probability_of_passing`:
99     ``PMEAN = 1 - PROBNORM((MEAN - U) * SQRT(N / EXPSM2))`` since there is
100     only a lower acceptance bound on the mean. The full grid is evaluated
101     as one broadcasted array.
102
103     Returns
104     -----
105     pandas.DataFrame with columns ["U", "SIGSE", "SIGSM", "PSUM"].
106     """
107     t = table.dropna(subset=["MEAN"])
108     nn, l = num, loc
109     n = nn * l
110
111     se = t["SE"].to_numpy()[:, None, None, None]
112     sm = t["SM"].to_numpy()[:, None, None, None]
113     mean = t["MEAN"].to_numpy()[:, None, None, None]
114
115     u = np.asarray(u_values, dtype=float)[None, :, None, None]
116     sigse = np.asarray(sigse_values, dtype=float)[None, None, :, None]
117     sigsm = np.asarray(sigsm_values, dtype=float)[None, None, None, :]
118
119     expse2 = sigse**2
120     expsm2 = expse2 + nn * sigsm**2
121
122     pmean = 1 - probnorm((mean - u) * np.sqrt(n / expsm2))
123     pse = probchi(1 * (nn - 1) * se**2 / expse2, 1 * (nn - 1)) - probchi(
124         1 * (nn - 1) * (se - dse) ** 2 / expse2, 1 * (nn - 1)

```

```

125 )
126 psm = probchi((1 - 1) * nn * sm**2 / expsm2, 1 - 1) - probchi(
127     (1 - 1) * nn * (sm - dsm) ** 2 / expsm2, 1 - 1
128 )
129
130 psum = np.sum(pmean * pse * psm, axis=0) # (U, SIGSE, SIGSM)
131
132 uu, ee, mm = np.meshgrid(
133     np.asarray(u_values, dtype=float),
134     np.asarray(sigse_values, dtype=float),
135     np.asarray(sigsm_values, dtype=float),
136     indexing="ij",
137 )
138 return pd.DataFrame(
139     {
140         "U": uu.ravel(),
141         "SIGSE": ee.ravel(),
142         "SIGSM": mm.ravel(),
143         "PSUM": psum.ravel(),
144     }
145 )
146
147
148 def sample_probability(
149     mean: float,
150     se: float,
151     sm: float,
152     num: int,
153     loc: int,
154     q: float,
155     cilevel: float,
156 ) -> dict:
157     """
158     SAS: ``%SMPDISP2`` (A3DISP2=Y).
159
160     Given an observed sample MEAN, within-location SD (SE) and
161     between-location SD (SM), computes the probability ("OVERBD") that
162     future samples pass USP <711>.
163     """
164     nn, l = num, loc
165     n = nn * l
166     z = probit(np.sqrt(cilevel / 100))
167     chierr = cinv(1 - np.sqrt(cilevel / 100), 1 * (nn - 1))
168     chiloc = cinv(1 - np.sqrt(cilevel / 100), 1 - 1)
169
170     var, mvar = _variance_components(se, sm, nn, l, chierr, chiloc)
171     sigma = np.sqrt(var)
172     se_of_mean = np.sqrt(mvar / n)
173
174     meanadj = mean - q
175     llu = meanadj - z * se_of_mean
176     overbd = dissolution_bound(llu, sigma)
177
178     return {
179         "MEAN": mean,
180         "SE": se,
181         "SM": sm,
182         "VAR": var,
183         "MVAR": mvar,
184         "SIGMA": sigma,
185         "OVERBD": overbd,
186     }

```

pycudal/cudal/gui.py

```

1 """
2 cudal_gui_pyside6.py -- FINAL integrated PySide6 version
3
4 CuDAL GUI: 4 tabs (CU Plan 1/2, Dissolution Plan 1/2) x 3 modes
5 (table / evaluate / sample), with:
6
7 * modern flat "card" design, strict column-aligned parameter forms
8 * background-thread calculations with real progress bar + status text
9 * scrollable parameters, live validation, tooltips, Reset defaults
10 * results table: sorting, zebra stripes, conditional P(pass) coloring,
11   Ctrl+C copy, CSV export, modal matplotlib dialog (lines+spline and
12   heatmap with 80/90% contours), Save PNG
13 * settings persistence (parameters, mode, tab, window geometry)
14 * logo.png + fonts/ (TTF) support on Windows AND Linux
15 * menu, shortcuts (Ctrl+R/E/P, F1), About dialog, --selftest
16
17 Requirements:
18     pip install PySide6 numpy pandas scipy matplotlib openpyxl
19
20 Run:
21     python cudal_gui_pyside6.py                # GUI
22     python cudal_gui_pyside6.py --selftest      # quick unit checks
23 """
24
25 from __future__ import annotations
26
27 import csv
28 import json
29 import math
30 import os
31 import sys
32 import time
33 import traceback
34
35 from PySide6.QtCore import QSize, Qt, QThread, QUrl, Signal
36 from PySide6.QtGui import (
37     QAction,
38     QBrush,
39     QColor,
40     QDesktopServices,
41     QFont,
42     QFontDatabase,
43     QIcon,
44     QKeySequence,
45     QPainter,
46     QPixmap,
47     QShortcut,
48 )
49 from PySide6.QtWidgets import (
50     QAbstractItemView,
51     QApplication,
52     QButtonGroup,
53     QComboBox,
54     QDialog,
55     QFileDialog,
56     QFrame,
57     QGridLayout,
58     QGroupBox,
59     QHBoxLayout,
60     QLabel,
61     QLineEdit,
62     QMainWindow,

```

```

63     QMessageBox,
64     QProgressBar,
65     QPushButton,
66     QRadioButton,
67     QScrollArea,
68     QTableWidget,
69     QTableWidgetItem,
70     QTabWidget,
71     QVBoxLayout,
72     QWidget,
73)
74
75try:
76     from reportlab.lib.pagesizes import landscape, letter
77     from reportlab.pdfgen import canvas as rl_canvas
78
79     HAVE_PDF = True
80except Exception: # pragma: no cover
81     HAVE_PDF = False
82
83# Heavy / optional dependencies load in stages after the splash appears.
84np = pd = None
85cusp1 = cusp2 = disp1 = disp2 = None
86Figure = FigureCanvas = NavigationToolbar = None
87make_interp_spline = None
88HAVE_CUDAL = HAVE_MPL = HAVE_SPLINE = HAVE_XLSX = False
89
90REPO_URL = "https://github.com/moazelessawey/pycudal"
91
92# -----
93# Constants
94# -----
95VERSION = "1.0.8 (PySide6, modern UI)"
96
97BG = "#f4f6f9"
98PANEL_BG = "#ffffff"
99ACCENT = "#2f6fed"
100ACCENT_DARK = "#204ea6"
101TEXT = "#1c2733"
102MUTED = "#64748b"
103BORDER = "#e2e8f0"
104OK_GREEN = "#1a8754"
105ERR_RED = "#c0392b"
106
107SERIES_COLORS = [ACCENT, OK_GREEN, ERR_RED, "#8e44ad", "#e67e22", "#16a085", "#c2
417d", "#5b6470"]
108
109SETTINGS_PATH = os.path.join(os.path.dirname(os.path.abspath(__file__)), "cudal_g
ui_settings.json")
110
111# -----
112# SAS-style PDF listing export (reportlab, Courier, column wrapping)
113# -----
114_PDF_FONT = "Courier"
115_PDF_FS = 8.0
116_PDF_LEAD = 11.0
117_PDF_CHAR = 0.6 * _PDF_FS
118
119
120def _fmt_num(v):
121     """Format a numeric value to 2 decimal places; '*' for NaN/Inf."""
122     try:
123         if isinstance(v, float) and math.isnan(v):
124             return "*"

```

```

125         if pd.isna(v):
126             return "*"
127         f = float(v)
128         if math.isnan(f) or math.isinf(f):
129             return "*"
130         return f"{f:.2f}"
131     except Exception:
132         return str(v)
133
134
135 def _cell(v):
136     return _fmt_num(v)
137
138
139 def write_sas_pdf(path, df, title_lines):
140     """Write `df` to `path` as a SAS-listing-style PDF (Courier, wrapped).
141
142     The report title block AND the table header are repeated on every page.
143     """
144     page_w, page_h = landscape(letter)
145     margin = 36.0
146     usable = int((page_w - 2 * margin) / _PDF_CHAR)
147
148     c = rl_canvas.Canvas(path, pagesize=landscape(letter))
149     c.setTitle("CuDAL results")
150     st = {"y": page_h - margin, "table_header_fn": None}
151
152     # -- low-level drawing (no page checks, used inside new_page) -----
153     def raw(text, indent=0):
154         c.setFont(_PDF_FONT, _PDF_FS)
155         c.drawString(margin + indent * _PDF_CHAR, st["y"], text)
156         st["y"] -= _PDF_LEAD
157
158     def raw_centered(text):
159         raw(text, max(0, (usable - len(text)) // 2))
160
161     def draw_title():
162         for t in title_lines:
163             raw_centered(t)
164         raw("")
165
166     def new_page():
167         c.showPage()
168         st["y"] = page_h - margin
169         draw_title() # title on every page
170         if st["table_header_fn"] is not None: # table header on every page
171             st["table_header_fn"]()
172
173     # -- page-checking drawing -----
174     def put(text, indent=0):
175         if st["y"] < margin:
176             new_page()
177         raw(text, indent)
178
179     def centered(text):
180         put(text, max(0, (usable - len(text)) // 2))
181
182     def blank(n=1):
183         for _ in range(n):
184             put("")
185
186     def need(n):
187         """Force a page break unless `n` more lines fit on this page."""
188         if st["y"] - n * _PDF_LEAD < margin:

```

```

189         new_page()
190
191     draw_title() # first-page title
192
193     cols = [str(x) for x in df.columns]
194     low = {cl.lower(): cl for cl in cols}
195
196     # ----- Plan-2 style matrix: SE rows x SM groups of LL/UL -----
197     if {"se", "sm", "ll", "ul"} <= set(low):
198         se_c, sm_c, ll_c, ul_c = low["se"], low["sm"], low["ll"], low["ul"]
199         tmp = df.copy()
200         tmp["_se"] = pd.to_numeric(tmp[se_c], errors="coerce")
201         tmp["_sm"] = pd.to_numeric(tmp[sm_c], errors="coerce")
202         ses = sorted(tmp["_se"].dropna().unique())
203         sms = sorted(tmp["_sm"].dropna().unique())
204
205         get = {}
206         for r in tmp.itertuples():
207             if pd.notna(r._se) and pd.notna(r._sm):
208                 get[(r._se, r._sm)] = (_fmt_num(r[ll_c]), _fmt_num(r[ul_c]))
209
210         ll_w = max([2] + [len(v[0]) for v in get.values()])
211         ul_w = max([2] + [len(v[1]) for v in get.values()])
212         grp_w = ll_w + 1 + ul_w
213         se_w = max([2] + [len(_fmt_num(s)) for s in ses])
214         gap = 3
215         per_page = max(1, (usable - se_w + gap) // (grp_w + gap))
216
217         def header(seg):
218             centered("STANDARD DEVIATION OF LOCATION MEANS")
219             blank()
220             line = " " * se_w
221             for sm in seg:
222                 line += (" " * gap) + f" {_fmt_num(sm):>{grp_w}}"
223             put(line)
224             line = f"{'SE':>{se_w}}"
225             for _ in seg:
226                 line += (" " * gap) + f"{'LL':>{ll_w}} {'UL':>{ul_w}}"
227             put(line)
228             blank()
229
230         HDR_LINES = 5 # banner + blank + SM row + LL/UL row + blank
231
232         for start in range(0, len(sms), per_page): # column wrapping
233             seg = sms[start : start + per_page]
234             st["table_header_fn"] = None # don't repeat old header
235             need(HDR_LINES + 1)
236             header(seg)
237             st["table_header_fn"] = lambda seg=seg: header(seg)
238             for se in ses:
239                 line = f" {_fmt_num(se):>{se_w}}"
240                 for sm in seg:
241                     ll, ul = get.get((se, sm), ("*", "*"))
242                     line += (" " * gap) + f" {ll:>{ll_w}} {ul:>{ul_w}}"
243                 put(line) # auto page-break repeats
244                 blank() # title + segment header
245             st["table_header_fn"] = None
246             c.save()
247             return
248
249     # ----- Plan-1 style: long rows wrapped into side-by-side blocks --
250     body = [[_cell(v) for v in row] for _, row in df.iterrows()]
251     widths = [

```

```

252     max(len(cols[j]), max((len(r[j]) for r in body), default=0)) for j in range(len(cols))
253 ]
254
255     hdr = []
256     for j, h in enumerate(cols):
257         parts = h.split(" ", 1)
258         if len(parts) == 2 and len(parts[0]) <= widths[j] and len(parts[1]) <= widths[j]:
259             hdr.append((parts[0], parts[1]))
260         else:
261             hdr.append((h, ""))
262     hdr_lines = 2 if any(b for _a, b in hdr) else 1
263
264     gap, block_gap = 3, 4
265     block_w = sum(widths) + gap * (len(widths) - 1)
266     n_blocks = max(1, (usable + block_gap) // (block_w + block_gap))
267     rpb = max(1, -(len(body) // n_blocks)) # rows per block
268     chunks = [body[i : i + rpb] for i in range(0, len(body), rpb)]
269
270     def header_line(li):
271         return (" " * gap).join(f"{hdr[j][li]:^{widths[j]}}" for j in range(len(cols)))
272
273     def block_line(chunk, i):
274         parts = []
275         for j in range(len(cols)):
276             val = chunk[i][j] if i < len(chunk) else ""
277             parts.append(f"{val:>{widths[j]}}")
278         return (" " * gap).join(parts)
279
280     for p in range(0, len(chunks), n_blocks):
281         page_chunks = chunks[p : p + n_blocks]
282
283         def draw_hdr(_pc=page_chunks):
284             for li in range(hdr_lines):
285                 put((" " * block_gap).join(header_line(li) for _ in _pc))
286             blank()
287
288         st["table_header_fn"] = None # don't repeat old header
289         need(hdr_lines + 2)
290         draw_hdr()
291         st["table_header_fn"] = draw_hdr # repeat on page breaks
292         for i in range(rpb):
293             put((" " * block_gap).join(block_line(ch, i) for ch in page_chunks))
294         blank()
295     st["table_header_fn"] = None
296
297     c.save()
298
299
300 def resource_path(relative: str) -> str:
301     """Resolve bundled files both from source and from a PyInstaller bundle."""
302     base = getattr(sys, "_MEIPASS", os.path.dirname(os.path.abspath(__file__)))
303     return os.path.join(base, relative)
304
305
306 def _register_local_fonts() -> str | None:
307     """Load every TTF/OTF in fonts/ into the Qt application font database."""
308     font_dir = resource_path("fonts")
309     if not os.path.isdir(font_dir):
310         return None
311     family = None
312     for f in sorted(os.listdir(font_dir)):

```

```

313         if f.lower().endswith((".ttf", ".otf")):
314             fid = QFontDatabase.addApplicationFont(os.path.join(font_dir, f))
315             if fid != -1 and family is None:
316                 fams = QFontDatabase.applicationFontFamilies(fid)
317                 if fams:
318                     family = fams[0]
319         return family
320
321
322 def build_stylesheet(family: str) -> str:
323     """Balanced 'classic-modern' theme: familiar desktop look, clean palette."""
324     return f"""
325     QMainWindow, QDialog {{ background: {BG}; }}
326     QWidget {{ background: {BG}; color: #1f2937; font-family: "{family}"; }}
327
328     QLabel {{ background: transparent; }}
329     QLabel#header {{ font-size: 17px; font-weight: 700; }}
330     QLabel#subheader {{ color: {MUTED}; }}
331     QLabel#muted {{ color: {MUTED}; font-size: 9pt; }}
332     QLabel#section {{ color: {ACCENT_DARK}; font-weight: 700; }}
333     QLabel#fieldlabel {{ color: #2f3a4d; }}
334
335     /* panels & group boxes: light cards with a classic frame */
336     QFrame#panel, QGroupBox#card {{
337         background: {PANEL_BG};
338         border: 1px solid #cfd8e3;
339         border-radius: 6px;
340     }}
341     QGroupBox#card {{ margin-top: 12px; font-weight: 600; color: #2f3a4d; }}
342     QGroupBox#card::title {{
343         subcontrol-origin: margin; left: 10px; padding: 0 4px;
344         color: {ACCENT_DARK};
345     }}
346
347     /* radio buttons: classic circle + dot, accent colour */
348     QRadioButton {{ spacing: 7px; padding: 3px; background: transparent; }}
349     QRadioButton::indicator {{
350         width: 15px; height: 15px;
351         border: 1px solid #8b97a8; border-radius: 8px; background: white;
352     }}
353     QRadioButton::indicator: hover {{ border-color: {ACCENT}; }}
354     QRadioButton::indicator: checked {{
355         border: 1px solid {ACCENT_DARK};
356         background: radialgradient(cx:0.5, cy:0.5, radius:0.45, fx:0.5, fy:0.5,
357             stop:0 {ACCENT}, stop:0.55 {ACCENT},
358             stop:0.56 white, stop:1 white);
359     }}
360
361     /* inputs: white with a firm border */
362     QLineEdit {{
363         background: white; border: 1px solid #aeb9c8; border-radius: 3px;
364         padding: 4px 8px; selection-background-color: {ACCENT};
365     }}
366     QLineEdit: hover {{ border-color: #8b97a8; }}
367     QLineEdit: focus {{ border: 1px solid {ACCENT}; }}
368     QLineEdit[invalid="true"] {{ background: #fdf1f1; border: 1px solid {ERR_RED}
369 ; color: {ERR_RED}; }}
370
371     /* buttons: subtle gradient like classic toolkits */
372     QPushButton {{
373         background: qlineargradient(x1:0, y1:0, x2:0, y2:1, stop:0 #fdfefe, stop:
374 1 #e7ebf2);
375         color: #2f3a4d; border: 1px solid #aeb9c8; border-radius: 4px;
376         padding: 7px 14px;

```



```

375     }}
376     QPushButton:hover {{
377         background: qlineargradient(x1:0, y1:0, x2:0, y2:1, stop:0 #f4f8ff, stop:
1 #dfe8f7);
378         border-color: {ACCENT}; color: {ACCENT_DARK};
379     }}
380     QPushButton:pressed {{ background: #dfe4ec; border-color: #8b97a8; }}
381     QPushButton:disabled {{ background: #eef1f5; color: #9aa7b8; border-
color: #d5dce6; }}
382     QPushButton#accent {{
383         background: qlineargradient(x1:0, y1:0, x2:0, y2:1, stop:0 #5a8ff2, stop:
1 {ACCENT});
384         border: 1px solid {ACCENT_DARK}; color: white; padding: 8px 20px;
385     }}
386     QPushButton#accent:hover {{
387         background: qlineargradient(x1:0, y1:0, x2:0, y2:1, stop:0 #4c82f3, stop:
1 {ACCENT_DARK});
388     }}
389     QPushButton#accent:disabled {{ background: #b6c6ea; border-
color: #9db1dd; color: #f8fafc; }}
390     QPushButton#outline {{ background: white; border: 1px solid {ACCENT}; color:
{ACCENT_DARK}; }}
391     QPushButton#outline:hover {{ background: #eaf1fe; }}
392
393     /* progress: framed bar with accent fill, same height as the Run button */
394     QProgressBar {{
395         background: white;
396         border: 1px solid #aeb9c8;
397         border-radius: 4px;
398         text-align: center;
399     }}
400     QProgressBar::chunk {{
401         background: qlineargradient(x1:0, y1:0, x2:0, y2:1, stop:0 #6ea0f5, stop:
1 {ACCENT});
402         border-radius: 3px;
403     }}
404
405     /* scrollbars: classic width, soft colours, no arrows */
406     QScrollArea {{ border: none; background: transparent; }}
407     QScrollBar:vertical {{ background: #eef1f5; width: 12px; border: 1px solid #d
fe4ec; }}
408     QScrollBar::handle:vertical {{ background: #c3ccd8; border-radius: 6px; min-
height: 24px; margin: 1px; }}
409     QScrollBar::handle:vertical:hover {{ background: #a9b4c4; }}
410     QScrollBar::add-line:vertical, QScrollBar::sub-line:vertical {{ height: 0; }}
411     QScrollBar:horizontal {{ background: #eef1f5; height: 12px; border: 1px solid
#dfe4ec; }}
412     QScrollBar::handle:horizontal {{ background: #c3ccd8; border-
radius: 6px; min-width: 24px; margin: 1px; }}
413     QScrollBar::add-line:horizontal, QScrollBar::sub-
line:horizontal {{ width: 0; }}
414
415     /* tabs: classic raised tabs on a framed pane */
416     QTabWidget::pane {{
417         border: 1px solid #cfd8e3; background: {PANEL_BG}; border-radius: 4px;
418     }}
419     QTabBar::tab {{
420         background: qlineargradient(x1:0, y1:0, x2:0, y2:1, stop:0 #f2f5f9, stop:
1 #e2e7ee);
421         color: {MUTED};
422         border: 1px solid #cfd8e3; border-bottom: none;
423         border-top-left-radius: 5px; border-top-right-radius: 5px;
424         padding: 7px 16px; margin-right: 2px;
425     }}

```

```

426     QTabBar::tab: hover {{ color: #2f3a4d; background: #eef3fa; }}
427     QTabBar::tab: selected {{ background: {PANEL_BG}; color: {ACCENT_DARK}; font-
weight: 600; }}
428
429     /* table: classic grid with soft header */
430     QTableWidget {{
431         background: white; alternate-background-color: #f4f7fb;
432         border: 1px solid #cfd8e3; border-radius: 4px;
433         gridline-color: #dde3ec;
434         selection-background-color: #cfdffc; selection-color: #101828;
435     }}
436     QHeaderView::section {{
437         background: qlineargradient(x1:0, y1:0, x2:0, y2:1, stop:0 #fbfcfe, stop:
1 #edf0f5);
438         color: #334155; border: none;
439         border-right: 1px solid #dde3ec; border-bottom: 1px solid #cfd8e3;
440         padding: 5px; font-weight: 600;
441     }}
442
443     QComboBox {{
444         background: white; border: 1px solid #aeb9c8; border-radius: 3px;
445         padding: 4px 8px; min-width: 140px;
446     }}
447     QComboBox: hover {{ border-color: {ACCENT}; }}
448     QComboBox::drop-down {{
449         subcontrol-origin: padding; subcontrol-position: center right;
450         width: 20px; border: none;
451         background: qlineargradient(x1:0, y1:0, x2:0, y2:1, stop:0 #f6f8fb, stop:
1 #e2e7ee);
452     }}
453     QComboBox QAbstractItemView {{
454         background: white; border: 1px solid #aeb9c8;
455         selection-background-color: #cfdffc;
456     }}
457
458     QMenuBar {{ background: #eef1f6; border-bottom: 1px solid #cfd8e3; }}
459     QMenuBar::item {{ background: transparent; padding: 4px 10px; }}
460     QMenuBar::item: selected {{ background: #d7e3fc; border-radius: 3px; }}
461     QMenu {{ background: {PANEL_BG}; border: 1px solid #aeb9c8; padding: 4px; }}
462     QMenu::item {{ padding: 5px 24px; border-radius: 3px; }}
463     QMenu::item: selected {{ background: #cfdffc; }}
464     QMenu::separator {{ height: 1px; background: #dde3ec; margin: 4px 8px; }}
465
466     QStatusBar {{ background: #eef1f6; color: {MUTED}; border-
top: 1px solid #cfd8e3; }}
467
468     /* classic pale-yellow tooltip */
469     QToolTip {{
470         background: #ffffel; color: #333333; border: 1px solid #767676; padding:
4px;
471     }}
472     QToolBar {{ background: #eef1f6; border: 0; border-bottom: 1px solid #cfd8e3;
spacing: 4px; padding: 4px; }}
473     QToolBar QToolButton {{ background: transparent; border: 1px solid transparen
t;
474         border-
radius: 4px; padding: 4px 8px; color: #2f3a4d; }}
475     QToolBar QToolButton: hover {{ background: #dfe8f7; border-color: #aeb9c8; }}
476     QToolBar QToolButton: pressed {{ background: #cfdffc; }}
477     ""
478
479
480
481# -----
482# Plot helpers

```

```

483# -----
484def _spline_xy(xs, ys, samples=300):
485    """Smooth cubic spline through (xs, ys); linear fallback w/o scipy."""
486    xs = np.asarray(xs, dtype=float)
487    ys = np.asarray(ys, dtype=float)
488    ux, inv = np.unique(xs, return_inverse=True)
489    uy = np.array([ys[inv == i].mean() for i in range(ux.size)]) if ux.size != xs
.size else ys
490    order = np.argsort(ux)
491    ux, uy = ux[order], uy[order]
492    if ux.size < 2:
493        return ux, uy
494    if HAVE_SPLINE:
495        k = min(3, ux.size - 1)
496        try:
497            spl = make_interp_spline(ux, uy, k=k)
498            xx = np.linspace(ux[0], ux[-1], samples)
499            return xx, spl(xx)
500        except Exception:
501            pass
502    xx = np.linspace(ux[0], ux[-1], samples)
503    return xx, np.interp(xx, ux, uy)
504
505
506def _grid_columns(df):
507    """Detect a 3-column grid and return (x, y, z) column *names*.
508
509    The surface value (z) is the column with the most distinct values
510    (it changes on nearly every row); the two axis columns are the rest,
511    with x = the axis having more distinct values. Returns None when the
512    DataFrame is not a proper 3-column grid (e.g. the acceptance-limit or
513    probability tables in a different column order are still handled).
514    """
515    num = df.select_dtypes(include=[np.number]).dropna()
516    if num.shape[1] != 3 or len(num) < 4:
517        return None
518
519    uniq = [num[c].nunique() for c in num.columns]
520    if min(uniq) < 2:
521        return None
522
523    zcol = num.columns[int(np.argmax(uniq))]
524    axes = sorted(
525        (c for c in num.columns if c != zcol), key=lambda c: num[c].nunique(), re
verse=True
526    )
527
528    # sanity check: the surface must vary more than either axis
529    if num[zcol].nunique() <= max(num[axes[0]].nunique(), num[axes[1]].nunique())
:
530        return None
531
532    return str(axes[0]), str(axes[1]), str(zcol)
533
534
535def _is_plan2_table(df):
536    """Detect a Plan 2 acceptance limit table (SE, SM, and >=2 value columns)."""
537    cols = [str(c).lower() for c in df.columns]
538    return "se" in cols and "sm" in cols and len(df.columns) >= 4
539
540
541def _get_plan2_axes_and_z(df):
542    """Return (x_col, y_col, [z_cols]) for Plan 2 tables."""
543    cols = [str(c) for c in df.columns]

```

```

544     lower_cols = [c.lower() for c in cols]
545     x_col = cols[lower_cols.index("se")] if "se" in lower_cols else cols[0]
546     y_col = cols[lower_cols.index("sm")] if "sm" in lower_cols else cols[1]
547     z_cols = [
548         c
549         for c, lc in zip(cols, lower_cols)
550         if lc not in ("se", "sm") and pd.api.types.is_numeric_dtype(df[c])
551     ]
552     return x_col, y_col, z_cols
553
554
555 def _plan2_eval_columns(df):
556     """Detect a Plan-2 probability-of-passing grid (U x within-SD x between-SD -
557     > P).
558     Returns (u_col, se_col, sm_col, p_col) or None.
559     """
560     num = df.select_dtypes(include=[np.number])
561     if num.shape[1] != 4:
562         return None
563     cols = [str(c) for c in num.columns]
564     low = [c.lower() for c in cols]
565
566     def find(*keys):
567         for k in keys:
568             for c, lc in zip(cols, low):
569                 if k in lc:
570                     return c
571         return None
572
573     p_col = find("prob", "pass", "psum")
574     se_col = find("within", "sigse") or next((c for c, lc in zip(cols, low) if lc
575 == "se"), None)
576     sm_col = find("between", "sigsm") or next((c for c, lc in zip(cols, low) if l
577 c == "sm"), None)
578     if None in (p_col, se_col, sm_col):
579         return None
580     rest = [c for c in cols if c not in (p_col, se_col, sm_col)]
581     if len(rest) != 1:
582         return None
583     return rest[0], se_col, sm_col, p_col
584
585 def build_plan2_eval_plot(fig, df, u_col, se_col, sm_col, p_col):
586     """Faceted view of the Plan-2 probability-of-passing surface.
587     One panel per between-location SD (SM); inside each panel one spline
588     curve per within-location SD (SE): P(pass) vs true mean U.
589     Dashed lines mark the usual 80 % / 90 % coverage levels.
590     """
591     work = pd.DataFrame(
592         {
593             "u": pd.to_numeric(df[u_col], errors="coerce"),
594             "se": pd.to_numeric(df[se_col], errors="coerce"),
595             "sm": pd.to_numeric(df[sm_col], errors="coerce"),
596             "p": pd.to_numeric(df[p_col], errors="coerce"),
597         }
598     ).dropna()
599
600     if work.empty:
601         fig.text(0.5, 0.5, "Not enough numeric data to plot.", ha="center", va="c
602 enter")
603     fig.tight_layout()
604     return

```

```

604
605     sm_vals = sorted(work["sm"].unique())
606     se_vals = sorted(work["se"].unique())
607     n = len(sm_vals)
608     ncols = min(3, n)
609     nrows = -(-n // ncols) # ceil without importing math
610     axes = fig.subplots(nrows, ncols, squeeze=False, sharex=True, sharey=True)
611
612     for i, smv in enumerate(sm_vals):
613         ax = axes[i // ncols][i % ncols]
614         panel = work[work["sm"] == smv]
615         for color, sev in zip(SERIES_COLORS, se_vals):
616             sub = panel[panel["se"] == sev].sort_values("u")
617             xs = sub["u"].to_numpy(dtype=float)
618             ys = sub["p"].to_numpy(dtype=float)
619             if xs.size == 0:
620                 continue
621             ax.plot(xs, ys, "o", color=color, ms=3, alpha=0.7)
622             xx, yy = _spline_xy(xs, ys)
623             ax.plot(xx, yy, "-",
624                   ", color=color, lw=1.6, label=f'{se_col} = {sev:g}"))
625             for t in (0.8, 0.9):
626                 ax.axhline(t, ls="--", lw=0.8, color="0.5")
627             ax.set_title(f'{sm_col} = {smv:g}', fontsize=9)
628             ax.grid(True, alpha=0.3)
629             if i // ncols == nrows - 1:
630                 ax.set_xlabel(u_col, fontsize=9)
631             if i % ncols == 0:
632                 ax.set_ylabel(p_col, fontsize=9)
633
634     for j in range(n, nrows * ncols): # hide unused panels
635         axes[j // ncols][j % ncols].set_axis_off()
636
637     handles, labels = axes[0][0].get_legend_handles_labels()
638     if handles:
639         fig.legend(
640             handles, labels, loc="lower center", ncol=min(len(labels), 4), fontsi
641             ze=8, frameon=False
642         )
643     fig.suptitle(
644         f"Probability of passing vs {u_col} (panels: {sm_col}, curves: {se_col})
645     ", fontsize=10
646     )
647     fig.tight_layout(rect=(0, 0.05, 1, 0.95))
648
649 def build_results_plot_plan2(fig, df):
650     """Family of curves for Plan 2 tables: LL/UL vs SE, grouped by SM."""
651     x_col, y_col, z_cols = _get_plan2_axes_and_z(df)
652     if not z_cols:
653         fig.text(0.5, 0.5, "Not enough data to plot.", ha="center", va="center")
654         return
655
656     n_z = len(z_cols)
657     axes = fig.subplots(1, n_z, squeeze=False)
658
659     for idx, z_col in enumerate(z_cols):
660         ax = axes[0, idx]
661         grouped = df.groupby(y_col, sort=True)
662         for color, (sm_val, sub) in zip(SERIES_COLORS, grouped):
663             xs = sub[x_col].to_numpy(dtype=float)
664             ys = sub[z_col].to_numpy(dtype=float)
665             order = np.argsort(xs)
666             xs, ys = xs[order], ys[order]

```

```

665
666         xx, yy = _spline_xy(xs, ys)
667         ax.plot(xx, yy, "-
", color=color, lw=1.6, label=f"{y_col}={sm_val:g}")
668         ax.plot(xs, ys, "o", color=color, ms=3, alpha=0.7)
669
670         ax.set_xlabel(x_col)
671         ax.set_ylabel(z_col)
672         ax.set_title(f"{z_col} vs {x_col}")
673         ncol = 2 if len(grouped) > 6 else 1
674         ax.legend(fontsize=7, loc="best", ncol=ncol)
675         ax.grid(True, alpha=0.3)
676
677     fig.tight_layout()
678
679
680 def build_heatmap_plan2(fig, df, z_col, thresholds=(0.8, 0.9)):
681     """Contour-filled heatmap for Plan 2 tables (SE x SM -> Z)."""
682     x_col, y_col, z_cols = _get_plan2_axes_and_z(df)
683     if not z_cols:
684         fig.text(0.5, 0.5, "Not enough data to plot.", ha="center", va="center")
685         return
686     if z_col not in z_cols:
687         z_col = z_cols[0]
688
689     work = df[[x_col, y_col, z_col]].copy()
690     work = work.apply(pd.to_numeric, errors="coerce").dropna()
691     piv = work.pivot_table(index=y_col, columns=x_col, values=z_col, aggfunc="mean")
692     piv = piv.sort_index(axis=0).sort_index(axis=1)
693
694     X = piv.columns.to_numpy(dtype=float)
695     Y = piv.index.to_numpy(dtype=float)
696     Z = np.ma.masked_invalid(piv.to_numpy(dtype=float))
697
698     ax = fig.add_subplot(111)
699     if Z.count() == 0 or X.size < 2 or Y.size < 2:
700         ax.text(0.5, 0.5, "Not enough grid points for a heatmap.", ha="center", va="center")
701     fig.tight_layout()
702     return
703
704     cs = ax.contourf(X, Y, Z, levels=min(24, max(6, X.size + Y.size)), cmap="viridis")
705     fig.colorbar(cs, ax=ax, label=z_col)
706
707     if X.size <= 15:
708         ax.set_xticks(X)
709     if Y.size <= 15:
710         ax.set_yticks(Y)
711
712     if float(Z.max()) <= 1.01 and float(Z.min()) >= -0.01:
713         for i, t in enumerate(thresholds):
714             if float(Z.min()) <= t <= float(Z.max()):
715                 ax.contour(X, Y, Z, levels=[t], colors="white", linewidths=1.2)
716                 ax.text(
717                     0.02,
718                     0.98 - 0.06 * i,
719                     f"white line = {t:.0%}",
720                     transform=ax.transAxes,
721                     fontsize=8,
722                     color="white",
723                     va="top",
724                 )

```

```

725
726     ax.set_xlabel(x_col)
727     ax.set_ylabel(y_col)
728     ax.set_title(f"{z_col} over {x_col} / {y_col}", fontsize=10)
729     fig.tight_layout()
730
731
732 def build_results_plot(fig, df):
733     """Points + spline curves for the results DataFrame."""
734     p2 = _plan2_eval_columns(df)
735     if p2 is not None:
736         build_plan2_eval_plot(fig, df, *p2)
737         return
738
739     ax = fig.add_subplot(111)
740     num = df.select_dtypes(include=[np.number]).replace([np.inf, -
np.inf], np.nan).dropna()
741     num = num.copy()
742     num.columns = [str(c) for c in num.columns]
743     cols = list(num.columns)
744
745     if num.shape[1] >= 2 and len(num) >= 2:
746         grid = _grid_columns(df)
747         if grid:
748             xcol, gcol, ycol = grid
749             for color, (g, sub) in zip(SERIES_COLORS, num.groupby(gcol, sort=True
)):
750                 xs = sub[xcol].to_numpy(dtype=float)
751                 ys = sub[ycol].to_numpy(dtype=float)
752                 order = np.argsort(xs)
753                 xs, ys = xs[order], ys[order]
754                 ax.plot(xs, ys, "o", color=color, ms=3, alpha=0.7, label=f"{gcol}
= {g:g}")
755                 xx, yy = _spline_xy(xs, ys)
756                 ax.plot(xx, yy, "-", color=color, lw=1.6)
757                 ax.set_xlabel(xcol)
758                 ax.set_ylabel(ycol)
759                 ax.legend(fontsize=8, loc="best")
760         else:
761             xcol = cols[0]
762             xs_all = num[cols[0]].to_numpy(dtype=float)
763             for color, ycol in zip(SERIES_COLORS, cols[1:]):
764                 ys = num[ycol].to_numpy(dtype=float)
765                 ax.plot(xs_all, ys, "o", color=color, ms=3, alpha=0.7, label=ycol
)
766                 xx, yy = _spline_xy(xs_all, ys)
767                 ax.plot(xx, yy, "-", color=color, lw=1.6)
768                 ax.set_xlabel(xcol)
769                 ax.legend(fontsize=8, loc="best")
770
771             kind = "cubic spline" if HAVE_SPLINE else "linear interpolation"
772             ax.set_title(f"Results with {kind}", fontsize=10)
773         else:
774             ax.text(0.5, 0.5, "Not enough numeric data to plot.", ha="center", va="ce
nter")
775
776     ax.grid(True, alpha=0.3)
777     ax.set_facecolor("#fbfcfe")
778     fig.tight_layout()
779
780
781 def build_heatmap(fig, df, thresholds=(0.8, 0.9)):
782     """Contour-filled heatmap for 3-column grids (x, y, z) with correct axes."""
783     ax = fig.add_subplot(111)

```

```

784     grid = _grid_columns(df)
785     if grid is None:
786         ax.text(0.5, 0.5, "Data is not a 3-
column grid.", ha="center", va="center")
787         fig.tight_layout()
788         return
789
790     xcol, ycol, zcol = grid
791     work = df[[c for c in df.columns if str(c) in grid]].copy()
792     work.columns = [str(c) for c in work.columns]
793     work = work.apply(pd.to_numeric, errors="coerce").dropna()
794
795     piv = work.pivot_table(index=ycol, columns=xcol, values=zcol, aggfunc="mean")
796     piv = piv.sort_index(axis=0).sort_index(axis=1)
797
798     X = piv.columns.to_numpy(dtype=float)
799     Y = piv.index.to_numpy(dtype=float)
800     Z = np.ma.masked_invalid(piv.to_numpy(dtype=float)) # holes -
> masked, not artifacts
801
802     if Z.count() == 0 or X.size < 2 or Y.size < 2:
803         ax.text(0.5, 0.5, "Not enough grid points for a heatmap.", ha="center", v
a="center")
804         fig.tight_layout()
805         return
806
807     cs = ax.contourf(X, Y, Z, levels=min(24, max(6, X.size + Y.size)), cmap="viri
dis")
808     fig.colorbar(cs, ax=ax, label=zcol)
809
810     if X.size <= 12:
811         ax.set_xticks(X)
812     if Y.size <= 12:
813         ax.set_yticks(Y)
814
815     if float(Z.max()) <= 1.01: # probability-scale surface -
> draw 80/90% contours
816         for i, t in enumerate(thresholds):
817             if float(Z.min()) <= t <= float(Z.max()):
818                 ax.contour(X, Y, Z, levels=[t], colors="white", linewidths=1.2)
819                 ax.text(
820                     0.02,
821                     0.98 - 0.06 * i,
822                     f"white line = {t:.0%}",
823                     transform=ax.transAxes,
824                     fontsize=8,
825                     color="white",
826                     va="top",
827                 )
828
829     ax.set_xlabel(xcol)
830     ax.set_ylabel(ycol)
831     ax.set_title(f"{zcol} over {xcol} / {ycol}", fontsize=10)
832     fig.tight_layout()
833
834
835 # -----
836 # Form fields (strict column alignment)
837 # -----
838 class LabeledField:
839     """Label + line-edit pair placed into a SHARED grid so every input sits in
840     one strict, perfectly aligned column. Live validation via a dynamic Qt
841     property styled in QSS."""
842

```



```

843 EDIT_WIDTH = 120
844 EDIT_HEIGHT = 30
845
846 def __init__(self, layout, row, key, label, default, cast=float, tip=None):
847     self.key = key
848     self.default = default
849     self.cast = cast
850
851     self.label = QLabel(label)
852     self.label.setObjectName("fieldlabel")
853     self.label.setAlignment(Qt.AlignmentFlag.AlignLeft | Qt.AlignmentFlag.Ali
gnVCenter)
854
855     self.edit = QLineEdit(str(default))
856     self.edit.setFixedWidth(self.EDIT_WIDTH)
857     self.edit.setFixedHeight(self.EDIT_HEIGHT)
858     self.edit.setToolTip(tip or f"{label}\nDefault: {default}")
859
860     layout.addWidget(self.label, row, 0)
861     layout.addWidget(self.edit, row, 1, alignment=Qt.AlignmentFlag.AlignLeft)
862
863     self.edit.textChanged.connect(self._validate)
864     self._validate()
865
866 def _validate(self, *_):
867     raw = self.edit.text().strip()
868     ok = bool(raw)
869     if ok:
870         try:
871             self.cast(raw)
872         except ValueError:
873             ok = False
874     self.edit.setProperty("invalid", "false" if ok else "true")
875     self.edit.style().unpolish(self.edit)
876     self.edit.style().polish(self.edit)
877
878 def reset(self):
879     self.edit.setText(str(self.default))
880
881 def get(self, cast=None):
882     cast = cast or self.cast
883     raw = self.edit.text().strip()
884     if raw == "":
885         raise ValueError(f"{self.key}' cannot be empty")
886     try:
887         return cast(raw)
888     except ValueError:
889         raise ValueError(f"{self.key}' must be a number, got {raw!r}")
890
891
892 def build_form(layout, specs, start_row=0, registry=None):
893     """Fill a shared QGridLayout; returns dict[key] -> LabeledField."""
894     fields = {}
895     for i, (key, label, default) in enumerate(specs):
896         fields[key] = LabeledField(layout, start_row + i, key, label, default)
897     if registry is not None:
898         registry.update(fields)
899     return fields
900
901
902 class NumItem(QTableWidgetItem):
903     """Table item that sorts numerically instead of lexicographically."""
904
905     def __lt__(self, other):

```

```

906         a = self.data(Qt.ItemDataRole.UserRole)
907         b = other.data(Qt.ItemDataRole.UserRole)
908         try:
909             return float(a) < float(b)
910         except Exception:
911             return super().__lt__(other)
912
913
914 def fmt_num(v, digits=4):
915     try:
916         if pd.isna(v):
917             return ""
918     except Exception:
919         pass
920     if isinstance(v, (float, np.floating)):
921         return f"{v:,.{digits}f}"
922     return str(v)
923
924
925 # -----
926 # Background worker
927 # -----
928 class Worker(QThread):
929     progress = Signal(float, str)
930     finished_ok = Signal(object)
931     failed = Signal(str)
932
933     def __init__(self, job, parent=None):
934         super().__init__(parent)
935         self._job = job
936
937     def run(self):
938         try:
939             result = self._job(self.progress.emit)
940             self.finished_ok.emit(result)
941         except Exception: # noga: BLE001
942             self.failed.emit(traceback.format_exc())
943
944
945 # -----
946 # Plot dialog (modal)
947 # -----
948 class PlotDialog(QDialog):
949     def __init__(self, df, parent=None, save_base=None):
950         super().__init__(parent)
951         self.setWindowTitle("Results plot")
952         self.resize(880, 640)
953         self.setMinimumSize(520, 380)
954         self.setModal(True)
955         self._df = df
956         self._save_base = save_base or "results"
957
958         self._is_plan2 = _is_plan2_table(df)
959         self._can_heat = self._is_plan2 or _grid_columns(df) is not None
960
961         lay = QVBoxLayout(self)
962         top = QHBoxLayout()
963         top.addWidget(QLabel("Plot style:"))
964
965         self.style_cb = QComboBox()
966         self.style_cb.addItem("Lines + spline") + (["Heatmap (grid)"] if self._
can_heat else [])
967         top.addWidget(self.style_cb)
968

```

```

969         # Z-axis selector for Plan 2 tables
970         self.z_cb = None
971         if self._is_plan2:
972             _, _, z_cols = _get_plan2_axes_and_z(df)
973             if len(z_cols) > 1:
974                 top.addWidget(QLabel("Z-axis:"))
975                 self.z_cb = QComboBox()
976                 self.z_cb.addItems([str(c) for c in z_cols])
977                 top.addWidget(self.z_cb)
978                 self.z_cb.currentIndexChanged.connect(self._redraw)
979
980         top.addStretch(1)
981         save_btn = QPushButton("Save PNG")
982         save_btn.clicked.connect(self._save_png)
983         close_btn = QPushButton("Close")
984         close_btn.clicked.connect(self.accept)
985         top.addWidget(save_btn)
986         top.addWidget(close_btn)
987         lay.addLayout(top)
988
989         self._fig = Figure(dpi=100)
990         self._canvas = FigureCanvas(self._fig)
991         self._toolbar = NavigationToolbar(self._canvas, self)
992         lay.addWidget(self._toolbar)
993         lay.addWidget(self._canvas, 1)
994
995         self.style_cb.currentIndexChanged.connect(self._redraw)
996         self._redraw()
997
998     def _redraw(self):
999         self._fig.clear()
1000         style = self.style_cb.currentText()
1001         if self._is_plan2:
1002             if style == "Heatmap (grid)" and self._can_heat:
1003                 z_col = self.z_cb.currentText() if self.z_cb else None
1004                 build_heatmap_plan2(self._fig, self._df, z_col)
1005             else:
1006                 build_results_plot_plan2(self._fig, self._df)
1007         else:
1008             if style == "Heatmap (grid)" and self._can_heat:
1009                 build_heatmap(self._fig, self._df)
1010             else:
1011                 build_results_plot(self._fig, self._df)
1012         self._canvas.draw()
1013
1014     def _save_png(self):
1015         path, _ = QFileDialog.getSaveFileName(
1016             self, "Save figure", self._save_base + ".png", "PNG image (*.png)"
1017         )
1018         if path:
1019             self._fig.savefig(path, dpi=150)
1020             QMessageBox.information(self, "Saved", f"Figure saved to {path}")
1021
1022
1023# -----
1024# Results panel
1025# -----
1026class ResultsPanel(QFrame):
1027    def __init__(self, parent=None):
1028        super().__init__(parent)
1029        self.setObjectName("panel")
1030        self._df = None
1031        self._prob_col = None
1032

```

```

1033         lay = QVBoxLayout(self)
1034
1035         toolbar = QHBoxLayout()
1036         title = QLabel("Results")
1037         title.setObjectName("section")
1038
1039         self.row_count = QLabel("")
1040         self.row_count.setObjectName("muted")
1041
1042         self.plot_btn = QPushButton("Plot")
1043         self.plot_btn.setEnabled(False)
1044
1045         self.export_btn = QPushButton("Export CSV")
1046         self.export_btn.setEnabled(False)
1047
1048         self.pdf_btn = QPushButton("Export PDF")
1049         self.pdf_btn.setEnabled(False)
1050
1051         self.oc_btn = QPushButton("OC Curve")
1052         # self.oc_btn.setObjectName("outline")
1053         self.oc_btn.setEnabled(False)
1054         self.oc_btn.setToolTip("OC curve: computed plan vs USP <905> (CU tabs)")
1055
1056         toolbar.addWidget(title)
1057         toolbar.addStretch(1)
1058         toolbar.addWidget(self.row_count)
1059         toolbar.addWidget(self.plot_btn)
1060         toolbar.addWidget(self.oc_btn)
1061         toolbar.addWidget(self.export_btn)
1062         toolbar.addWidget(self.pdf_btn)
1063         lay.addLayout(toolbar)
1064
1065         self.table = QTableWidgetItem()
1066         self.table.setAlternatingRowColors(True)
1067         self.table.setEditTriggers(QAbstractItemView.EditTrigger.NoEditTriggers)
1068         self.table.setSelectionBehavior(QAbstractItemView.SelectionBehavior.SelectRows)
1069
1070         self.table.verticalHeader().setVisible(False)
1071         self.table.setSortingEnabled(True)
1072         self.table.setToolTip("Click a header to sort | Ctrl+C copies selected rows")
1073
1074         lay.addWidget(self.table, 1)
1075
1076         self.export_btn.clicked.connect(self._export_csv)
1077         self.plot_btn.clicked.connect(self._show_plot)
1078         self.pdf_btn.clicked.connect(self._export_pdf)
1079         self.report_meta = None
1080         QShortcut(QKeySequence.Copy, self.table, activated=self._copy_selection)
1081
1082         # -- population -----
1083         def clear(self):
1084             self.table.clear()
1085             self.table.setRowCount(0)
1086             self.table.setColumnCount(0)
1087             self._df = None
1088             self._prob_col = None
1089             self.export_btn.setEnabled(False)
1090             self.plot_btn.setEnabled(False)
1091             self.pdf_btn.setEnabled(False)
1092             self.row_count.setText("")
1093
1094         def show_dataframe(self, df: pd.DataFrame):
1095             self.clear()
1096             self._df = df

```

```

1095     cols = [str(c) for c in df.columns]
1096
1097     self._prob_col = None
1098     for c in df.columns:
1099         if any(s in str(c).lower() for s in ("prob", "pass")):
1100             ser = pd.to_numeric(df[c], errors="coerce").dropna()
1101             if len(ser) and ser.max() <= 1.0:
1102                 self._prob_col = c
1103                 break
1104
1105     self.table.setSortingEnabled(False)
1106     self.table.setColumnCount(len(cols))
1107     self.table.setHorizontalHeaderLabels(cols)
1108     self.table.setRowCount(len(df))
1109
1110     for r, (_, row) in enumerate(df.iterrows()):
1111         for ci, c in enumerate(df.columns):
1112             v = row[c]
1113             item = NumItem(fmt_num(v, 4))
1114             if isinstance(v, (float, np.floating)):
1115                 item.setData(Qt.ItemDataRole.UserRole, float(v))
1116             if self._prob_col is not None and c == self._prob_col:
1117                 try:
1118                     pv = float(v)
1119                     if pv < 0.8:
1120                         item.setForeground(QBrush(QColor(ERR_RED)))
1121                     elif pv >= 0.9:
1122                         item.setForeground(QBrush(QColor(OK_GREEN)))
1123                 except Exception:
1124                     pass
1125             self.table.setItem(r, ci, item)
1126
1127     self.table.setSortingEnabled(True)
1128     self.table.resizeColumnsToContents()
1129
1130     self.export_btn.setEnabled(True)
1131     can_plot = HAVE_MPL and len(df) >= 2 and df.select_dtypes(include=[np.number]).shape[1] >= 2
1132     self.plot_btn.setEnabled(bool(can_plot))
1133     self.pdf_btn.setEnabled(bool(HAVE_PDF))
1134     self.row_count.setText(f"{len(df)} rows")
1135
1136     def show_dict(self, d: dict):
1137         self.clear()
1138         self._df = pd.DataFrame([d])
1139         self.table.setSortingEnabled(False)
1140         self.table.setColumnCount(2)
1141         self.table.setHorizontalHeaderLabels(["Field", "Value"])
1142         self.table.setRowCount(len(d))
1143         for r, (k, v) in enumerate(d.items()):
1144             self.table.setItem(r, 0, QTableWidgetItem(str(k)))
1145             self.table.setItem(r, 1, QTableWidgetItem(fmt_num(v, 6)))
1146         self.table.resizeColumnsToContents()
1147         self.export_btn.setEnabled(True)
1148         self.plot_btn.setEnabled(False)
1149         self.pdf_btn.setEnabled(bool(HAVE_PDF))
1150         self.row_count.setText("1 result")
1151
1152     # -- actions -----
1153     def _copy_selection(self):
1154         rows = sorted({i.row() for i in self.table.selectedItems()})
1155         if not rows:
1156             return

```

```

1157         lines = []
1158         for r in rows:
1159             lines.append(
1160                 "\t".join(
1161                     self.table.item(r, c).text() if self.table.item(r, c) else "
1162                 )
1163             )
1164         QApplication.clipboard().setText("\n".join(lines))
1165
1166     def _show_plot(self):
1167         if not HAVE_MPL:
1168             QMessageBox.critical(
1169                 self,
1170                 "Plot unavailable",
1171                 "matplotlib is required for plotting.\nInstall it with: pip ins
1172 tall matplotlib",
1173             )
1174             return
1175         if self._df is None:
1176             return
1177         PlotDialog(self._df, self, save_base=self._default_name(suffix="-
1178 plot")).exec()
1179
1180     def _export_csv(self):
1181         if self._df is None:
1182             return
1183         path, _ = QFileDialog.getSaveFileName(
1184             self, "Export CSV", self._default_name(ext=".csv"), "CSV files (*.cs
1185 v)"
1186         )
1187         if not path:
1188             return
1189         self._df.to_csv(path, index=False, quoting=csv.QUOTE_MINIMAL)
1190         QMessageBox.information(self, "Exported", f"Saved to {path}")
1191
1192     def _export_pdf(self):
1193         if not HAVE_PDF:
1194             QMessageBox.critical(
1195                 self,
1196                 "PDF export unavailable",
1197                 "reportlab is required.\nInstall it with: pip install reportlab
1198 ",
1199             )
1200             return
1201         if self._df is None:
1202             return
1203         path, _ = QFileDialog.getSaveFileName(
1204             self, "Export PDF", self._default_name(ext=".pdf"), "PDF files (*.pd
1205 f)"
1206         )
1207         if not path:
1208             return
1209         meta = self.report_meta or {"title": ["CuDAL RESULTS"]}
1210         try:
1211             write_sas_pdf(path, self._df, meta["title"])
1212             QMessageBox.information(self, "Exported", f"Saved to {path}")
1213         except Exception as exc: # noqa: BLE001
1214             QMessageBox.critical(self, "PDF export failed", str(exc))
1215
1216     def _default_name(self, suffix="", ext=""):
1217         tab = getattr(self, "tab", None)
1218         try:

```

```

1215         base = tab._export_base_name() if tab is not None else "results"
1216     except Exception:
1217         base = "results"
1218
1219     return base + suffix + ext
1220
1221
1222# -----
1223# Base tab
1224# -----
1225class BaseTab(QWidget):
1226    MODES = [
1227        ("table", "Acceptance limit table"),
1228        ("evaluate", "Probability of passing"),
1229        ("sample", "Sample probability"),
1230    ]
1231
1232    def __init__(self, title, subtitle, parent=None):
1233        super().__init__(parent)
1234        self.worker = None
1235        self.table_cache = {}
1236        self.field_registry = {m: {} for m, _ in self.MODES}
1237
1238        outer = QVBoxLayout(self)
1239        outer.setContentsMargins(16, 14, 16, 14)
1240
1241        hdr = QLabel(title)
1242        hdr.setObjectName("header")
1243        sub = QLabel(subtitle)
1244        sub.setObjectName("subheader")
1245        outer.addWidget(hdr)
1246        outer.addWidget(sub)
1247
1248        body = QHBoxLayout()
1249        outer.addLayout(body, 1)
1250
1251        # --- left: controls -----
1252        controls = QFrame()
1253        controls.setObjectName("panel")
1254        controls.setFixedWidth(360)
1255        cl = QVBoxLayout(controls)
1256        cl.setContentsMargins(12, 12, 12, 12)
1257        cl.setSpacing(10)
1258
1259        mode_box = QGroupBox("Analysis mode")
1260        mode_box.setObjectName("card")
1261        ml = QVBoxLayout(mode_box)
1262        ml.setContentsMargins(14, 16, 14, 12)
1263        ml.setSpacing(2)
1264        self.mode_group = QButtonGroup(self)
1265        self._mode_buttons = {}
1266        for val, label in self.MODES:
1267            rb = QRadioButton(label)
1268            rb.setProperty("mode", val)
1269            rb.setFixedHeight(26)
1270            self.mode_group.addButton(rb)
1271            self._mode_buttons[val] = rb
1272            ml.addWidget(rb)
1273        self._mode_buttons["table"].setChecked(True)
1274        self.mode_group.buttonClicked.connect(self._switch_mode)
1275        cl.addWidget(mode_box)
1276
1277        self.scroll = QScrollArea()

```

```

1278         self.scroll.setWidgetResizable(True)
1279         self.scroll.setFrameShape(QFrame.Shape.NoFrame)
1280         content = QWidget()
1281         content.setStyleSheet("background: transparent;")
1282         self._content_layout = QVBoxLayout(content)
1283         self._content_layout.setContentsMargins(0, 0, 0, 0)
1284         self._content_layout.setSpacing(12)
1285         self.mode_frames = {}
1286         for val, _ in self.MODES:
1287             gb = QGroupBox("Parameters")
1288             gb.setObjectName("card")
1289             g = QGridLayout()
1290             g.setContentsMargins(14, 20, 14, 14) # strict, uniform card padding
1291             g.setHorizontalSpacing(12)
1292             g.setVerticalSpacing(8)
1293             g.setColumnStretch(0, 1) # labels fill col 0...
1294             g.setColumnMinimumWidth(1, LabeledField.EDIT_WIDTH) # inputs align
in col 1
1295             gb.setLayout(g)
1296             self.mode_frames[val] = gb
1297             self._content_layout.addWidget(gb)
1298         self._content_layout.addStretch(1)
1299         self.scroll.setWidget(content)
1300         cl.addWidget(self.scroll, 1)
1301
1302         self._build_mode_frames()
1303
1304         run_row = QHBoxLayout()
1305
1306         self.run_btn = QPushButton("Run")
1307         self.run_btn.setObjectName("accent")
1308         self.run_btn.setToolTip("Run the selected analysis (Ctrl+R)")
1309         self.reset_btn = QPushButton("Reset")
1310         self.progress = QProgressBar()
1311         self.progress.setRange(0, 100)
1312         self.progress.setValue(0)
1313
1314         # uniform run row: progress bar as tall as the Run button
1315         for w in (self.run_btn, self.reset_btn, self.progress):
1316             w.setFixedHeight(34)
1317
1318         run_row.addWidget(self.run_btn)
1319         run_row.addWidget(self.reset_btn)
1320         run_row.addWidget(self.progress, 1)
1321         cl.addLayout(run_row)
1322
1323         self.status_label = QLabel("Ready.")
1324         self.status_label.setObjectName("muted")
1325         self.status_label.setWordWrap(True)
1326         cl.addWidget(self.status_label)
1327
1328         body.addWidget(controls)
1329
1330         # --- right: results -----
1331
1332         self.results = ResultsPanel()
1333         # self.results.tab = self
1334         self.results.tab = self
1335         body.addWidget(self.results, 1)
1336         self.results.oc_btn.clicked.connect(self._show_oc)
1337
1338         self.run_btn.clicked.connect(self._on_run)
1339         self.reset_btn.clicked.connect(self._reset_defaults)
1340         self._switch_mode()

```



```

1340
1341 # -- subclass hooks -----
1342
1343 def _build_mode_frames(self):
1344     raise NotImplementedError
1345
1346 def _run_table(self):
1347     raise NotImplementedError
1348
1349 def _run_evaluate(self):
1350     raise NotImplementedError
1351
1352 def _run_sample(self):
1353     raise NotImplementedError
1354
1355 @staticmethod
1356 def _cache_key(*args):
1357     norm = []
1358     for a in args:
1359         if isinstance(a, (list, tuple)):
1360             norm.append(tuple(round(float(x), 12) for x in a))
1361         else:
1362             norm.append(a)
1363     return tuple(norm)
1364
1365 # -- settings -----
1366
1367 def collect_state(self):
1368     return {
1369         "mode": self._current_mode(),
1370         "fields": {
1371             mode: {k: f.edit.text() for k, f in reg.items()}
1372             for mode, reg in self.field_registry.items()
1373         },
1374     }
1375
1376 def apply_state(self, state):
1377     if not isinstance(state, dict):
1378         return
1379     for mode, reg in self.field_registry.items():
1380         for k, field in reg.items():
1381             val = state.get("fields", {}).get(mode, {}).get(k)
1382             if val is not None:
1383                 field.edit.setText(str(val))
1384     mode = state.get("mode")
1385     if mode in self._mode_buttons:
1386         self._mode_buttons[mode].setChecked(True)
1387         self._switch_mode()
1388
1389 def _reset_defaults(self):
1390     for reg in self.field_registry.values():
1391         for field in reg.values():
1392             field.reset()
1393     self.status_label.setText("Parameters reset to defaults.")
1394
1395 # -- plumbing -----
1396
1397 def _current_mode(self):
1398     return self.mode_group.checkedButton().property("mode")
1399
1400 def _switch_mode(self, *_):
1401     mode = self._current_mode()
1402     for val, gb in self.mode_frames.items():
1403         gb.setVisible(val == mode)

```

```

1401
1402 def _on_run(self):
1403     if self._worker is not None and self._worker.isRunning():
1404         return
1405     try:
1406         job = {
1407             "table": self._run_table,
1408             "evaluate": self._run_evaluate,
1409             "sample": self._run_sample,
1410         }[self._current_mode]()
1411     except ValueError as exc:
1412         QMessageBox.critical(self, "Invalid input", str(exc))
1413         return
1414     except Exception as exc: # noqa: BLE001
1415         QMessageBox.critical(self, "Invalid input", str(exc))
1416         return
1417
1418     self.run_btn.setEnabled(False)
1419     self.progress.setValue(0)
1420     self.status_label.setText("Calculating...")
1421
1422     self._worker = Worker(job, self)
1423     self._worker.progress.connect(self._on_progress)
1424     self._worker.finished_ok.connect(self._on_done)
1425     self._worker.failed.connect(self._on_fail)
1426     self._worker.finished.connect(self._worker_finished)
1427     self._worker.start()
1428
1429 def _on_progress(self, frac, msg):
1430     self.progress.setValue(int(min(100.0, frac * 100)))
1431     if msg:
1432         self.status_label.setText(msg)
1433
1434 def _on_done(self, result):
1435     self.progress.setValue(100)
1436     if isinstance(result, pd.DataFrame):
1437         self.results.show_dataframe(result)
1438
1439         self.status_label.setText(f"Done -- {len(result)} row(s) computed.")
1440     else:
1441         self.results.show_dict(result)
1442         self.status_label.setText("Done.")
1443     self.results.report_meta = self._report_meta()
1444
1445     # in _on_done, after showing the results:
1446     self.results.oc_btn.setEnabled(self._oc_available())
1447
1448 def _on_fail(self, tb):
1449     self.progress.setValue(0)
1450     self.status_label.setText("Calculation failed -- see error dialog.")
1451     QMessageBox.critical(self, "Calculation error", tb)
1452
1453 def _worker_finished(self):
1454     self.run_btn.setEnabled(True)
1455     if self._worker is not None:
1456         self._worker.deleteLater()
1457         self._worker = None
1458
1459     # -- SAS-style PDF title block -----
1460 def _fv(self, *keys):
1461     for d in (getattr(self, "table_fields", {}), getattr(self, "sample_fields", {})):
1462         for k in keys:
1463             if k in d:

```

```

1464         try:
1465             return d[k].get(float)
1466         except Exception:
1467             return None
1468     return None
1469
1470 def _report_meta(self):
1471     name = type(self).__name__
1472     dom = "DISSOLUTION" if name.startswith("Disp") else "CONTENT UNIFORMITY"
1473     plan = 2 if name.endswith("2Tab") else 1
1474     n = self._fv("number", "num")
1475     loc = self._fv("loc")
1476     target = self._fv("target")
1477     q = self._fv("q")
1478     lbound = self._fv("lbound")
1479     cilevel = self._fv("cilevel")
1480     lines = []
1481     if plan == 1:
1482         key = f"TARGET = {target:.1f}" if target is not None else f"Q = {q:.1f}"
1483         lines.append(f"ACCEPTANCE LIMITS FOR {dom} (N= {n:.0f}, {key})")
1484         lines.append("SAMPLING PLAN 1")
1485         lines.append(
1486             f"(MEETING LIMITS GUARANTEES, WITH {cilevel:.1f}% ASSURANCE, THAT AT LEAST"
1487         )
1488         lines.append(f"{lbound:.1f}% OF SAMPLES TESTED FOR {dom} WILL PASS THE USP TEST)")
1489     else:
1490         lines.append(f"ACCEPTANCE LIMITS FOR {dom}")
1491         lines.append("SAMPLING PLAN 2")
1492         base = f"TARGET={target:.1f}" if target is not None else f"Q={q:.1f}"
1493         lines.append(f"{base}, LOWER BOUND = {lbound:.1f}, CONFIDENCE LEVEL = {cilevel:.1f}")
1494         lines.append("TABLE ENTRIES ARE LOWER(LL) AND UPPER(UL) LIMITS ON THE MEAN")
1495         if n is not None and loc is not None:
1496             lines.append(
1497                 f"OF {int(n * loc)} ASSAYS: {int(n)} ASSAYS AT EACH OF "
1498                 f"{int(loc)} DIFFERENT LOCATIONS"
1499             )
1500             lines.append("SE IS THE POOLED WITHIN LOCATION STANDARD DEVIATION")
1501             lines.append("STANDARD DEVIATIONS AND MEANS ARE EXPRESSED IN % CLAIM")
1502         if self._current_mode() != "table":
1503             lines.append(f"MODE: {dict(self.MODES)[self._current_mode()].upper()}")
1504     return {"title": lines}
1505
1506 def _export_base_name(self):
1507     name = type(self).__name__
1508     dom = "DISSOLUTION" if name.startswith("Disp") else "CONTENT UNIFORMITY"
1509     plan = 2 if name.endswith("2Tab") else 1
1510     method = ("CUSP" if "CONTENT" in dom else "DISP") + str(plan)
1511     v = getattr(self, "table_fields", {})
1512
1513     def g(key):
1514         try:
1515             return float(v[key].get(float))
1516         except Exception:
1517             return None
1518
1519     toks = [method]

```

```

1520     q = g("q")
1521     if q is not None:
1522         toks.append(f"Q{q:g}")
1523     lb, ci = g("lbound"), g("cilevel")
1524     if lb is not None and ci is not None:
1525         toks.append(f"{lb:g}x{ci:g}")
1526     if plan == 1:
1527         n = g("number")
1528         if n is not None:
1529             toks.append(f"{n:g}N")
1530     else:
1531         loc, num = g("loc"), g("num")
1532         if loc is not None and num is not None:
1533             toks.append(f"{loc:g}Lx{num:g}N")
1534     base = "-".join(toks)
1535     mode = self._current_mode()
1536     if mode == "evaluate":
1537         base += "-EVAL"
1538     elif mode == "sample":
1539         base += "-SAMPLE"
1540     return base
1541
1542     # new methods:
1543     def _oc_available(self):
1544         return False
1545
1546     def _oc_context(self):
1547         return None
1548
1549     def _show_oc(self):
1550         if not HAVE_MPL:
1551             QMessageBox.critical(
1552                 self, "Plot unavailable", "matplotlib is required.\n"
1553                 "pip install matplotlib"
1554             )
1555             return
1556         ctx = self._oc_context()
1557         if ctx is None:
1558             QMessageBox.information(self, "OC curve", "Not available for this scenario.")
1559             return
1560         OCDialog(ctx, self, save_base=self._export_base_name() + "-OC").exec()
1561
1562     def make_grid(low: float, high: float, step: float, name: str):
1563         if step <= 0:
1564             raise ValueError(f"{name} step must be positive")
1565         if high < low:
1566             raise ValueError(f"{name} high must be >= {name} low")
1567         n = int(round((high - low) / step)) + 1
1568         return [low + i * step for i in range(n)]
1569
1570
1571 # -----
1572 # Tabs
1573 # -----
1574 class CusplTab(BaseTab):
1575     def __init__(self, parent=None):
1576         super().__init__(
1577             "Content Uniformity -
1578             - Sampling Plan 1", "Single composite sample (USP <905>).", parent
1579         )
1580         self._last_table = None

```

```

1581 def _build_mode_frames(self):
1582     self.table_fields = build_form(
1583         self.mode_frames["table"].layout(),
1584         [
1585             ("number", "Number of units (N)", 10),
1586             ("target", "Target / label claim (%)", 100.0),
1587             ("lbound", "Lower bound (%)", 95.0),
1588             ("cilevel", "Confidence level (%)", 95.0),
1589             ("mean_low", "Mean grid low", 85.1),
1590             ("mean_high", "Mean grid high", 114.9),
1591             ("mean_step", "Mean grid step", 0.5),
1592         ],
1593         registry=self.field_registry["table"],
1594     )
1595
1596     lay = self.mode_frames["evaluate"].layout()
1597     desc = QLabel("Builds the table above, then evaluates:")
1598     desc.setObjectName("muted")
1599     desc.setWordWrap(True)
1600     lay.addWidget(desc, 0, 0, 1, 2)
1601     self.eval_fields = build_form(
1602         lay,
1603         [
1604             ("u_low", "True mean U -- low", 95.0),
1605             ("u_high", "True mean U -- high", 105.0),
1606             ("u_step", "True mean U -- step", 2.5),
1607             ("cv_low", "True CV(%) -- low", 1.0),
1608             ("cv_high", "True CV(%) -- high", 4.0),
1609             ("cv_step", "True CV(%) -- step", 1.0),
1610         ],
1611         start_row=1,
1612         registry=self.field_registry["evaluate"],
1613     )
1614     for k in ("number", "target", "lbound", "cilevel"):
1615         self.eval_fields[k] = self.table_fields[k]
1616
1617     self.sample_fields = build_form(
1618         self.mode_frames["sample"].layout(),
1619         [
1620             ("mean", "Sample mean (%)", 100.0),
1621             ("cv", "Sample CV (%)", 2.0),
1622             ("number", "Number of units (N)", 10),
1623             ("target", "Target / label claim (%)", 100.0),
1624             ("lbound", "Lower bound (%)", 95.0),
1625             ("cilevel", "Confidence level (%)", 95.0),
1626         ],
1627         registry=self.field_registry["sample"],
1628     )
1629
1630 def _run_table(self):
1631     v = self.table_fields
1632     number = v["number"].get(int)
1633     target = v["target"].get(float)
1634     lbound = v["lbound"].get(float)
1635     cilevel = v["cilevel"].get(float)
1636     mean_low = v["mean_low"].get(float)
1637     mean_high = v["mean_high"].get(float)
1638     mean_step = v["mean_step"].get(float)
1639     key = self._cache_key(number, target, lbound, cilevel, mean_low, mean_high, mean_step)
1640
1641     def job(progress):
1642         hit = self._table_cache.get(key)
1643         if hit is not None:

```

```

1644         progress(0.5, "Using cached table...")
1645         return hit
1646     progress(0.2, "Computing acceptance table...")
1647     table = cuspl.acceptance_limit_table(
1648         number, target, lbound, cilevel, mean_low, mean_high, mean_step
1649     )
1650     self._table_cache[key] = table
1651     self._last_table = table
1652     progress(1.0, "Table complete.")
1653     return table
1654
1655     return job
1656
1657 def _run_evaluate(self):
1658     v = self.table_fields
1659     number = v["number"].get(int)
1660     target = v["target"].get(float)
1661     lbound = v["lbound"].get(float)
1662     cilevel = v["cilevel"].get(float)
1663     u_vals = make_grid(
1664         self.eval_fields["u_low"].get(float),
1665         self.eval_fields["u_high"].get(float),
1666         self.eval_fields["u_step"].get(float),
1667         "U",
1668     )
1669     cv_vals = make_grid(
1670         self.eval_fields["cv_low"].get(float),
1671         self.eval_fields["cv_high"].get(float),
1672         self.eval_fields["cv_step"].get(float),
1673         "CV",
1674     )
1675     key = self._cache_key(number, target, lbound, cilevel)
1676
1677     def job(progress):
1678         table = self._table_cache.get(key)
1679         if table is None:
1680             progress(0.2, "Building acceptance table...")
1681             table = cuspl.acceptance_limit_table(number, target, lbound, cil
evel)
1682             self._table_cache[key] = table
1683             self._last_table = table
1684         else:
1685             progress(0.2, "Using cached table...")
1686             progress(0.6, "Evaluating probability grid...")
1687             return cuspl.probability_of_passing(table, number, u_vals, cv_vals)
1688
1689     return job
1690
1691 def _run_sample(self):
1692     v = self.sample_fields
1693     mean = v["mean"].get(float)
1694     cv = v["cv"].get(float)
1695     number = v["number"].get(int)
1696     target = v["target"].get(float)
1697     lbound = v["lbound"].get(float)
1698     cilevel = v["cilevel"].get(float)
1699
1700     def job(progress):
1701         progress(0.4, "Computing sample probability...")
1702         return cuspl.sample_probability(mean, cv, number, target, lbound, ci
level)
1703
1704     return job
1705

```

```

1706 def _oc_available(self):
1707     return True
1708
1709 def _oc_context(self):
1710     v = self.table_fields
1711     number = v["number"].get(int)
1712     target = v["target"].get(float)
1713     lbound = v["lbound"].get(float)
1714     cilevel = v["cilevel"].get(float)
1715     table = (
1716         self._last_table
1717         if self._last_table is not None
1718         else cuspl.acceptance_limit_table(number, target, lbound, cilevel)
1719     )
1720
1721     def computed(xk, xs, fx):
1722         if xk == "cv":
1723             res = cuspl.probability_of_passing(table, number, [fx["U"]], [float(x) for x in xs])
1724         else:
1725             res = cuspl.probability_of_passing(
1726                 table, number, [float(x) for x in xs], [fx["CV"]]
1727             )
1728         return _prob_series(res)
1729
1730     def make_units(xk, x, fx, rng, reps):
1731         U = x if xk == "u" else fx["U"]
1732         CV = x if xk == "cv" else fx["CV"]
1733         return rng.normal(U, U * CV / 100.0, (reps, 30))
1734
1735     return make_oc_context(
1736         "cu",
1737         target,
1738         computed,
1739         make_units,
1740         [("cv", "True CV (%) [U fixed]"), ("u", "True mean U (%) [CV fixed]"]],
1741         {"cv": (0.5, 10.0, 0.25), "u": (85.0, 115.0, 1.0)},
1742         {"cv": [("U", target)], "u": [("CV", 2.0)]},
1743     )
1744
1745
1746 class Cusp2Tab(BaseTab):
1747     def __init__(self, parent=None):
1748         super().__init__(
1749             "Content Uniformity -- Sampling Plan 2",
1750             "Multiple locations, within/between-
location variance components (USP <905>).",
1751             parent,
1752         )
1753         self._last_table = None
1754
1755     def _build_mode_frames(self):
1756         self.table_fields = build_form(
1757             self.mode_frames["table"].layout(),
1758             [
1759                 ("num", "Units per location", 6),
1760                 ("loc", "Number of locations", 10),
1761                 ("target", "Target / label claim (%)", 100.0),
1762                 ("lbound", "Lower bound (%)", 95.0),
1763                 ("cilevel", "Confidence level (%)", 95.0),
1764                 ("se_low", "Within-loc SD -- low", 0.5),
1765                 ("se_high", "Within-loc SD -- high", 4.0),
1766                 ("se_step", "Within-loc SD -- step", 0.5),

```

```

1767         ("sm_low", "Between-loc SD -- low", 0.5),
1768         ("sm_high", "Between-loc SD -- high", 4.0),
1769         ("sm_step", "Between-loc SD -- step", 0.5),
1770     ],
1771     registry=self.field_registry["table"],
1772 )
1773
1774 lay = self.mode_frames["evaluate"].layout()
1775 desc = QLabel("Builds the table above, then evaluates:")
1776 desc.setObjectName("muted")
1777 desc.setWordWrap(True)
1778 lay.addWidget(desc, 0, 0, 1, 2)
1779 self.eval_fields = build_form(
1780     lay,
1781     [
1782         ("u_low", "True mean U -- low", 95.0),
1783         ("u_high", "True mean U -- high", 105.0),
1784         ("u_step", "True mean U -- step", 2.5),
1785         ("sigse_low", "True within-loc SD -- low", 1.0),
1786         ("sigse_high", "True within-loc SD -- high", 3.0),
1787         ("sigse_step", "True within-loc SD -- step", 1.0),
1788         ("sigsm_low", "True between-loc SD -- low", 1.0),
1789         ("sigsm_high", "True between-loc SD -- high", 3.0),
1790         ("sigsm_step", "True between-loc SD -- step", 1.0),
1791     ],
1792     start_row=1,
1793     registry=self.field_registry["evaluate"],
1794 )
1795
1796 self.sample_fields = build_form(
1797     self.mode_frames["sample"].layout(),
1798     [
1799         ("mean", "Sample mean (%)", 100.0),
1800         ("se", "Sample within-loc SD", 2.2),
1801         ("sm", "Sample between-loc SD", 2.46),
1802         ("num", "Units per location", 6),
1803         ("loc", "Number of locations", 10),
1804         ("target", "Target / label claim (%)", 100.0),
1805         ("cilevel", "Confidence level (%)", 95.0),
1806     ],
1807     registry=self.field_registry["sample"],
1808 )
1809
1810 def _table_args(self):
1811     v = self.table_fields
1812     num = v["num"].get(int)
1813     loc = v["loc"].get(int)
1814     target = v["target"].get(float)
1815     lbound = v["lbound"].get(float)
1816     cilevel = v["cilevel"].get(float)
1817     se_vals = make_grid(
1818         v["se_low"].get(float), v["se_high"].get(float), v["se_step"].get(fl
1819 oat), "SE"
1820 )
1821     sm_vals = make_grid(
1822         v["sm_low"].get(float), v["sm_high"].get(float), v["sm_step"].get(fl
1823 oat), "SM"
1824 )
1825     return num, loc, target, lbound, cilevel, se_vals, sm_vals
1826
1827 def _run_table(self):
1828     num, loc, target, lbound, cilevel, se_vals, sm_vals = self._table_args()
1829     key = self._cache_key(num, loc, target, lbound, cilevel, se_vals, sm_val
1830 s)

```



```

1828         self._last_table = None
1829
1830     def job(progress):
1831         hit = self._table_cache.get(key)
1832         if hit is not None:
1833             progress(0.5, "Using cached table...")
1834             return hit
1835         progress(0.2, "Computing acceptance table (Plan 2)...")
1836         table = cusp2.acceptance_limit_table(
1837             num, loc, target, lbound, cilevel, se_vals, sm_vals
1838         )
1839         self._table_cache[key] = table
1840         self._last_table = table
1841         progress(1.0, "Table complete.")
1842         return table
1843
1844     return job
1845
1846 def _run_evaluate(self):
1847     num, loc, target, lbound, cilevel, se_vals, sm_vals = self._table_args()
1848     u_vals = make_grid(
1849         self.eval_fields["u_low"].get(float),
1850         self.eval_fields["u_high"].get(float),
1851         self.eval_fields["u_step"].get(float),
1852         "U",
1853     )
1854     sigse_vals = make_grid(
1855         self.eval_fields["sigse_low"].get(float),
1856         self.eval_fields["sigse_high"].get(float),
1857         self.eval_fields["sigse_step"].get(float),
1858         "within-loc SD",
1859     )
1860     sigsm_vals = make_grid(
1861         self.eval_fields["sigsm_low"].get(float),
1862         self.eval_fields["sigsm_high"].get(float),
1863         self.eval_fields["sigsm_step"].get(float),
1864         "between-loc SD",
1865     )
1866     key = self._cache_key(num, loc, target, lbound, cilevel, se_vals, sm_val
s)
1867
1868     def job(progress):
1869         table = self._table_cache.get(key)
1870         if table is None:
1871             progress(0.2, "Building acceptance table (Plan 2)...")
1872             table = cusp2.acceptance_limit_table(
1873                 num, loc, target, lbound, cilevel, se_vals, sm_vals
1874             )
1875             self._last_table = table
1876             self._table_cache[key] = table
1877             self._last_table = table
1878         else:
1879             progress(0.2, "Using cached table...")
1880             d1 = se_vals[1] - se_vals[0] if len(se_vals) > 1 else 0.1
1881             progress(0.6, "Evaluating probability grid...")
1882             return cusp2.probability_of_passing(table, num, loc, d1, u_vals, sig
se_vals, sigsm_vals)
1883
1884     return job
1885
1886 def _run_sample(self):
1887     v = self.sample_fields
1888     mean = v["mean"].get(float)
1889     se = v["se"].get(float)

```

```

1890         sm = v["sm"].get(float)
1891         num = v["num"].get(int)
1892         loc = v["loc"].get(int)
1893         target = v["target"].get(float)
1894         cilevel = v["cilevel"].get(float)
1895
1896         def job(progress):
1897             progress(0.4, "Computing sample probability...")
1898             return cusp2.sample_probability(mean, se, sm, num, loc, target, cilevel)
1899
1900         return job
1901
1902     def _oc_available(self):
1903         return True
1904
1905     def _oc_context(self):
1906         v = self.table_fields
1907         num, loc = v["num"].get(int), v["loc"].get(int)
1908         target = v["target"].get(float)
1909         lbound = v["lbound"].get(float)
1910         cilevel = v["cilevel"].get(float)
1911         se_vals = make_grid(
1912             v["se_low"].get(float), v["se_high"].get(float), v["se_step"].get(float), "SE"
1913         )
1914         sm_vals = make_grid(
1915             v["sm_low"].get(float), v["sm_high"].get(float), v["sm_step"].get(float), "SM"
1916         )
1917         dl = se_vals[1] - se_vals[0] if len(se_vals) > 1 else 0.1
1918         table = (
1919             self._last_table
1920             if self._last_table is not None
1921             else cusp2.acceptance_limit_table(num, loc, target, lbound, cilevel, se_vals, sm_vals)
1922         )
1923
1924     def computed(xk, xs, fx):
1925         U = [float(x) for x in xs] if xk == "u" else [fx["U"]]
1926         SE = [float(x) for x in xs] if xk == "se" else [fx["SE"]]
1927         return prob_series(
1928             cusp2.probability_of_passing(table, num, loc, dl, U, SE, [fx["SM"]])
1929         )
1930
1931     def make_units(xk, x, fx, rng, reps):
1932         U = x if xk == "u" else fx["U"]
1933         SE = x if xk == "se" else fx["SE"]
1934         return U + rng.normal(0.0, fx["SM"], (reps, 1)) + rng.normal(0.0, SE, (reps, 30))
1935
1936     return make_oc_context(
1937         "cu",
1938         target,
1939         computed,
1940         make_units,
1941         [("se", "True within-loc SD [U, SM fixed]"), ("u", "True mean U [SE, SM fixed]")],
1942         {"se": (0.5, 10.0, 0.25), "u": (85.0, 115.0, 1.0)},
1943         {"se": [("U", target), ("SM", 2.2)], "u": [("SE", 2.2), ("SM", 2.2)]},
1944     )
1945

```

```

1946
1947class DisplTab(BaseTab):
1948    def __init__(self, parent=None):
1949        super().__init__("Dissolution -
- Sampling Plan 1", "Single location (USP <711>).", parent)
1950        self._last_table = None
1951
1952    def _build_mode_frames(self):
1953        self.table_fields = build_form(
1954            self.mode_frames["table"].layout(),
1955            [
1956                ("number", "Number of units (N)", 6),
1957                ("q", "Q value (%)", 80.0),
1958                ("lbound", "Lower bound (%)", 95.0),
1959                ("cilevel", "Confidence level (%)", 95.0),
1960                ("meanadj_step", "Mean grid step", 1.0),
1961            ],
1962            registry=self.field_registry["table"],
1963        )
1964
1965        lay = self.mode_frames["evaluate"].layout()
1966        desc = QLabel("Builds the table above, then evaluates:")
1967        desc.setObjectName("muted")
1968        desc.setWordWrap(True)
1969        lay.addWidget(desc, 0, 0, 1, 2)
1970        self.eval_fields = build_form(
1971            lay,
1972            [
1973                ("u_low", "True mean U -- low", 90.0),
1974                ("u_high", "True mean U -- high", 100.0),
1975                ("u_step", "True mean U -- step", 2.5),
1976                ("cv_low", "True CV(%) -- low", 1.0),
1977                ("cv_high", "True CV(%) -- high", 4.0),
1978                ("cv_step", "True CV(%) -- step", 1.0),
1979            ],
1980            start_row=1,
1981            registry=self.field_registry["evaluate"],
1982        )
1983
1984        self.sample_fields = build_form(
1985            self.mode_frames["sample"].layout(),
1986            [
1987                ("mean", "Sample mean (%)", 90.0),
1988                ("cv", "Sample CV (%)", 3.0),
1989                ("number", "Number of units (N)", 6),
1990                ("q", "Q value (%)", 80.0),
1991                ("cilevel", "Confidence level (%)", 95.0),
1992            ],
1993            registry=self.field_registry["sample"],
1994        )
1995
1996    def _run_table(self):
1997        v = self.table_fields
1998        number = v["number"].get(int)
1999        q = v["q"].get(float)
2000        lbound = v["lbound"].get(float)
2001        cilevel = v["cilevel"].get(float)
2002        meanadj_step = v["meanadj_step"].get(float)
2003        key = self._cache_key(number, q, lbound, cilevel, meanadj_step)
2004
2005        def job(progress):
2006            hit = self._table_cache.get(key)
2007            if hit is not None:
2008                progress(0.5, "Using cached table...")

```

```

2009         return hit
2010         progress(0.2, "Computing acceptance table...")
2011         table = displ.acceptance_limit_table(number, q, lbound, cilevel, mea
nadj_step)
2012         self._table_cache[key] = table
2013         self._last_table = table
2014         progress(1.0, "Table complete.")
2015         return table
2016
2017     return job
2018
2019     def _run_evaluate(self):
2020         v = self.table_fields
2021         number = v["number"].get(int)
2022         q = v["q"].get(float)
2023         lbound = v["lbound"].get(float)
2024         cilevel = v["cilevel"].get(float)
2025         u_vals = make_grid(
2026             self.eval_fields["u_low"].get(float),
2027             self.eval_fields["u_high"].get(float),
2028             self.eval_fields["u_step"].get(float),
2029             "U",
2030         )
2031         cv_vals = make_grid(
2032             self.eval_fields["cv_low"].get(float),
2033             self.eval_fields["cv_high"].get(float),
2034             self.eval_fields["cv_step"].get(float),
2035             "CV",
2036         )
2037         key = self._cache_key(number, q, lbound, cilevel)
2038
2039         def job(progress):
2040             table = self._table_cache.get(key)
2041             if table is None:
2042                 progress(0.2, "Building acceptance table...")
2043                 table = displ.acceptance_limit_table(number, q, lbound, cilevel)
2044                 self._table_cache[key] = table
2045                 self._last_table = table
2046             else:
2047                 progress(0.2, "Using cached table...")
2048                 progress(0.6, "Evaluating probability grid...")
2049                 return displ.probability_of_passing(table, number, u_vals, cv_vals)
2050
2051         return job
2052
2053     def _run_sample(self):
2054         v = self.sample_fields
2055         mean = v["mean"].get(float)
2056         cv = v["cv"].get(float)
2057         number = v["number"].get(int)
2058         q = v["q"].get(float)
2059         cilevel = v["cilevel"].get(float)
2060
2061         def job(progress):
2062             progress(0.4, "Computing sample probability...")
2063             return displ.sample_probability(mean, cv, number, q, cilevel)
2064
2065         return job
2066
2067     def _oc_available(self):
2068         return True
2069
2070     def _oc_context(self):
2071         v = self.table_fields

```

```

2072         number = v["number"].get(int)
2073         q = v["q"].get(float)
2074         lbound = v["lbound"].get(float)
2075         cilevel = v["cilevel"].get(float)
2076         table = (
2077             self._last_table
2078             if self._last_table is not None
2079             else displ.acceptance_limit_table(number, q, lbound, cilevel)
2080         )
2081
2082         def computed(xk, xs, fx):
2083             if xk == "cv":
2084                 res = displ.probability_of_passing(table, number, [fx["U"]], [float(x) for x in xs])
2085             else:
2086                 res = displ.probability_of_passing(
2087                     table, number, [float(x) for x in xs], [fx["CV"]]
2088                 )
2089             return _prob_series(res)
2090
2091         def make_units(xk, x, fx, rng, reps):
2092             U = x if xk == "u" else fx["U"]
2093             CV = x if xk == "cv" else fx["CV"]
2094             return rng.normal(U, U * CV / 100.0, (reps, 24))
2095
2096         return make_oc_context(
2097             "disp",
2098             q,
2099             computed,
2100             make_units,
2101             [("cv", "True CV (%) [U fixed]"), ("u", "True mean U (%) [CV fixed]")],
2102             {"cv": (0.5, 25.0, 0.5), "u": (70.0, 120.0, 1.0)},
2103             {"cv": [("U", 100.0)], "u": [("CV", 3.0)]},
2104         )
2105
2106
2107 class Disp2Tab(BaseTab):
2108     def __init__(self, parent=None):
2109         super().__init__(
2110             "Dissolution -- Sampling Plan 2",
2111             "Multiple locations, within/between-
location variance components (USP <711>).",
2112             parent,
2113         )
2114         self._last_table = None
2115
2116     def _build_mode_frames(self):
2117         self.table_fields = build_form(
2118             self.mode_frames["table"].layout(),
2119             [
2120                 ("num", "Units per location", 6),
2121                 ("loc", "Number of locations", 5),
2122                 ("q", "Q value (%)", 80.0),
2123                 ("lbound", "Lower bound (%)", 95.0),
2124                 ("cilevel", "Confidence level (%)", 95.0),
2125                 ("se_low", "Within-loc SD -- low", 1.0),
2126                 ("se_high", "Within-loc SD -- high", 5.0),
2127                 ("se_step", "Within-loc SD -- step", 1.0),
2128                 ("sm_low", "Between-loc SD -- low", 1.0),
2129                 ("sm_high", "Between-loc SD -- high", 5.0),
2130                 ("sm_step", "Between-loc SD -- step", 1.0),
2131             ],
2132             registry=self.field_registry["table"],

```

```

2133     )
2134
2135     lay = self.mode_frames["evaluate"].layout()
2136     desc = QLabel("Builds the table above, then evaluates:")
2137     desc.setObjectName("muted")
2138     desc.setWordWrap(True)
2139     lay.addWidget(desc, 0, 0, 1, 2)
2140     self.eval_fields = build_form(
2141         lay,
2142         [
2143             ("u_low", "True mean U -- low", 90.0),
2144             ("u_high", "True mean U -- high", 100.0),
2145             ("u_step", "True mean U -- step", 2.5),
2146             ("sigse_low", "True within-loc SD -- low", 1.0),
2147             ("sigse_high", "True within-loc SD -- high", 3.0),
2148             ("sigse_step", "True within-loc SD -- step", 1.0),
2149             ("sigsm_low", "True between-loc SD -- low", 1.0),
2150             ("sigsm_high", "True between-loc SD -- high", 3.0),
2151             ("sigsm_step", "True between-loc SD -- step", 1.0),
2152         ],
2153         start_row=1,
2154         registry=self.field_registry["evaluate"],
2155     )
2156
2157     self.sample_fields = build_form(
2158         self.mode_frames["sample"].layout(),
2159         [
2160             ("mean", "Sample mean (%)", 90.0),
2161             ("se", "Sample within-loc SD", 2.2),
2162             ("sm", "Sample between-loc SD", 2.46),
2163             ("num", "Units per location", 6),
2164             ("loc", "Number of locations", 5),
2165             ("q", "Q value (%)", 80.0),
2166             ("cilevel", "Confidence level (%)", 95.0),
2167         ],
2168         registry=self.field_registry["sample"],
2169     )
2170
2171     def _table_args(self):
2172         v = self.table_fields
2173         num = v["num"].get(int)
2174         loc = v["loc"].get(int)
2175         q = v["q"].get(float)
2176         lbound = v["lbound"].get(float)
2177         cilevel = v["cilevel"].get(float)
2178         se_vals = make_grid(
2179             v["se_low"].get(float), v["se_high"].get(float), v["se_step"].get(fl
2180 oat), "SE"
2181         )
2182         sm_vals = make_grid(
2183             v["sm_low"].get(float), v["sm_high"].get(float), v["sm_step"].get(fl
2184 oat), "SM"
2185         )
2186         return num, loc, q, lbound, cilevel, se_vals, sm_vals
2187
2188     def _run_table(self):
2189         num, loc, q, lbound, cilevel, se_vals, sm_vals = self._table_args()
2190         key = self._cache_key(num, loc, q, lbound, cilevel, se_vals, sm_vals)
2191
2192         def job(progress):
2193             hit = self._table_cache.get(key)
2194             if hit is not None:
2195                 progress(0.5, "Using cached table...")
2196             return hit

```

```

2195         progress(0.2, "Computing acceptance table (Plan 2)...")
2196         table = disp2.acceptance_limit_table(num, loc, q, lbound, cilevel, s
e_vals, sm_vals)
2197         self._last_table = table
2198         self._table_cache[key] = table
2199         progress(1.0, "Table complete.")
2200         return table
2201
2202     return job
2203
2204     def _run_evaluate(self):
2205         num, loc, q, lbound, cilevel, se_vals, sm_vals = self._table_args()
2206         dse = se_vals[1] - se_vals[0] if len(se_vals) > 1 else 1.0
2207         dsm = sm_vals[1] - sm_vals[0] if len(sm_vals) > 1 else 1.0
2208         u_vals = make_grid(
2209             self.eval_fields["u_low"].get(float),
2210             self.eval_fields["u_high"].get(float),
2211             self.eval_fields["u_step"].get(float),
2212             "U",
2213         )
2214         sigse_vals = make_grid(
2215             self.eval_fields["sigse_low"].get(float),
2216             self.eval_fields["sigse_high"].get(float),
2217             self.eval_fields["sigse_step"].get(float),
2218             "within-loc SD",
2219         )
2220         sigsm_vals = make_grid(
2221             self.eval_fields["sigsm_low"].get(float),
2222             self.eval_fields["sigsm_high"].get(float),
2223             self.eval_fields["sigsm_step"].get(float),
2224             "between-loc SD",
2225         )
2226         key = self._cache_key(num, loc, q, lbound, cilevel, se_vals, sm_vals)
2227
2228         def job(progress):
2229             table = self._table_cache.get(key)
2230             if table is None:
2231                 progress(0.2, "Building acceptance table (Plan 2)...")
2232                 table = disp2.acceptance_limit_table(num, loc, q, lbound, cileve
l, se_vals, sm_vals)
2233                 self._table_cache[key] = table
2234                 self._last_table = table
2235             else:
2236                 progress(0.2, "Using cached table...")
2237                 progress(0.6, "Evaluating probability grid...")
2238                 return disp2.probability_of_passing(
2239                     table, num, loc, dse, dsm, u_vals, sigse_vals, sigsm_vals
2240                 )
2241
2242         return job
2243
2244     def _run_sample(self):
2245         v = self.sample_fields
2246         mean = v["mean"].get(float)
2247         se = v["se"].get(float)
2248         sm = v["sm"].get(float)
2249         num = v["num"].get(int)
2250         loc = v["loc"].get(int)
2251         q = v["q"].get(float)
2252         cilevel = v["cilevel"].get(float)
2253
2254         def job(progress):
2255             progress(0.4, "Computing sample probability...")
2256             return disp2.sample_probability(mean, se, sm, num, loc, q, cilevel)

```

```

2257
2258         return job
2259
2260     def _oc_available(self):
2261         return True
2262
2263     def _oc_context(self):
2264         v = self.table_fields
2265         num, loc = v["num"].get(int), v["loc"].get(int)
2266         q = v["q"].get(float)
2267         lbound = v["lbound"].get(float)
2268         cilevel = v["cilevel"].get(float)
2269         se_vals = make_grid(
2270             v["se_low"].get(float), v["se_high"].get(float), v["se_step"].get(fl
2271 oat), "SE"
2272 )
2273         sm_vals = make_grid(
2274             v["sm_low"].get(float), v["sm_high"].get(float), v["sm_step"].get(fl
2275 oat), "SM"
2276 )
2277         dse = se_vals[1] - se_vals[0] if len(se_vals) > 1 else 1.0
2278         dsm = sm_vals[1] - sm_vals[0] if len(sm_vals) > 1 else 1.0
2279         table = (
2280             self._last_table
2281             if self._last_table is not None
2282             else disp2.acceptance_limit_table(num, loc, q, lbound, cilevel, se_v
2283 als, sm_vals)
2284         )
2285
2286     def computed(xk, xs, fx):
2287         U = [float(x) for x in xs] if xk == "u" else [fx["U"]]
2288         SE = [float(x) for x in xs] if xk == "se" else [fx["SE"]]
2289         return _prob_series(
2290             disp2.probability_of_passing(table, num, loc, dse, dsm, U, SE, [
2291 fx["SM"]])
2292         )
2293
2294     def make_units(xk, x, fx, rng, reps):
2295         U = x if xk == "u" else fx["U"]
2296         SE = x if xk == "se" else fx["SE"]
2297         return U + rng.normal(0.0, fx["SM"], (reps, 1)) + rng.normal(0.0, SE
2298 , (reps, 24))
2299
2300     return make_oc_context(
2301         "disp",
2302         q,
2303         computed,
2304         make_units,
2305         [("se", "True within-
2306 loc SD [U, SM fixed]"), ("u", "True mean U [SE, SM fixed]")],
2307         {"se": (0.5, 25.0, 0.5), "u": (70.0, 120.0, 2.0)},
2308         {"se": [("U", 100.0), ("SM", 2.2)], "u": [("SE", 2.2), ("SM", 2.2)]}
2309     ),
2310 )
2311
2312 # -----
2313 # OC-curve engine (unified): Monte-Carlo probability of passing the
2314 # compendial test itself -- USP <905> (2 stages) or USP <711> (3 stages).
2315 # -----
2316
2317 def _oc_cu_pass(units: np.ndarray, target: float) -> float:
2318     """Two-stage USP <905> decision. units: (reps, 30) -> P(pass)."""
2319     """
2320     This checks for an absolute shift of 25.0 units rather than 25% of $M$.

```



```

2314 - If $M = 98.5$, the lower bound should be $73.875$ ($98.5 \times 0.75$), meaning a deviation of at most $24.625$.
2315 - code allows a deviation up to $25.0$ (down to $73.5$), falsely passing extreme outliers.
2316 - If $M = 101.5$, the upper bound should be $126.875$, meaning a deviation up to $25.375$.
2317 - code caps it strictly at $25.0$ ($126.5$), falsely failing valid units.
2318 """
2319 passed = np.zeros(units.shape[0], dtype=bool)
2320 hi = target if target > 101.5 else 101.5
2321
2322 def M(m):
2323     return np.where(m <= 100.0, np.maximum(98.5, m), np.minimum(hi, m))
2324
2325 # --- Stage 1 (10 units) ---
2326 x1 = units[:, :10]
2327 m1, s1 = x1.mean(axis=1), x1.std(axis=1, ddof=1)
2328
2329 p1 = (np.abs(M(m1) - m1) + 2.4 * s1) <= 15.0
2330 passed |= p1
2331
2332 # --- Stage 2 (30 units) ---
2333 live = np.where(~p1)[0]
2334 if live.size:
2335     x30 = units[live]
2336     m2, s2 = x30.mean(axis=1), x30.std(axis=1, ddof=1)
2337     M2 = M(m2)
2338
2339     av_ok = (np.abs(M2 - m2) + 2.0 * s2) <= 15.0
2340     # FIXED: 0.25 * M2 instead of hardcoded 25.0
2341     within_ok = np.abs(x30 - M2[:, None]).max(axis=1) <= (0.25 * M2)
2342
2343     passed[live[av_ok & within_ok]] = True
2344
2345 return float(passed.mean())
2346
2347
2348 def _oc_disp_pass(units, q):
2349     """Three-stage USP <711> decision. units: (reps, 24) -> P(pass)."""
2350     passed = np.zeros(units.shape[0], dtype=bool)
2351
2352     x6 = units[:, :6] # Stage 1: all >= Q+5
2353     p = np.all(x6 >= q + 5.0, axis=1)
2354     passed |= p
2355     live = np.where(~p)[0]
2356
2357     if live.size: # Stage 2: 12 units
2358         x12 = units[live][:, :12]
2359         # FIXED: Changed > to >= for Q-15 boundary
2360         p = (x12.mean(axis=1) >= q) & np.all(x12 >= q - 15.0, axis=1)
2361         passed[live[p]] = True
2362         live = live[~p]
2363
2364     if live.size: # Stage 3: 24 units
2365         x24 = units[live][:, :24]
2366         ok_mean = x24.mean(axis=1) >= q
2367         # FIXED: Changed <= to < to match "less than Q-15%"
2368         n_l15 = (x24 < q - 15.0).sum(axis=1)
2369         # FIXED: Changed <= to < to match "less than Q-25%"
2370         any_l25 = (x24 < q - 25.0).any(axis=1)
2371
2372         p = ok_mean & (n_l15 <= 2) & ~any_l25
2373         passed[live[p]] = True
2374

```

```

2375     return float(passed.mean())
2376
2377
2378 def _prob_series(df):
2379     for c in df.columns:
2380         if any(s in str(c).lower() for s in ("prob", "pass", "ptrap")):
2381             return pd.to_numeric(df[c], errors="coerce").to_numpy(dtype=float)
2382     return pd.to_numeric(df.iloc[:, -1], errors="coerce").to_numpy(dtype=float)
2383
2384
2385 def make_oc_context(test, ref, computed, make_units, x_choices, grid, fixed_specs):
2386     """Single factory for every tab's OC context.
2387
2388     test      : "cu"  -> USP <905> two-stage decision, ref = target
2389                 "disp"-> USP <711> three-stage decision, ref = Q
2390     computed  : fn(xk, xs, fx) -> analytic P(pass table) (per-tab)
2391     make_units: fn(xk, x, fx, rng, reps) -> (reps, n_units) simulated units
2392     """
2393     decision = _oc_cu_pass if test == "cu" else _oc_disp_pass
2394
2395     def usp(xk, xs, fx, reps):
2396         out = []
2397         for x in xs:
2398             rng = np.random.default_rng(12345) # seeded -> reproducible
2399             out.append(decision(make_units(xk, float(x), fx, rng, reps), ref))
2400         return np.array(out)
2401
2402     return {
2403         "test": test,
2404         "x_choices": x_choices,
2405         "grid": grid,
2406         "fixed_specs": fixed_specs,
2407         "computed": computed,
2408         "usp": usp,
2409     }
2410
2411
2412 class OCDialog(QDialog):
2413     """OC curve: computed acceptance-limit plan vs the USP <905> test itself."""
2414
2415     def __init__(self, ctx, parent=None, save_base=None):
2416         super().__init__(parent)
2417         self.ctx = ctx
2418         self.usp_label = (
2419             "USP <905> two-stage test (Monte Carlo)"
2420             if ctx["test"] == "cu"
2421             else "USP <711> three-stage test (Monte Carlo)"
2422         )
2423         self._title = "OC Curve -- computed plan vs " + (
2424             "USP <905>" if ctx["test"] == "cu" else "USP <711>"
2425         )
2426         self.setWindowTitle(self._title)
2427         self.resize(900, 640)
2428         self.setModal(True)
2429         self._save_base = save_base or "oc"
2430
2431         lay = QVBoxLayout(self)
2432         top = QGridLayout()
2433         top.addWidget(QLabel("X axis:"), 0, 0)
2434         self.x_cb = QComboBox()
2435         for key, label in ctx["x_choices"]:
2436             self.x_cb.addItem(label, key)
2437         top.addWidget(self.x_cb, 0, 1)

```

```

2438         top.addWidget(QLabel("low/high/step:"), 0, 2)
2439         self.g_lo, self.g_hi, self.g_st = (QLineEdit() for _ in range(3))
2440         for w in (self.g_lo, self.g_hi, self.g_st):
2441             w.setFixedWidth(70)
2442         row = QHBoxLayout()
2443         row.addWidget(self.g_lo)
2444         row.addWidget(self.g_hi)
2445         row.addWidget(self.g_st)
2446         top.addLayout(row, 0, 3)
2447         top.addWidget(QLabel("MC reps:"), 0, 4)
2448         self.rep_ed = QLineEdit("2000")
2449         self.rep_ed.setFixedWidth(70)
2450         top.addWidget(self.rep_ed, 0, 5)
2451
2452         self.fixed_box = QWidget()
2453         self.fixed_layout = QHBoxLayout(self.fixed_box)
2454         self.fixed_layout.setContentsMargins(0, 0, 0, 0)
2455         redraw = QPushButton("Redraw")
2456         redraw.setObjectName("accent")
2457         redraw.clicked.connect(self._redraw)
2458         save = QPushButton("Save PNG")
2459         save.clicked.connect(self._save_png)
2460         close = QPushButton("Close")
2461         close.clicked.connect(self.accept)
2462         wrap = QHBoxLayout()
2463         wrap.addWidget(self.fixed_box, 1)
2464         wrap.addWidget(redraw)
2465         wrap.addWidget(save)
2466         wrap.addWidget(close)
2467         top.addLayout(wrap, 1, 0, 1, 6)
2468         lay.addLayout(top)
2469
2470         self._fig = Figure(dpi=100)
2471         self._canvas = FigureCanvas(self._fig)
2472         lay.addWidget(NavigationToolbar(self._canvas, self))
2473         lay.addWidget(self._canvas, 1)
2474
2475         self.x_cb.currentIndexChanged.connect(lambda _i: (self._build_fixed(), s
self._redraw()))
2476         self._build_fixed()
2477         self._redraw()
2478
2479     def _build_fixed(self):
2480         while self.fixed_layout.count():
2481             it = self.fixed_layout.takeAt(0)
2482             if it.widget():
2483                 it.widget().deleteLater()
2484         self._fixed_edits = {}
2485         xk = self.x_cb.currentData()
2486         lbl = QLabel("Fixed:")
2487         self.fixed_layout.addWidget(lbl)
2488         for name, d in self._ctx["fixed_specs"][xk]:
2489             self.fixed_layout.addWidget(QLabel(f"{name} ="))
2490             ed = QLineEdit(str(d))
2491             ed.setFixedWidth(70)
2492             self.fixed_layout.addWidget(ed)
2493             self._fixed_edits[name] = ed
2494         lo, hi, st = self._ctx["grid"][xk]
2495         self.g_lo.setText(str(lo))
2496         self.g_hi.setText(str(hi))
2497         self.g_st.setText(str(st))
2498
2499     def _inputs(self):
2500         xk = self.x_cb.currentData()

```

```

2501         lo, hi, st = (float(w.text()) for w in (self.g_lo, self.g_hi, self.g_st)
2502         )
2503         xs = np.arange(lo, hi + st / 2.0, st)
2504         fx = {n: float(w.text()) for n, w in self._fixed_edits.items()}
2505         reps = max(200, int(self.rep_ed.text() or 2000))
2506         return xk, xs, fx, reps
2507
2508     def _redraw(self):
2509         xk, xs, fx, reps = self._inputs()
2510         p_comp = self._ctx["computed"](xk, xs, fx)
2511         p_usp = self._ctx["usp"](xk, xs, fx, reps)
2512         self._fig.clear()
2513         ax = self._fig.add_subplot(111)
2514         ax.plot(
2515             xs,
2516             p_comp,
2517             "o-",
2518             color=SERIES_COLORS[0],
2519             lw=1.8,
2520             ms=4,
2521             label="Computed plan (acceptance-limit table)",
2522         )
2523         ax.plot(xs, p_usp, "s--",
2524             color=SERIES_COLORS[2], lw=1.8, ms=4, label=self._usp_label)
2525         for t in (0.8, 0.9):
2526             ax.axhline(t, ls=":", lw=0.8, color="0.5")
2527         ax.set_ylim(0.0, 1.05)
2528         ax.set_xlabel(self.x_cb.currentText())
2529         ax.set_ylabel("Probability of passing")
2530         ax.set_title(self._title)
2531         ax.legend(fontsize=9, loc="best")
2532         ax.grid(True, alpha=0.3)
2533         self._fig.tight_layout()
2534         self._canvas.draw()
2535
2536     def _save_png(self):
2537         path, _ = QFileDialog.getSaveFileName(
2538             self, "Save figure", self._save_base + ".png", "PNG image (*.png)"
2539         )
2540         if path:
2541             self._fig.savefig(path, dpi=150)
2542             QMessageBox.information(self, "Saved", f"Figure saved to {path}")
2543
2544 class SplashScreen(QDialog):
2545     """Frameless startup splash: logo, note, progress bar, developer footer."""
2546
2547     def __init__(self):
2548         super().__init__(
2549             None, Qt.WindowType.FramelessWindowHint | Qt.WindowType.WindowStaysOn
2550             nTopHint
2551         )
2552         self.setFixedSize(620, 400)
2553         self.setStyleSheet("""
2554             QDialog { background:#0d2b55; }
2555             QLabel { background:transparent; }
2556             #title { color:white; font-size:26px; font-weight:700; }
2557             #note { color:#bcd4f5; font-size:10pt; }
2558             #status{ color:#9fc3f2; font-size:9pt; }
2559             #foot { color:#bcd4f5; font-size:9pt; }
2560             #foot a{ color:#4da3ff; }
2561             QProgressBar { background:#123a6e; border:0; border-radius:5px;
2562                 height:10px; text-align: center; }
2563             QProgressBar::chunk { background:#4da3ff; border-radius:5px; }

```

```

2562         """
2563         lay = QVBoxLayout(self)
2564         lay.setContentsMargins(30, 22, 30, 14)
2565         center = Qt.AlignmentFlag.AlignCenter
2566
2567         logo_file = resource_path("logo.png")
2568         if os.path.exists(logo_file):
2569             pix = QPixmap(logo_file)
2570             if not pix.isNull():
2571                 if pix.height() > 96:
2572                     pix = pix.scaledToHeight(96, Qt.TransformationMode.SmoothTra
nsformation)
2573             ll = QLabel()
2574             ll.setPixmap(pix)
2575             ll.setAlignment(center)
2576             lay.addWidget(ll)
2577
2578             t = QLabel("PyCuDAL")
2579             t.setObjectName("title")
2580             t.setAlignment(center)
2581             lay.addWidget(t)
2582             n = QLabel(
2583                 "Parametric acceptance limits for USP <905> Content\n"
2584                 "Uniformity and USP <711> Dissolution"
2585             )
2586             n.setObjectName("note")
2587             n.setAlignment(center)
2588             lay.addWidget(n)
2589             lay.addSpacing(14)
2590
2591             self.bar = QProgressBar()
2592             self.bar.setRange(0, 100)
2593             lay.addWidget(self.bar)
2594             self.status = QLabel("Starting...")
2595             self.status.setObjectName("status")
2596             self.status.setAlignment(center)
2597             lay.addWidget(self.status)
2598             lay.addStretch(1)
2599
2600             foot = QLabel(f'Program developed by: <a href="{REPO_URL}">Moaz El-
Essauey</a>')
2601             foot.setObjectName("foot")
2602             foot.setOpenExternalLinks(True) # click opens the GitHub repo
2603             foot.setAlignment(center)
2604             lay.addWidget(foot)
2605
2606             geo = QApplication.primaryScreen().availableGeometry()
2607             self.move(geo.center() - self.rect().center())
2608
2609             def set_progress(self, frac, msg):
2610                 self.bar.setValue(int(frac * 100))
2611                 self.status.setText(msg)
2612                 QApplication.processEvents()
2613                 time.sleep(0.08)
2614
2615             def _load_libraries(splash=None):
2616                 global np, pd, cuspl, cuspl2, disp1, disp2
2617                 global Figure, FigureCanvas, NavigationToolbar, make_interp_spline
2618                 global HAVE_CUDAL, HAVE_MPL, HAVE_SPLINE, HAVE_XLSX
2619
2620             def step(frac, msg):
2621                 if splash is not None:
2622                     splash.set_progress(frac, msg)
2623

```

```

2624
2625     step(0.05, "Loading NumPy...")
2626     import numpy as _np
2627
2628     np = _np
2629
2630     step(0.20, "Loading Pandas...")
2631     import pandas as _pd
2632
2633     pd = _pd
2634
2635     step(0.40, "Loading CuDAL core...")
2636     try:
2637         from cudal import cusp1 as _a
2638         from cudal import cusp2 as _b
2639         from cudal import displ as _c
2640         from cudal import disp2 as _d
2641
2642         cusp1, cusp2, displ, disp2 = _a, _b, _c, _d
2643         HAVE_CUDAL = True
2644     except Exception:
2645         HAVE_CUDAL = False
2646
2647     step(0.60, "Loading SciPy...")
2648     try:
2649         from scipy.interpolate import make_interp_spline as _mis
2650
2651         make_interp_spline = _mis
2652         HAVE_SPLINE = True
2653     except Exception:
2654         HAVE_SPLINE = False
2655
2656     step(0.75, "Loading Matplotlib...")
2657     try:
2658         from matplotlib.backends.backend_qtagg import FigureCanvasQTAgg as _C
2659         from matplotlib.backends.backend_qtagg import NavigationToolbar2QT as _T
2660         from matplotlib.figure import Figure as _F
2661
2662         Figure, FigureCanvas, NavigationToolbar = _F, _C, _T
2663         HAVE_MPL = True
2664     except Exception:
2665         HAVE_MPL = False
2666
2667     step(0.90, "Loading export engines...")
2668     try:
2669         import openpyxl # noqa: F401
2670
2671         HAVE_XLSX = True
2672     except Exception:
2673         HAVE_XLSX = False
2674
2675     step(1.0, "Ready.")
2676
2677
2678 # -----
2679 # Main window
2680 # -----
2681 class CudalApp(QMainWindow):
2682     def __init__(self):
2683         super().__init__()
2684         self.setWindowTitle("PyCuDAL -
2685 - Content Uniformity & Dissolution Acceptance Limits")
2686         self.resize(1180, 720)
2687         self.setMinimumSize(980, 600)

```

```

2687
2688     self._settings = self._load_settings()
2689
2690     # ---- logo -----
2691     -
2691     logo_file = resource_path("logo.png")
2692     if os.path.exists(logo_file):
2693         self.setWindowIcon(QIcon(logo_file))
2694
2695     central = QWidget()
2696     self.setCentralWidget(central)
2697     outer = QVBoxLayout(central)
2698
2699     header = QHBoxLayout()
2700     if os.path.exists(logo_file):
2701         pix = QPixmap(logo_file)
2702         if not pix.isNull():
2703             logo_lbl = QLabel()
2704             logo_lbl.setPixmap(
2705                 pix.scaledToHeight(40, Qt.TransformationMode.SmoothTransform
2706             )
2707             header.addWidget(logo_lbl)
2708     else:
2709         t = QLabel("CuDAL")
2710         t.setObjectName("header")
2711         s = QLabel(
2712             "    Parametric acceptance limits for USP <905> Content Uniformity "
2713             "and USP <711> Dissolution"
2714         )
2715         s.setObjectName("subheader")
2716         if not logo_file:
2717             header.addWidget(t)
2718         header.addWidget(s)
2719         header.addStretch(1)
2720         outer.addLayout(header)
2721
2722     # ---- tabs -----
2723     ---
2723     self.notebook = QTabWidget()
2724     self.tabs = [Cusp1Tab(), Cusp2Tab(), Disp1Tab(), Disp2Tab()]
2725     for tab, text in zip(
2726         self.tabs,
2727         [
2728             "Content Uniformity -- Plan 1",
2729             "Content Uniformity -- Plan 2",
2730             "Dissolution -- Plan 1",
2731             "Dissolution -- Plan 2",
2732         ],
2733     ):
2734         self.notebook.addTab(tab, text)
2735     outer.addWidget(self.notebook, 1)
2736
2737     # ---- comprehensive menu bar -----
2738     menubar = self.menuBar()
2739
2740     file = menubar.addMenu("&File")
2741     a = file.addAction("Export current results (CSV)")
2742     a.setShortcut(QKeySequence("Ctrl+E"))
2743     a.triggered.connect(self._export_current_csv)
2744     a = file.addAction("Export current results (PDF)")
2745     a.triggered.connect(self._export_current_pdf)
2746     a = file.addAction("Export all results (XLSX)")
2747     a.triggered.connect(self._export_all_xlsx)

```

```

2748     filem.addSeparator()
2749     a = filem.addAction("Save settings now")
2750     a.setShortcut(QKeySequence("Ctrl+S"))
2751     a.triggered.connect(self._save_settings_now)
2752     filem.addSeparator()
2753     a = filem.addAction("Exit")
2754     a.setShortcut(QKeySequence("Ctrl+Q"))
2755     a.triggered.connect(self.close)
2756
2757     runm = menubar.addMenu("&Run")
2758     a = runm.addAction("Run analysis")
2759     a.setShortcut(QKeySequence("Ctrl+R"))
2760     a.triggered.connect(lambda: self._current_tab()._on_run())
2761     a = runm.addAction("Reset parameters")
2762     a.triggered.connect(lambda: self._current_tab()._reset_defaults())
2763     runm.addSeparator()
2764     a = runm.addAction("Plot results")
2765     a.setShortcut(QKeySequence("Ctrl+P"))
2766     a.triggered.connect(lambda: self._current_tab().results._show_plot())
2767     a = runm.addAction("OC curve\u2026")
2768     a.setShortcut(QKeySequence("Ctrl+O"))
2769     a.triggered.connect(self._show_oc_current)
2770     runm.addSeparator()
2771     a = runm.addAction("Copy selection")
2772     a.setShortcut(QKeySequence.Copy)
2773     a.triggered.connect(self._copy_current_selection)
2774     a = runm.addAction("Clear results")
2775     a.triggered.connect(lambda: self._current_tab().results.clear())
2776
2777     viewm = menubar.addMenu("&View")
2778     for i, label in enumerate(
2779         (
2780             "Content Uniformity \u2013 Plan 1",
2781             "Content Uniformity \u2013 Plan 2",
2782             "Dissolution \u2013 Plan 1",
2783             "Dissolution \u2013 Plan 2",
2784         )
2785     ):
2786         a = viewm.addAction(label)
2787         a.setShortcut(QKeySequence(f"Ctrl+{i + 1}"))
2788         a.triggered.connect(lambda _, i=i: self.notebook.setCurrentIndex(i))
2789
2790     helpm = menubar.addMenu("&Help")
2791     a = helpm.addAction("Documentation (online)")
2792     a.triggered.connect(lambda: QDesktopServices.openUrl(QUrl(REPO_URL)))
2793     a = helpm.addAction("Report an issue")
2794     a.triggered.connect(lambda: QDesktopServices.openUrl(QUrl(REPO_URL + "/issues")))
2795
2796     helpm.addSeparator()
2797     a = helpm.addAction("About / Help")
2798     a.setShortcut(QKeySequence("F1"))
2799     a.triggered.connect(self._show_about)
2800
2801     # ---- action toolbar -----
2802     toolbar = self.addToolBar("Main")
2803     toolbar.setMovable(False)
2804     toolbar.setFloatable(False)
2805     toolbar.setToolButtonStyle(Qt.ToolButtonStyle.ToolButtonTextBesideIcon)
2806     toolbar.setIconSize(QSize(18, 18))
2807
2808     def _glyph_icon(glyph, color=ACCENT_DARK):
2809         px = QPixmap(18, 18)
2810         px.fill(Qt.GlobalColor.transparent)

```



```

2810         p = QPainter(px)
2811         p.setPen(QColor(color))
2812         f = QFont("Segoe UI Symbol", 11)
2813         f.setBold(True)
2814         p.setFont(f)
2815         p.drawText(px.rect(), Qt.AlignmentFlag.AlignCenter, glyph)
2816         p.end()
2817         return QIcon(px)
2818
2819     def add_tool(text, glyph, slot, tip):
2820         a = QAction(_glyph_icon(glyph), text, self)
2821         a.triggered.connect(slot)
2822         a.setToolTip(tip)
2823         toolbar.addAction(a)
2824         return a
2825
2826     add_tool(
2827         "Run",
2828         "\u25b6",
2829         lambda: self._current_tab()._on_run(),
2830         "Run the selected analysis (Ctrl+R)",
2831     )
2832     add_tool(
2833         "Plot",
2834         "\u223f",
2835         lambda: self._current_tab().results._show_plot(),
2836         "Plot results (Ctrl+P)",
2837     )
2838     add_tool(
2839         "OC Curve",
2840         "\u2277",
2841         self._show_oc_current,
2842         "OC curve: computed plan vs USP test (Ctrl+O)",
2843     )
2844     add_tool(
2845         "CSV", "\u2913", self._export_current_csv, "Export current results t
o CSV (Ctrl+E)"
2846     )
2847     add_tool(
2848         "PDF", "\u2261", self._export_current_pdf, "Export current results t
o SAS-style PDF"
2849     )
2850     add_tool("XLSX", "\u25a6", self._export_all_xlsx, "Export all results to
Excel")
2851     add_tool(
2852         "Reset",
2853         "\u21ba",
2854         lambda: self._current_tab()._reset_defaults(),
2855         "Reset parameters to defaults",
2856     )
2857     add_tool("About", "?", self._show_about, "About PyCuDAL (F1)")
2858
2859     # # ---- shortcuts -----
2860     # QShortcut(QKeySequence("Ctrl+R"), self, activated=lambda: self._curren
t_tab()._on_run())
2861     # QShortcut(QKeySequence("Ctrl+P"), self,
2862     #           activated=lambda: self._current_tab().results._show_plot())
2863
2864     # ---- restore persisted state -----
2865     for tab in self.tabs:
2866         tab.apply_state(self._settings.get("tabs", {}).get(tab.__class__.__n
ame_))

```

```

2867         try:
2868             idx = int(self._settings.get("tab_index", 0))
2869             self.notebook.setCurrentIndex(max(0, min(idx, 3)))
2870         except Exception:
2871             pass
2872         g = self._settings.get("geometry")
2873         if isinstance(g, (list, tuple)) and len(g) == 4:
2874             self.setGeometry(int(g[0]), int(g[1]), int(g[2]), int(g[3]))
2875
2876         # ---- footer / status bar -----
2877         self.statusBar().showMessage("Ready.")
2878         credit_lbl = QLabel(
2879             'Created by <a href="https://github.com/moazelessawey/pycudal" '
2880             'style="color: #2f6fed; text-decoration: none;">Moaz El-Essawey</a>'
2881         )
2882         credit_lbl.setOpenExternalLinks(True)
2883         credit_lbl.setTextFormat(Qt.TextFormat.RichText)
2884         credit_lbl.setStyleSheet("margin-right: 10px; font-size: 9pt;")
2885         self.statusBar().addPermanentWidget(credit_lbl)
2886
2887         # -- helpers -----
2888         def _current_tab(self):
2889             return self.notebook.currentWidget()
2890
2891         @staticmethod
2892         def _load_settings():
2893             try:
2894                 with open(SETTINGS_PATH, encoding="utf-8") as fh:
2895                     return json.load(fh)
2896             except Exception:
2897                 return {}
2898
2899         def _save_settings(self):
2900             data = {
2901                 "geometry": [self.x(), self.y(), self.width(), self.height()],
2902                 "tab_index": self.notebook.currentIndex(),
2903                 "tabs": {tab.__class__.__name__: tab.collect_state() for tab in self
2904                 .tabs},
2905             }
2906             try:
2907                 with open(SETTINGS_PATH, "w", encoding="utf-8") as fh:
2908                     json.dump(data, fh, indent=2)
2909             except Exception:
2910                 pass
2911
2912         def closeEvent(self, event):
2913             self._save_settings()
2914             event.accept()
2915
2916         # -- actions -----
2917         def _export_current_csv(self):
2918             self._current_tab().results._export_csv()
2919
2920         def _export_all_xlsx(self):
2921             if not HAVE_XLSX:
2922                 QMessageBox.critical(
2923                     self,
2924                     "Excel export unavailable",
2925                     "openpyxl is required.\nInstall it with: pip install openpyxl",
2926                 )
2927             return

```

```

2927     sheets = {
2928         tab.__class__.__name__: tab.results._df
2929         for tab in self.tabs
2930         if tab.results._df is not None
2931     }
2932     if not sheets:
2933         QMessageBox.information(self, "Nothing to export", "Run at least one
analysis first.")
2934         return
2935     path, _ = QFileDialog.getSaveFileName(
2936         self,
2937         "Export all results",
2938         f"PyCuDAL-all-results-{time.strftime('%Y%m%d-%H%M')}.xlsx",
2939         "Excel workbook (*.xlsx)",
2940     )
2941     if not path:
2942         return
2943     with pd.ExcelWriter(path, engine="openpyxl") as writer:
2944         for name, df in sheets.items():
2945             df.to_excel(writer, sheet_name=name[:31], index=False)
2946     QMessageBox.information(self, "Exported", f"Saved {len(sheets)} sheet(s)
to {path}")
2947
2948     # -- top-bar dispatchers (guarded for optional features) -----
2949     def _export_current_pdf(self):
2950         r = self._current_tab().results
2951         if hasattr(r, "_export_pdf"):
2952             r._export_pdf()
2953         else:
2954             QMessageBox.information(
2955                 self, "Unavailable", "PDF export is not included in this build."
2956             )
2957
2958     def _show_oc_current(self):
2959         t = self._current_tab()
2960         if hasattr(t, "_show_oc"):
2961             t._show_oc()
2962         else:
2963             QMessageBox.information(
2964                 self, "Unavailable", "OC curves are not included in this build."
2965             )
2966
2967     def _copy_current_selection(self):
2968         r = self._current_tab().results
2969         if hasattr(r, "_copy_selection"):
2970             r._copy_selection()
2971
2972     def _save_settings_now(self):
2973         self._save_settings()
2974         self.statusBar().showMessage("Settings saved.", 3000)
2975
2976     def _show_about(self):
2977         deps = (
2978             f"matplotlib: {'yes' if HAVE_MPL else 'no'}\n"
2979             f"scipy splines: {'yes' if HAVE_SPLINE else 'no'}\n"
2980             f"openpyxl (xlsx): {'yes' if HAVE_XLSX else 'no'}"
2981         )
2982         about_text = (
2983             f"PyCuDAL v{VERSION}\n\n"
2984             "Parametric acceptance limits for USP <905> Content Uniformity\n"
2985             "and USP <711> Dissolution.\n\n"
2986             "This tool mirrors the functionality of the original SAS programs\n"
2987             "(CALCUSPx/CALDISPx, EVCUSPx/EVDISPx, SMPCUSPx/SMPDISPx)\n"
2988             "developed by James Bergum, Ph.D.\n\n"

```

```

2989         "Created & Maintained by: Moaz El-Essawey\n"
2990         "GitHub: https://github.com/moazelessawey/pycudal\n\n"
2991         "Shortcuts:\n"
2992         "  Ctrl+R  run analysis\n"
2993         "  Ctrl+E  export current results (CSV)\n"
2994         "  Ctrl+P  plot results\n"
2995         "  Ctrl+C  copy selected table rows\n"
2996         "  F1      this dialog\n\n"
2997         f"Optional dependencies:\n{deps}"
2998     )
2999     QMessageBox.about(self, "About PyCuDAL", about_text)
3000
3001
3002# -----
3003# Self-test & entry point
3004# -----
3005def run_selftest():
3006    assert make_grid(1.0, 2.0, 0.5) == [1.0, 1.5, 2.0]
3007    try:
3008        make_grid(2.0, 1.0, 1.0, "x")
3009        raise AssertionError("make_grid should reject high < low")
3010    except ValueError:
3011        pass
3012    assert fmt_num(1234.56789) == "1,234.5679"
3013    assert fmt_num("abc") == "abc"
3014    xx, yy = _spline_xy([1, 2, 3, 4], [1, 4, 9, 16])
3015    assert len(xx) == len(yy) == 300
3016    print("selftest OK")
3017
3018
3019def main():
3020    if "--selftest" in sys.argv:
3021        _load_libraries(None)
3022        run_selftest()
3023        return
3024
3025    app = QApplication(sys.argv)
3026    app.setStyle("Fusion")
3027
3028    splash = SplashScreen()
3029    splash.show()
3030    app.processEvents()
3031    _load_libraries(splash)
3032
3033    if not HAVE_CUDAL:
3034        splash.close()
3035        QMessageBox.critical(
3036            None,
3037            "Missing dependency",
3038            "The `cudal` package could not be imported.\n"
3039            "Put this script next to the `cudal` package folder\n"
3040            "or install it, then restart the GUI.",
3041        )
3042        sys.exit(1)
3043
3044    family = _register_local_fonts()
3045    if not family:
3046        family = "Segoe UI" if sys.platform.startswith("win") else "DejaVu Sans"
3047    app.setFont(QFont(family, 10))
3048    app.setStyleSheet(build_styleshet(family))
3049    if HAVE_MPL:
3050        try:
3051            fdir = resource_path("fonts")
3052            if os.path.isdir(fdir):

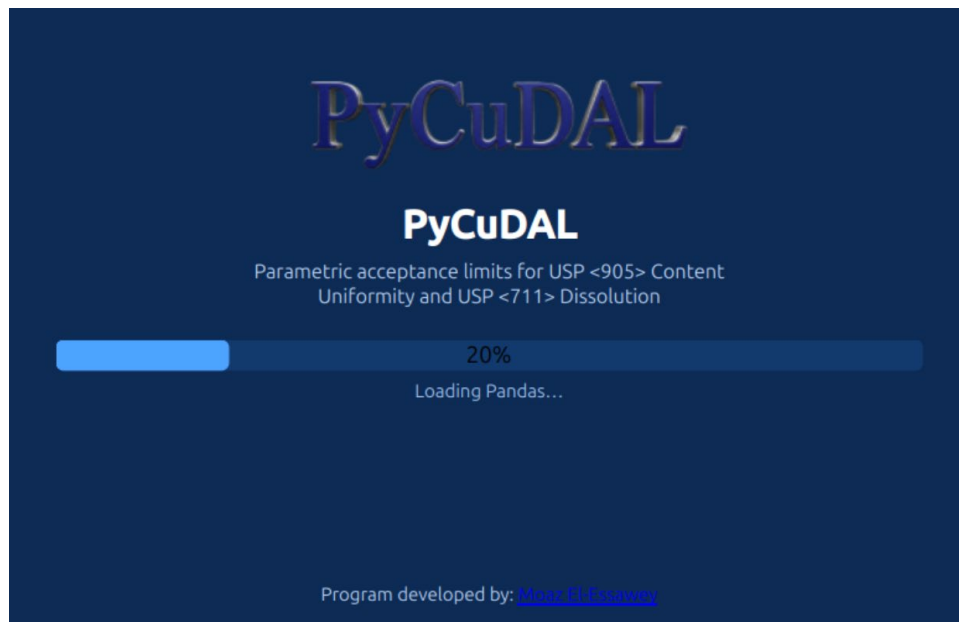
```

```
3053         import matplotlib as mpl
3054         from matplotlib import font_manager as fm
3055
3056         for f in os.listdir(fdir):
3057             if f.lower().endswith((".ttf", ".otf")):
3058                 fm.fontManager.addfont(os.path.join(fdir, f))
3059                 mpl.rcParams["font.family"] = family
3060     except Exception:
3061         pass
3062
3063     win = CudalApp()
3064     win.show()
3065     splash.close()
3066     sys.exit(app.exec())
3067
3068
3069 if __name__ == "__main__":
3070     main()
```

APPENDIX B — PRIMARY WINDOWS

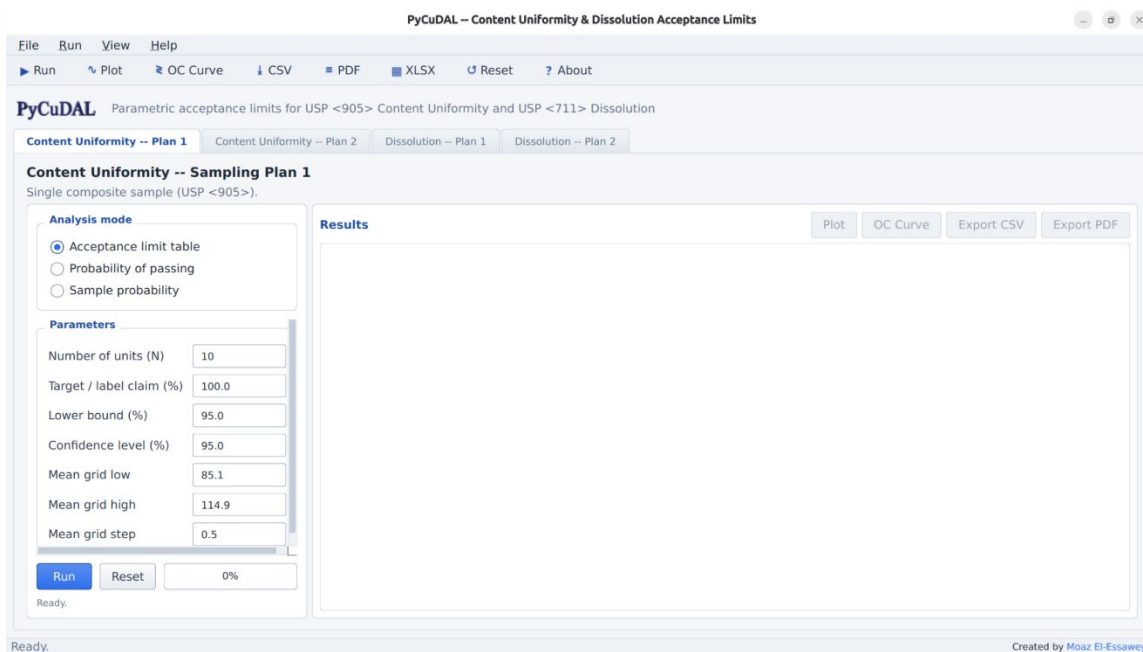
Startup (splash screen)

When the program starts, a splash screen appears while the libraries load; the progress bar tracks each stage (NumPy → Pandas → CuDAL core → SciPy → Matplotlib → export engines). The footer links to the project repository.



The main window

The main window contains: a header (logo + scope), an action toolbar, the four scenario tabs, and a status bar whose right side credits the author.



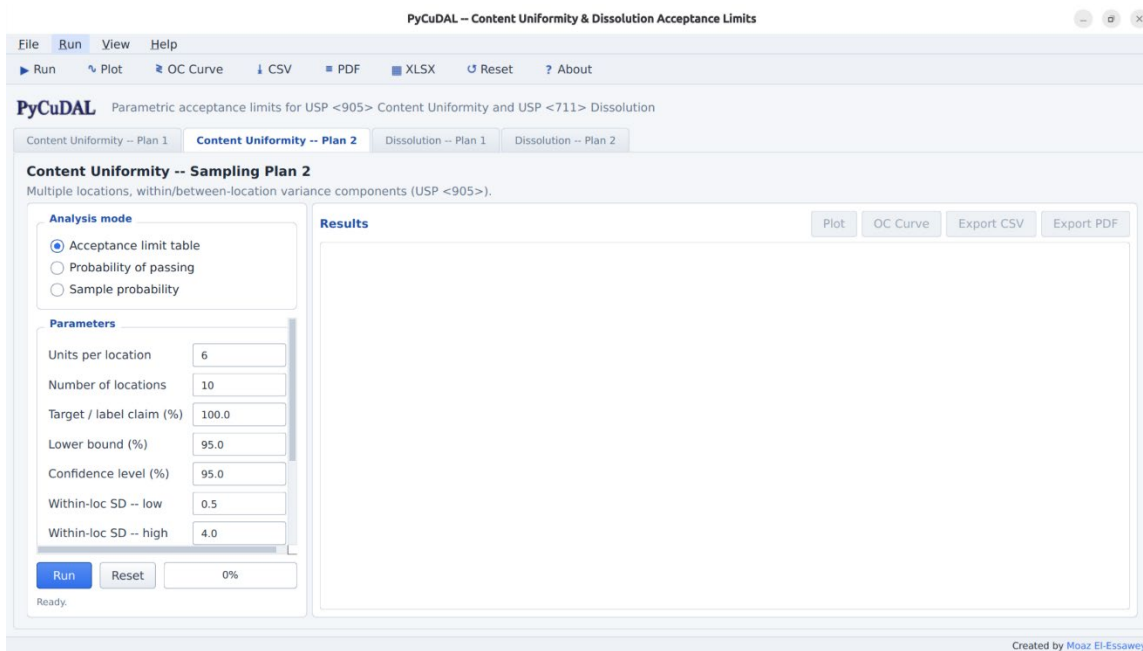
Toolbar / Run menu

Button	Action	Shortcut
► Run	Run the selected analysis	Ctrl+R
~ Plot	Open the plot dialog for the current results	Ctrl+P
≧ OC Curve	Open the OC-curve dialog	Ctrl+O
↓ CSV / ≡ PDF / XLSX	Export current table / PDF / all results	Ctrl+E (CSV)
↺ Reset	Restore default parameters	
? About	About / help dialog	F1

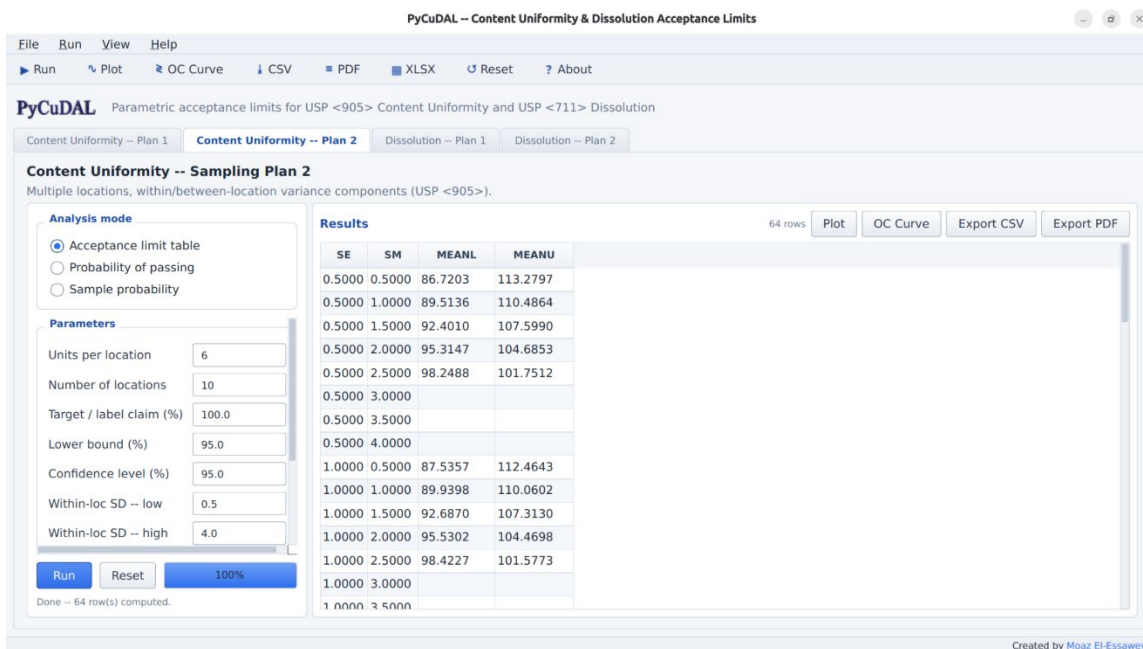
The **File** menu adds *Save settings now* (Ctrl+S) and *Exit*; **View** switches tabs (Ctrl+1...4); **Help** opens the online documentation and issue tracker. Parameters, mode, active tab and window geometry are persisted between sessions.

A scenario tab

Every tab has the same layout: an **Analysis mode** panel (three radio buttons), a scrollable **Parameters** panel (one card per mode, live-validated entries with tooltips), a **Run** row (Run, Reset, progress bar, status line) and the **Results** panel.



The **Results** panel shows the computed table with zebra striping, click-to-sort headers, conditional coloring of probability values (red < 0.80 , green ≥ 0.90), a row counter, and Ctrl+C copy of the selection.

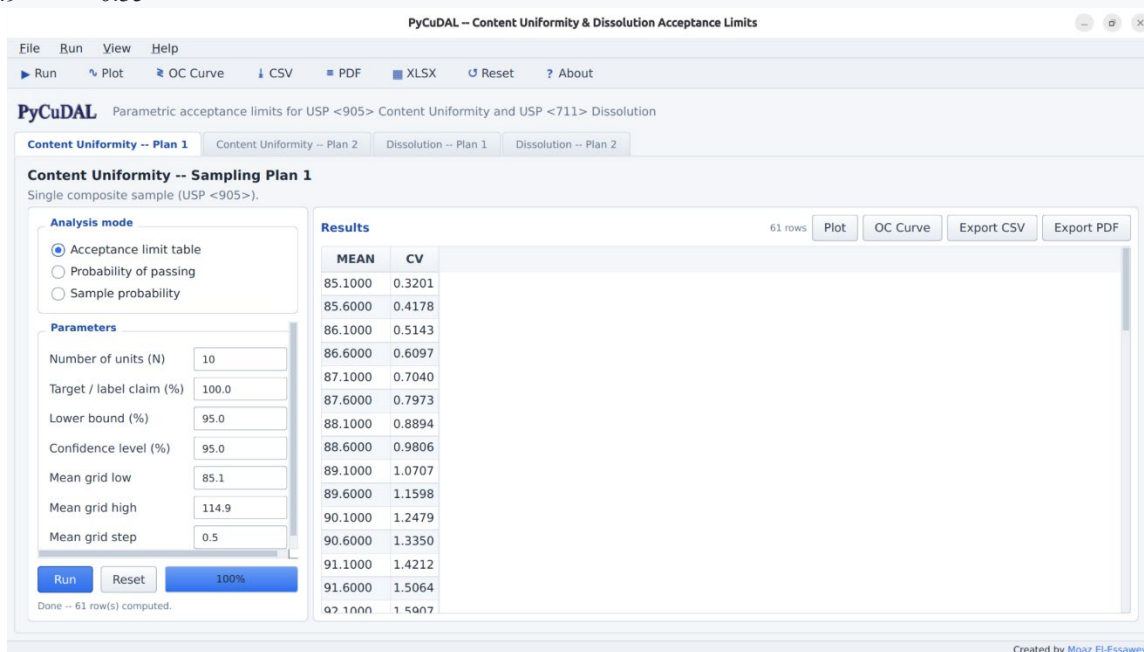


Content Uniformity / Sampling Plan 1

The user enters the sample size (number of dosage units tested), the target (usually the average of potency limits), the coverage percentage (usually 90 or 95) and the confidence level (usually 90 or 95), plus the mean grid for the table (default 85.1–114.9 by 0.5).

1) Acceptance limit table. The output lists, for each candidate mean, the corresponding acceptance limit on the CV:

MEAN (% CLAIM)	CV (%)	MEAN (% CLAIM)	CV (%)
85.1	0.48	100.0	4.18
85.2	0.51	100.1	4.16
...			
114.9	0.35		



2) Probability of passing. The user provides "true" values for the population mean U and coefficient of variation (CV/RSD) as grids (defaults U 95–105 × 2.5, CV 1–4 × 1); the program calculates the probability that sample results fall within the acceptance limits:

U	CV	PROBABILITY OF PASSING
95	1	1.00000
100	1	1.00000
95	4	0.05220
100	4	0.56434

3) Sample probability. The user enters the sample mean and sample CV (defaults 100, 2.0); the output is the lower bound (e.g. mean 100, CV 4 → **0.98003**).

Content Uniformity / Sampling Plan 2

The user enters the number of locations, the number of dosage units per location, target, coverage and confidence, and (for the table) grids of the within-location SD (SE) and between-location SD (SM) (defaults $0.5\text{--}4.0 \times 0.5$). Table entries are **lower (LL) and upper (UL) limits on the mean**; SE is the pooled within-location standard deviation; all SDs and means are in % claim.

STANDARD DEVIATION OF LOCATION MEANS

	0.1		0.2	
SE	LL	UL	LL	UL
0.1	84.8	115.2	84.8	115.2
0.2	84.7	115.3	84.8	115.2

PyCuDAL – Content Uniformity & Dissolution Acceptance Limits

File Run View Help

Run Plot OC Curve CSV PDF XLSX Reset ? About

PyCuDAL Parametric acceptance limits for USP <905> Content Uniformity and USP <711> Dissolution

Content Uniformity -- Plan 1 **Content Uniformity -- Plan 2** Dissolution -- Plan 1 Dissolution -- Plan 2

Content Uniformity -- Sampling Plan 2

Multiple locations, within/between-location variance components (USP <905>).

Analysis mode

☒ Acceptance limit table
☐ Probability of passing
☐ Sample probability

Parameters

Units per location: 6
Number of locations: 10
Target / label claim (%): 100.0
Lower bound (%): 95.0
Confidence level (%): 95.0
Within-loc SD -- low: 0.5
Within-loc SD -- high: 4.0

Run Reset 100%

Done -- 64 row(s) computed.

Results 64 rows Plot OC Curve Export CSV Export PDF

SE	SM	MEANL	MEANU
0.5000	0.5000	86.7203	113.2797
0.5000	1.0000	89.5136	110.4864
0.5000	1.5000	92.4010	107.5990
0.5000	2.0000	95.3147	104.6853
0.5000	2.5000	98.2488	101.7512
0.5000	3.0000		
0.5000	3.5000		
0.5000	4.0000		
1.0000	0.5000	87.5357	112.4643
1.0000	1.0000	89.9398	110.0602
1.0000	1.5000	92.6870	107.3130
1.0000	2.0000	95.5302	104.4698
1.0000	2.5000	98.4227	101.5773
1.0000	3.0000		
1.0000	3.5000		

Created by Moaz El-Essawy

For the evaluation the user provides true U, within-location SD and between-location SD grids; for the sample probability the sample mean, within sample SD and between sample SD. [Note: the latter is just the sample standard deviation of the sample means, not a variance component!] Reference values: evaluation (U 95, SE 2.2, SM 2.2, 4×10 design) → **0.09180**; sample (100, 2.2, 2.46) → **0.98750**.

Dissolution / Sampling Plan 1

The user enters Q, the sample size, coverage and confidence (table grid step default 1.0). The table entry is the **upper limit on the CV of 6 dissolution assays** for each mean (80.2 $\rightarrow$ 0.09 ... 100.0 $\rightarrow$ 4.97). Evaluation and sample modes mirror CU Plan 1 (defaults U 90–100 $\times$ 2.5, CV 1–4 $\times$ 1; sample mean 90, CV 3). Reference values: evaluation (95, 4) $\rightarrow$ **0.73988**, (100, 4) $\rightarrow$ **0.81098**; sample (100, 4) $\rightarrow$ **0.99824**.

PyCuDAL – Content Uniformity & Dissolution Acceptance Limits

File Run View Help

Run Plot OC Curve CSV PDF XLSX Reset ? About

PyCuDAL Parametric acceptance limits for USP <905> Content Uniformity and USP <711> Dissolution

Content Uniformity -- Plan 1 Content Uniformity -- Plan 2 **Dissolution -- Plan 1** Dissolution -- Plan 2

Dissolution -- Sampling Plan 1
Single location (USP <711>).

Analysis mode

☒ Acceptance limit table
☐ Probability of passing
☐ Sample probability

Parameters

Number of units (N)
Q value (%)
Lower bound (%)
Confidence level (%)
Mean grid step

Run Reset 100%

Done -- 20 row(s) computed.

Results 20 rows Plot OC Curve Export CSV Export PDF

MEAN	CV
81.0000	0.4453
82.0000	0.8797
83.0000	1.3037
84.0000	1.7176
85.0000	2.1217
86.0000	2.5164
87.0000	2.9012
88.0000	3.2690
89.0000	3.5986
90.0000	3.8700
91.0000	4.0835
92.0000	4.2505
93.0000	4.3832
94.0000	4.4924
95.0000	4.5865

Created by Moaz El-Essawy

Dissolution / Sampling Plan 2

The user enters Q, locations, units per location, coverage and confidence, and SE/SM grids (defaults $1.0\text{--}5.0 \times 1.0$). Table entries are **lower limits on the mean** (e.g. SE 0.25 / SM 0.25 $\rightarrow$ 80.50). Evaluation and sample modes mirror CU Plan 2 (sample defaults 90, 2.2, 2.46; reference bound 1 for the 6×10 design).

PyCuDAL -- Content Uniformity & Dissolution Acceptance Limits

File Run View Help

Run Plot OC Curve CSV PDF XLSX Reset ? About

PyCuDAL Parametric acceptance limits for USP <905> Content Uniformity and USP <711> Dissolution

Content Uniformity -- Plan 1 Content Uniformity -- Plan 2 Dissolution -- Plan 1 **Dissolution -- Plan 2**

Dissolution -- Sampling Plan 2

Multiple locations, within/between-location variance components (USP <711>).

Analysis mode

☒ Acceptance limit table
☐ Probability of passing
☐ Sample probability

Parameters

Units per location: 6
 Number of locations: 5
 Q value (%): 80.0
 Lower bound (%): 95.0
 Confidence level (%): 95.0
 Within-loc SD -- low: 1.0
 Within-loc SD -- high: 5.0

Run Reset 100%

Done -- 25 row(s) computed.

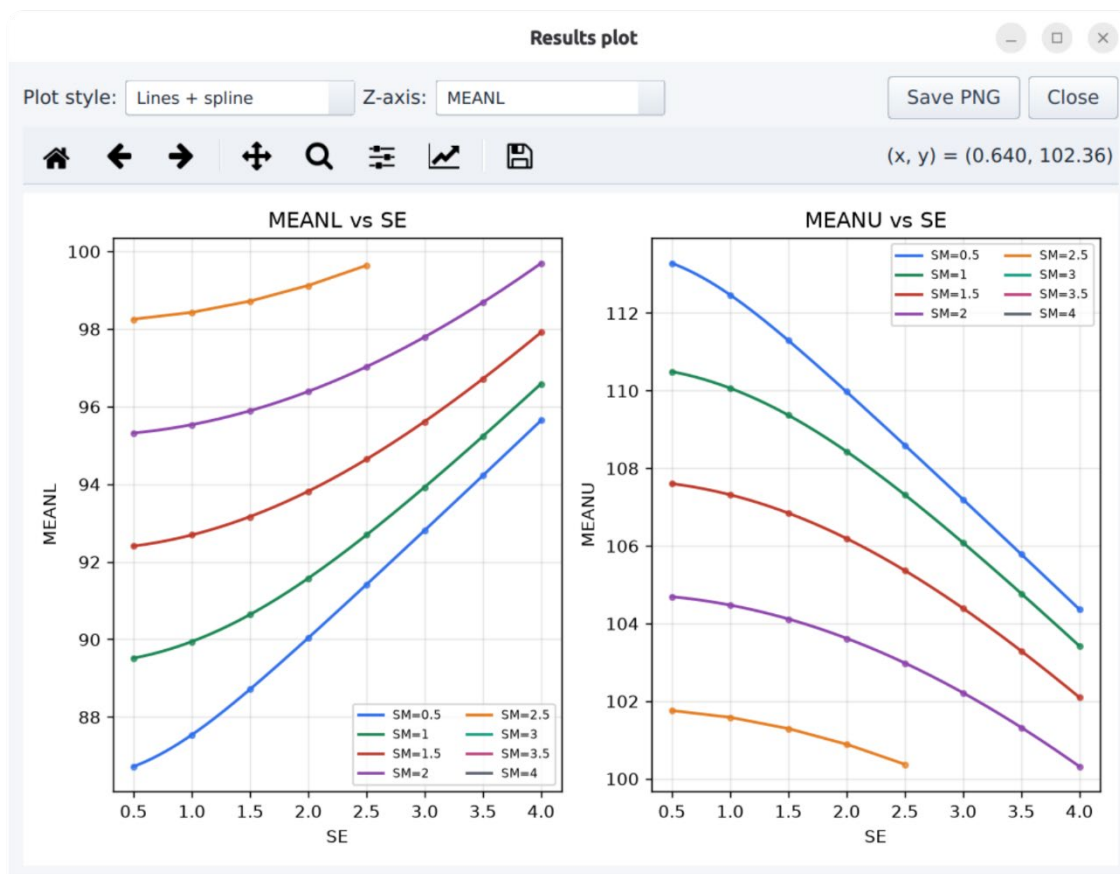
Results 25 rows Plot OC Curve Export CSV Export PDF

SE	SM	MEAN
1.0000	1.0000	83.5144
1.0000	2.0000	86.9542
1.0000	3.0000	90.8137
1.0000	4.0000	98.5938
1.0000	5.0000	
2.0000	1.0000	83.6727
2.0000	2.0000	87.0301
2.0000	3.0000	90.9466
2.0000	4.0000	98.8796
2.0000	5.0000	
3.0000	1.0000	83.9425
3.0000	2.0000	87.1629
3.0000	3.0000	91.2006
3.0000	4.0000	99.3641
3.0000	5.0000	

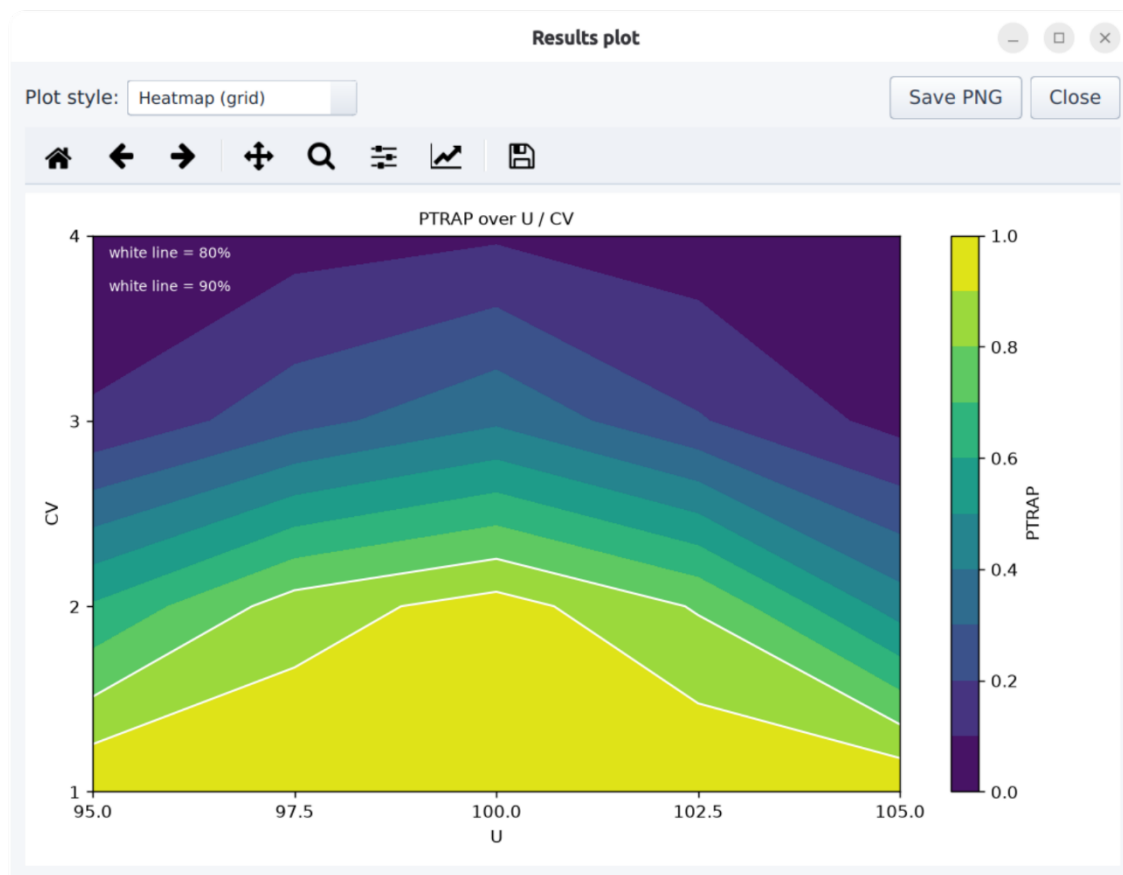
Created by Moaz El-Essawy

Plot dialog

Plot opens a modal dialog. *Lines + spline* draws the results as points with a cubic spline (Plan 2 tables draw one curve family per SM level, with a Z-axis selector when both LL and UL are present).

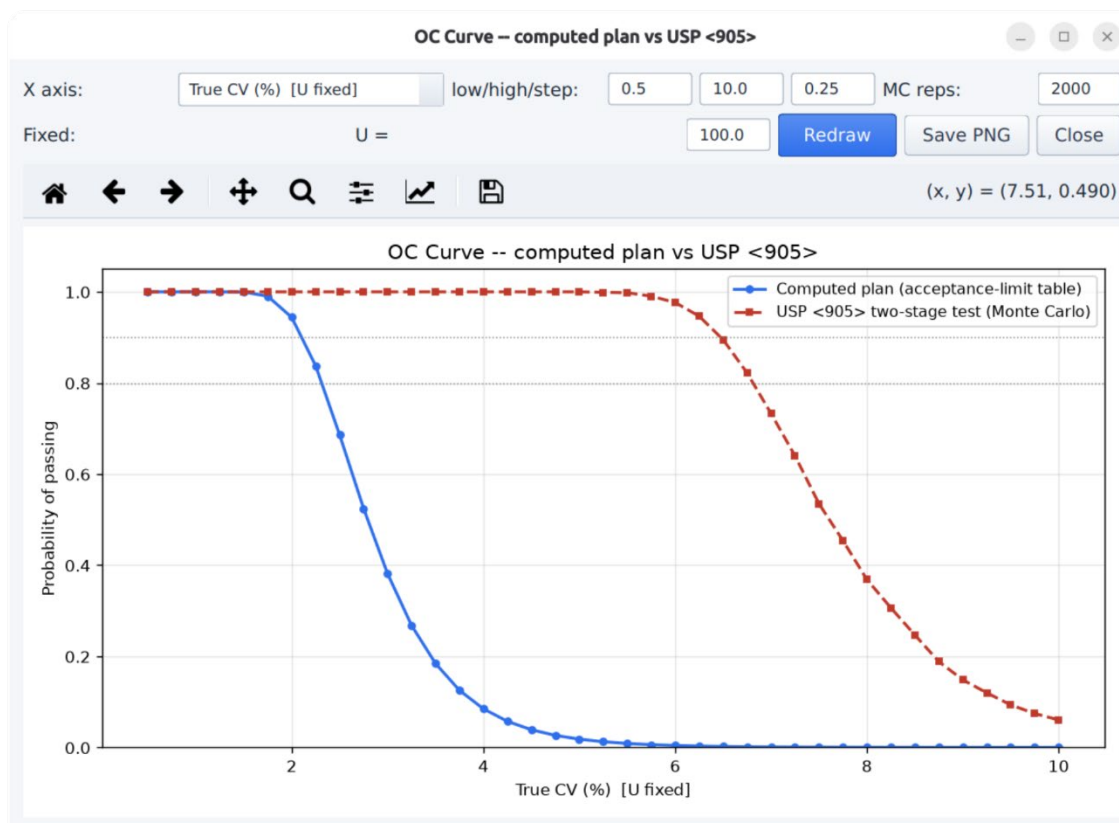


Heatmap (*grid*) draws a contour-filled surface for 3-column grids; probability surfaces include white 80 %/90 % threshold contours. Figures can be saved as PNG



OC-Curve dialog

OC Curve overlays two operating-characteristic curves on one axes: the analytic probability of passing *your computed acceptance-limit table*, and a seeded Monte-Carlo estimate of passing the *raw USP <905>/<711> multi-stage test* itself. The X axis (true CV, true mean, or within-loc SD), the fixed companion parameters, the grid and the MC replication count are editable; dotted lines mark the 80/90 % levels.



Exports

- **CSV** – the current table; **PDF** – SAS-listing style (Courier, wrapped blocks / SE×SM LL-UL matrices, 2 dp right-aligned, * for missing, title and headers repeated on every page); **XLSX** – one sheet per scenario.
- Default filenames describe their content, e.g. CUSP2-95x95-10Lx6N.pdf, DISP1-Q80-95x95-6N-EVAL.csv, CUSP1-95x95-10N-SAMPLE.pdf, ...-plot.png, ...-OC.png, PyCuDAL-all-results-<date>-<time>.xlsx.

APPENDIX C — DEFAULT OUTPUT FOR ANALYSIS (R CODE)

pycudal/utis/cudal_cu.r

```

1# =====
2# CuDAL R Verification Functions
3# CUSP1 and CUSP2 Acceptance-Limit Table Calculations
4# =====
5#
6# Purpose
7# -----
8# Independent R re-implementation of the PyCuDAL CUSP1 (Content Uniformity,
9# Sampling Plan 1) and CUSP2 (Content Uniformity, Sampling Plan 2)
10# acceptance-limit table calculations, USP <905>.
11#
12# This file is written to be read and verified line-by-line against the
13# PyCuDAL Python source (cudal/core.py, cudal/cuspl.py, cudal/cusp2.py) by a
14# second reviewer, then run on the shared test data set (Appendix G) so its
15# output can be compared directly against PyCuDAL's own output.
16#
17# Design notes for the reviewer
18# -----
19# 1. Every root search below works on ONE grid point (one MEAN, or one
20#    (SE, SM) pair) at a time, using base R's `uniroot()`. This is
21#    deliberately simpler than a hand-rolled vectorized bisection: it is
22#    slower, but every step is a call to a standard, well-tested R function,
23#    which makes this file much easier to audit and trust.
24#
25# 2. `content_uniformity_bound()` never uses `ifelse(<scalar condition>, ...)`
26#    with vector branches. That specific pattern is a known R pitfall:
27#    `ifelse()`'s output length follows the length of its *condition*
28#    argument, not its `yes`/`no` branches -- so if the condition happens to
29#    be a single TRUE/FALSE (as it is here, since `target` is one fixed
30#    value for an entire table) while the branches are vectors, R silently
31#    truncates the result to length 1 and then recycles that single value
32#    against everything downstream. Every element after the first ends up
33#    silently reusing element 1's value. This file avoids the pattern
34#    entirely: the target/label-claim branch (E) is computed once with a
35#    plain `if/else` (correct, since `target` is always scalar here) and
36#    then reused directly wherever the same branch is needed again, instead
37#    of re-deriving it with a second `ifelse()`.
38#
39# =====
40
41
42# =====
43# SECTION 1: Core statistical primitives
44# =====
45
46#' Standard Normal Cumulative Distribution Function
47#'
48#' Direct wrapper around pnorm(). Named to match the SAS PROBNORM
49#' function and the PyCuDAL probnorm() for side-by-side comparison.
50#'
51#' @param x Numeric. Value(s) at which to evaluate the standard normal CDF.
52#' @return Numeric.  $P(Z \leq x)$  for standard normal  $Z$ .
53probnorm <- function(x) pnorm(x)
54
55#' Standard Normal Quantile Function
56#'
57#' Direct wrapper around qnorm(). Matches SAS PROBIT / PyCuDAL
58#' probit().
59#'

```



```

60#' @param p Numeric. Probability (or probabilities), in (0, 1).
61#' @return Numeric. The standard normal quantile (z-value) for p.
62probit <- function(p) qnorm(p)
63
64#' Chi-Square Cumulative Distribution Function
65#'
66#' Direct wrapper around \code{pchisq()}. Matches SAS PROBCHI / PyCuDAL
67#' \code{probchi()}.
68#'
69#' @param x Numeric. Value(s) at which to evaluate the CDF.
70#' @param df Numeric. Degrees of freedom.
71#' @return Numeric.  $P(X \leq x)$  for chi-square  $X$  with  $df$  degrees of freedom.
72probchi <- function(x, df) pchisq(x, df)
73
74#' Chi-Square Quantile Function
75#'
76#' Direct wrapper around \code{qchisq()}. Matches SAS CINV / PyCuDAL
77#' \code{cinv()}.
78#'
79#' @param p Numeric. Probability.
80#' @param df Numeric. Degrees of freedom.
81#' @return Numeric. The chi-square quantile for  $p$  at  $df$  degrees of freedom.
82cinv <- function(p, df) qchisq(p, df)
83
84#' Content Uniformity Stage Probability
85#'
86#' Computes one stage's pass probability for the USP <905> Content Uniformity
87#' test (Stage 1:  $n = 10$ ,  $k = 2.4$ ; Stage 2:  $n = 30$ ,  $k = 2.0$ ), via the same
88#'  $h = 0.05$  numerical (Riemann-sum) integration used by the original SAS
89#' macros and by PyCuDAL's \code{cu_stage_prob()}.
90#'
91#' All arguments here are scalars: this function is only ever called with a
92#' single (mu, sigma) pair at a time (see the design note at the top of this
93#' file), so no vector broadcasting is required or attempted.
94#'
95#' @param mu Numeric scalar. True population mean (% of label claim).
96#' @param sigma Numeric scalar. True population standard deviation.
97#' @param E Numeric scalar. Adjusted upper test limit (see
98#' \code{content_uniformity_bound}).
99#' @param n Integer. Stage sample size (10 or 30).
100#' @param k Numeric. Stage acceptance constant (2.4 or 2.0).
101#' @return Numeric scalar. The stage pass probability.
102cu_stage_prob <- function(mu, sigma, E, n, k) {
103  h <- 0.05
104  L1 <- 15.0
105
106  z1 <- (E - mu) * sqrt(n) / sigma
107  z2 <- (98.5 - mu) * sqrt(n) / sigma
108  chi_a <- probchi((n - 1) * L1^2 / (k * sigma)^2, df = n - 1)
109  int1 <- (probnorm(z1) - probnorm(z2)) * chi_a
110
111  n_steps <- round(15 / h)
112
113  # int2: Riemann sum over [E, E + 15), step h
114  steps <- 0:(n_steps - 1)
115  xs <- E + steps * h
116  x1 <- (xs - mu) * sqrt(n) / sigma
117  x2 <- (xs + h - mu) * sqrt(n) / sigma
118  chi_args <- (n - 1) * (E + 15 - xs - h / 2)^2 / (k * sigma)^2
119  int2 <- sum((probnorm(x2) - probnorm(x1)) * probchi(chi_args, df = n - 1))
120
121  # int3: Riemann sum over [98.5 - 15, 98.5), step h
122  xs3 <- 98.5 - 15 + steps * h

```

```

123 x1b <- (xs3 - mu) * sqrt(n) / sigma
124 x2b <- (xs3 + h - mu) * sqrt(n) / sigma
125 chi_args3 <- (n - 1) * (15 - 98.5 + xs3 + h / 2)^2 / (k * sigma)^2
126 int3 <- sum((probnorm(x2b) - probnorm(x1b)) * probchi(chi_args3, df = n - 1))
127
128 int1 + int2 + int3
129}
130
131#' Content Uniformity Overall Pass-Probability Bound
132#'
133#' Estimated probability that a batch with true population mean \code{mu}
134#' and true standard deviation \code{sigma} passes the USP <905> Content
135#' Uniformity test (Stage 1 and Stage 2 combined), for label claim/target
136#' \code{target}. Matches PyCuDAL's \code{content_uniformity_bound()}
137#' exactly, including the Stage-2 compound tail term (P2b).
138#'
139#' \code{mu} and \code{sigma} are scalars in every call in this file (see the
140#' design note at the top): \code{target} is always a single fixed value for
141#' an entire acceptance-limit table, so there is no vector-broadcasting
142#' concern here to begin with.
143#'
144#' @param mu Numeric scalar. True population mean (% of label claim).
145#' @param sigma Numeric scalar. True population standard deviation.
146#' @param target Numeric scalar. Label claim / target value (usually 100).
147#' @return Numeric scalar. The overall pass probability ("OVERBD").
148content_uniformity_bound <- function(mu, sigma, target) {
149  E <- if (target <= 101.5) 101.5 else target
150
151  P1 <- cu_stage_prob(mu, sigma, E, n = 10, k = 2.4)
152  P2a <- cu_stage_prob(mu, sigma, E, n = 30, k = 2.0)
153
154  zzz1 <- (123.125 - mu) / sigma
155  zzz2 <- (E - 24.625 - mu) / sigma # reuses E; see design note above
156  P2b <- (probnorm(zzz1) - probnorm(zzz2))^30
157  P2 <- max(0, P2a + P2b - 1)
158
159  max(P1, P2)
160}
161
162#' Two-Tier Variance Components (CUSP2)
163#'
164#' Computes the total variance (VAR) and the upper-confidence-bound
165#' between-location variance (MVAR) from the within-location standard
166#' deviation (SE) and between-location standard deviation (SM). Matches
167#' PyCuDAL's \code{_variance_components()}.
168#'
169#' @param se Numeric. Within-location standard deviation.
170#' @param sm Numeric. Between-location standard deviation.
171#' @param nn Integer. Units assayed per location.
172#' @param l Integer. Number of locations.
173#' @param chierr Numeric. Chi-square quantile for the error (within-location)
174#' variance component.
175#' @param chiloc Numeric. Chi-square quantile for the location (between-
176#' location) variance component.
177#' @return A list with elements \code{var} and \code{mvar}.
178variance_components <- function(se, sm, nn, l, chierr, chiloc) {
179  se2 <- se^2
180  h2 <- 1 * (nn - 1) / chierr - 1
181  sec <- ((1 - 1 / nn) * h2 * se2)^2
182
183  sl2 <- sm^2 * nn
184  sl2ub <- (1 - 1) * sl2 / chiloc
185  h1 <- (1 - 1) / chiloc - 1

```

```

186 first <- ((1 / nn) * h1 * sl2)^2
187
188 ptest <- (1 / nn) * sl2 + (1 - 1 / nn) * se2
189 var_val <- ptest + sqrt(first + sec)
190 mvar_val <- sl2ub
191
192 list(var = var_val, mvar = mvar_val)
193}
194
195
196# =====
197# SECTION 2: Root finding (per grid point, via base R uniroot())
198# =====
199
200#' Find a Single Root of a Monotonically Decreasing Function
201#'
202#' Used by CUSP1: for a fixed candidate MEAN, the pass-probability bound is
203#' monotonically decreasing in the candidate standard deviation, so it has at
204#' most one crossing of the target probability in [lo, hi].
205#'
206#' @param f Function of one scalar argument, decreasing over [lo, hi].
207#' @param lo,hi Numeric. Search interval.
208#' @return A list with \code{root} (numeric) and \code{floor_fail} (logical,
209#' TRUE if even \code{f(lo)} is already below zero -- i.e. no standard
210#' deviation, however small, meets the requirement).
211find_single_root <- function(f, lo, hi) {
212  f_lo <- f(lo)
213  if (f_lo < 0) {
214    return(list(root = lo, floor_fail = TRUE))
215  }
216  f_hi <- f(hi)
217  if (f_hi >= 0) {
218    # Never crosses within [lo, hi]: even the least favorable case in range
219    # still passes. Report hi, matching PyCuDAL's fallback convention.
220    return(list(root = hi, floor_fail = FALSE))
221  }
222  root <- uniroot(f, interval = c(lo, hi))$root
223  list(root = root, floor_fail = FALSE)
224}
225
226#' Find the First and Last Root of a Bump-Shaped Function
227#'
228#' Used by CUSP2: the pass-probability bound is bump-shaped in the candidate
229#' MEAN (rises above the target probability, then falls again), so it has up
230#' to two crossings in [lo, hi]. A coarse scan first brackets each crossing;
231#' \code{uniroot()} then refines each bracket to a precise root.
232#'
233#' @param f Function of one scalar argument.
234#' @param lo,hi Numeric. Search interval.
235#' @param scan_points Integer. Number of points used to bracket the crossings
236#' before refining (default 200 -- the interval here typically spans
237#' 40-50 units, so this resolves brackets to roughly 0.2-0.25 units before
238#' \code{uniroot()} refines further).
239#' @return A list with \code{first} and \code{last} (each NA if no crossing
240#' was found).
241find_first_last_root <- function(f, lo, hi, scan_points = 200) {
242  xs <- seq(lo, hi, length.out = scan_points)
243  vals <- vapply(xs, f, numeric(1))
244  s <- sign(vals)
245  chg <- which(diff(s) != 0)
246
247  if (length(chg) == 0) {
248    return(list(first = NA_real_, last = NA_real_))

```

```

249 }
250
251 first_i <- chg[1]
252 last_i <- chg[length(chg)]
253 first_root <- uniroot(f, interval = c(xs[first_i], xs[first_i + 1]))$root
254 last_root <- uniroot(f, interval = c(xs[last_i], xs[last_i + 1]))$root
255
256 list(first = first_root, last = last_root)
257}
258
259
260# =====
261# SECTION 3: CUSP1 -- Content Uniformity, Sampling Plan 1
262# =====
263
264#' CUSP1 Acceptance-Limit Table
265#'
266#' For each candidate sample MEAN in the grid, finds the largest sample %CV
267#' for which the probability of passing USP <905> still meets the required
268#' lower bound at the given confidence level. Matches PyCuDAL's
269#' {cuspl.acceptance_limit_table()}.
270#'
271#' @param number Integer. Sample size (N).
272#' @param target Numeric. Label claim / target (usually 100).
273#' @param lbound Numeric. Required lower bound on pass probability, as a
274#' percentage (e.g. 95 for 95%).
275#' @param cilevel Numeric. Confidence level, as a percentage (e.g. 95).
276#' @param mean_low Numeric. Low end of the candidate MEAN grid (default 85.1).
277#' @param mean_high Numeric. High end of the candidate MEAN grid (default 114.9).
278#' @param mean_step Numeric. Step size of the candidate MEAN grid (default 0.1).
279#'
280#' @return A data frame with columns {MEAN} and {CV}. A CV of 0
281#' marks a mean where even a (near) zero standard deviation fails to meet
282#' the bound.
283cuspl_acceptance_limit_table <- function(number, target, lbound, cilevel,
284                                         mean_low = 85.1, mean_high = 114.9,
285                                         mean_step = 0.1) {
286  z <- probit((1 + sqrt(cilevel / 100)) / 2)
287  n <- number
288  chi <- cinv(1 - sqrt(cilevel / 100), df = n - 1)
289  target_prob <- lbound / 100
290
291  means <- seq(mean_low, mean_high, by = mean_step)
292  cv <- numeric(length(means))
293
294  for (i in seq_along(means)) {
295    mean_i <- means[i]
296
297    f <- function(sd) {
298      sigma <- sqrt((n - 1) * sd^2 / chi)
299      ll_u <- mean_i - z * sigma / sqrt(n)
300      ul_u <- mean_i + z * sigma / sqrt(n)
301      overlbd <- content_uniformity_bound(ll_u, sigma, target)
302      overubd <- content_uniformity_bound(ul_u, sigma, target)
303      min(overlbd, overubd) - target_prob
304    }
305
306    res <- find_single_root(f, lo = 0.01, hi = 7.8)
307    cv[i] <- if (res$floor_fail) 0 else 100 * res$root / mean_i
308  }
309
310  data.frame(MEAN = means, CV = cv)
311}

```

```

312
313
314# =====
315# SECTION 4: CUSP2 -- Content Uniformity, Sampling Plan 2
316# =====
317
318#' CUSP2 Acceptance-Limit Table
319#'
320#' For each (SE, SM) combination, finds the acceptable range of sample mean
321#' [MEANL, MEANU] for which the probability of passing USP <905> meets the
322#' required lower bound. Matches PyCuDAL's \code{cusp2.acceptance_limit_table()}.
323#'
324#' @param num Integer. Units assayed per location.
325#' @param loc Integer. Number of locations.
326#' @param target Numeric. Label claim / target (usually 100).
327#' @param lbound Numeric. Required lower bound on pass probability (%).
328#' @param cilevel Numeric. Confidence level (%).
329#' @param se_values Numeric vector. Within-location standard deviations to
330#'   evaluate.
331#' @param sm_values Numeric vector. Between-location standard deviations to
332#'   evaluate.
333#' @param mean_search_range Numeric vector of length 2. Search bounds for the
334#'   mean (default c(80, 120)).
335#'
336#' @return A data frame with columns \code{SE}, \code{SM}, \code{MEANL},
337#'   \code{MEANU}. MEANL/MEANU are \code{NA} where no acceptable mean range
338#'   exists for that (SE, SM) combination.
339cusp2_acceptance_limit_table <- function(num, loc, target, lbound, cilevel,
340                                         se_values, sm_values,
341                                         mean_search_range = c(80.0, 120.0)) {
342   nn <- num
343   l <- loc
344   n <- nn * l
345   z <- probit((1 + sqrt(cilevel / 100)) / 2)
346   chierr <- cinv(1 - sqrt(cilevel / 100), df = 1 * (nn - 1))
347   chiloc <- cinv(1 - sqrt(cilevel / 100), df = 1 - 1)
348   target_prob <- lbound / 100
349   lo <- mean_search_range[1]
350   hi <- mean_search_range[2]
351
352   grid <- expand.grid(SE = se_values, SM = sm_values)
353   n_grid <- nrow(grid)
354   meanl <- numeric(n_grid)
355   meanu <- numeric(n_grid)
356
357   for (i in seq_len(n_grid)) {
358     se_i <- grid$SE[i]
359     sm_i <- grid$SM[i]
360
361     vc <- variance_components(se_i, sm_i, nn, l, chierr, chiloc)
362     sigma <- sqrt(vc$var)
363     se_of_mean <- sqrt(vc$mvar / n)
364
365     f <- function(mean_val) {
366       ll_u <- mean_val - z * se_of_mean
367       ul_u <- mean_val + z * se_of_mean
368       overlbd <- content_uniformity_bound(ll_u, sigma, target)
369       overubd <- content_uniformity_bound(ul_u, sigma, target)
370       min(overlbd, overubd) - target_prob
371     }
372
373     res <- find_first_last_root(f, lo, hi)
374

```

```
375   if (is.na(res$first) || is.na(res$last) || res$last <= res$first) {
376     meanl[i] <- NA_real_
377     meanu[i] <- NA_real_
378   } else {
379     meanl[i] <- res$first
380     meanu[i] <- res$last
381   }
382 }
383
384 data.frame(SE = grid$SE, SM = grid$SM, MEANL = meanl, MEANU = meanu)
385}
```

pycudal/utis/cuspl_script.r

```

1source("cudal_cu.r")
2
3groups <- list(
4  c(5, 100.0, 50.0, 50.0), c(2000, 100.0, 50.0, 50.0),
5  c(5, 100.0, 50.0, 99.0), c(2000, 100.0, 50.0, 99.0),
6  c(5, 100.0, 99.0, 99.0), c(2000, 100.0, 99.0, 99.0),
7  c(5, 104.5, 50.0, 50.0), c(2000, 104.5, 50.0, 50.0),
8  c(5, 104.5, 50.0, 99.0), c(2000, 104.5, 50.0, 99.0),
9  c(5, 104.5, 99.0, 99.0), c(2000, 104.5, 99.0, 99.0)
10)
11
12results <- list()
13for (g in groups) {
14  size <- g[1]; target <- g[2]; lbound <- g[3]; ci <- g[4]
15  tab <- cuspl_acceptance_limit_table(number=size, target=target, lbound=lbound, c
ilevel=ci,
16                                     mean_low=85.1, mean_high=114.9, mean_step=1
4.9)
17  for (i in 1:nrow(tab)) {
18    results[[length(results)+1]] <- list(size=size, target=target, lbound=lbound,
ci=ci,
19                                         mean=tab$MEAN[i], cv=tab$CV[i])
20  }
21}
22df <- do.call(rbind, lapply(results, as.data.frame))
23print(round(df, 2))

```

pycudal/utis/cusp1_script.r output

	size	target	lbound	ci	mean	cv
1	5	100.0	50	50	85.1	0.56
2	5	100.0	50	50	100.0	4.88
3	5	100.0	50	50	114.9	0.42
4	2000	100.0	50	50	85.1	0.93
5	2000	100.0	50	50	100.0	7.31
6	2000	100.0	50	50	114.9	0.69
7	5	100.0	50	99	85.1	0.13
8	5	100.0	50	99	100.0	1.16
9	5	100.0	50	99	114.9	0.10
10	2000	100.0	50	99	85.1	0.88
11	2000	100.0	50	99	100.0	7.06
12	2000	100.0	50	99	114.9	0.65
13	5	100.0	99	99	85.1	0.11
14	5	100.0	99	99	100.0	0.94
15	5	100.0	99	99	114.9	0.08
16	2000	100.0	99	99	85.1	0.64
17	2000	100.0	99	99	100.0	5.41
18	2000	100.0	99	99	114.9	0.48
19	5	104.5	50	50	85.1	0.56
20	5	104.5	50	50	100.0	4.75
21	5	104.5	50	50	114.9	1.20
22	2000	104.5	50	50	85.1	0.93
23	2000	104.5	50	50	100.0	7.21
24	2000	104.5	50	50	114.9	1.98
25	5	104.5	50	99	85.1	0.13
26	5	104.5	50	99	100.0	1.14
27	5	104.5	50	99	114.9	0.28
28	2000	104.5	50	99	85.1	0.88
29	2000	104.5	50	99	100.0	6.94
30	2000	104.5	50	99	114.9	1.88
31	5	104.5	99	99	85.1	0.11
32	5	104.5	99	99	100.0	0.93
33	5	104.5	99	99	114.9	0.23
34	2000	104.5	99	99	85.1	0.64
35	2000	104.5	99	99	100.0	5.29
36	2000	104.5	99	99	114.9	1.37

pycudal/utis/cusp2_script.r

```

1source("cudal_cu.r")
2
3full_four <- list(c(0.1,0.1), c(0.1,3.0), c(3.0,0.1), c(3.0,3.0))
4blocks <- list(
5  list(3,2,100.0,50.0,50.0, full_four),
6  list(3,300,100.0,50.0,50.0, full_four),
7  list(300,2,100.0,50.0,50.0, full_four),
8  list(300,300,100.0,50.0,50.0, full_four),
9  list(3,2,100.0,99.0,50.0, full_four),
10 list(3,300,100.0,99.0,50.0, full_four),
11 list(300,2,100.0,99.0,50.0, full_four),
12 list(300,300,100.0,99.0,50.0, full_four),
13 list(3,2,100.0,99.0,99.0, list(c(0.1,0.1), c(3.0,3.0))),
14 list(3,300,100.0,99.0,99.0, full_four),
15 list(300,2,100.0,99.0,99.0, full_four),
16 list(300,300,100.0,99.0,99.0, full_four)
17)
18singles <- list(
19  list(3,2,102.5,50.0,50.0,0.1,3.0),
20  list(300,300,102.5,50.0,99.0,3.0,3.0),
21  list(3,2,102.5,50.0,99.0,0.1,0.1),
22  list(3,300,102.5,50.0,99.0,0.1,3.0)
23)
24
25results <- list()
26for (b in blocks) {
27  loc <- b[[1]]; perloc <- b[[2]]; target <- b[[3]]; lbound <- b[[4]]; ci <- b[[5]]
28  ; pairs <- b[[6]]
29  for (p in pairs) {
30    se <- p[1]; sm <- p[2]
31    tab <- cusp2_acceptance_limit_table(num=perloc, loc=loc, target=target, lbound=lbound, cilevel=ci,
32                                     se_values=c(se), sm_values=c(sm))
33    results[[length(results)+1]] <- list(loc=loc, perloc=perloc, target=target, lb
34    ound=lbound, ci=ci,
35                                     se=se, sm=sm, meanl=tab$MEANL[1], meanu=
36    tab$MEANU[1])
37  }
38}
39for (s in singles) {
40  loc <- s[[1]]; perloc <- s[[2]]; target <- s[[3]]; lbound <- s[[4]]; ci <- s[[5]]
41  ; se <- s[[6]]; sm <- s[[7]]
42  tab <- cusp2_acceptance_limit_table(num=perloc, loc=loc, target=target, lbound=lbound, cilevel=ci,
43                                     se_values=c(se), sm_values=c(sm))
44  results[[length(results)+1]] <- list(loc=loc, perloc=perloc, target=target, lbou
45  nd=lbound, ci=ci,
46                                     se=se, sm=sm, meanl=tab$MEANL[1], meanu=ta
47  b$MEANU[1])
48}
49df2 <- do.call(rbind, lapply(results, as.data.frame))
50print(round(df2, 2))

```

pycudal/utis/cusp2_script.r output

	loc	perloc	target	lbound	ci	se	sm	meanl	meanu
1	3	2	100.0	50	50	0.1	0.1	83.97	116.03
2	3	2	100.0	50	50	0.1	3.0	96.74	103.26
3	3	2	100.0	50	50	3.0	0.1	89.77	110.23
4	3	2	100.0	50	50	3.0	3.0	97.75	102.25
5	3	300	100.0	50	50	0.1	0.1	83.99	116.01
6	3	300	100.0	50	50	0.1	3.0	96.74	103.26
7	3	300	100.0	50	50	3.0	0.1	89.61	110.39
8	3	300	100.0	50	50	3.0	3.0	98.42	101.58
9	300	2	100.0	50	50	0.1	0.1	83.75	116.25
10	300	2	100.0	50	50	0.1	3.0	89.77	110.23
11	300	2	100.0	50	50	3.0	0.1	87.81	112.19
12	300	2	100.0	50	50	3.0	3.0	91.09	108.91
13	300	300	100.0	50	50	0.1	0.1	83.79	116.21
14	300	300	100.0	50	50	0.1	3.0	89.77	110.23
15	300	300	100.0	50	50	3.0	0.1	89.44	110.56
16	300	300	100.0	50	50	3.0	3.0	92.20	107.80
17	3	2	100.0	99	50	0.1	0.1	84.12	115.88
18	3	2	100.0	99	50	0.1	3.0	NA	NA
19	3	2	100.0	99	50	3.0	0.1	92.14	107.86
20	3	2	100.0	99	50	3.0	3.0	NA	NA
21	3	300	100.0	99	50	0.1	0.1	84.15	115.85
22	3	300	100.0	99	50	0.1	3.0	NA	NA
23	3	300	100.0	99	50	3.0	0.1	91.93	108.07
24	3	300	100.0	99	50	3.0	3.0	NA	NA
25	300	2	100.0	99	50	0.1	0.1	83.85	116.15
26	300	2	100.0	99	50	0.1	3.0	92.11	107.89
27	300	2	100.0	99	50	3.0	0.1	89.47	110.53
28	300	2	100.0	99	50	3.0	3.0	93.94	106.06
29	300	300	100.0	99	50	0.1	0.1	83.90	116.10
30	300	300	100.0	99	50	0.1	3.0	92.11	107.89
31	300	300	100.0	99	50	3.0	0.1	91.73	108.27
32	300	300	100.0	99	50	3.0	3.0	95.46	104.54
33	3	2	100.0	99	99	0.1	0.1	89.67	110.33
34	3	2	100.0	99	99	3.0	3.0	NA	NA
35	3	300	100.0	99	99	0.1	0.1	89.66	110.34
36	3	300	100.0	99	99	0.1	3.0	NA	NA
37	3	300	100.0	99	99	3.0	0.1	95.00	105.00
38	3	300	100.0	99	99	3.0	3.0	NA	NA
39	300	2	100.0	99	99	0.1	0.1	83.89	116.11
40	300	2	100.0	99	99	0.1	3.0	93.23	106.77
41	300	2	100.0	99	99	3.0	0.1	90.02	109.98
42	300	2	100.0	99	99	3.0	3.0	95.01	104.99
43	300	300	100.0	99	99	0.1	0.1	83.93	116.07
44	300	300	100.0	99	99	0.1	3.0	93.23	106.77
45	300	300	100.0	99	99	3.0	0.1	91.78	108.22
46	300	300	100.0	99	99	3.0	3.0	96.38	103.62

47	3	2	102.5	50	50	0.1	3.0	96.79	104.21
48	300	300	102.5	50	99	3.0	3.0	93.00	108.00
49	3	2	102.5	50	99	0.1	0.1	88.59	112.41
50	3	300	102.5	50	99	0.1	3.0	NA	NA

APPENDIX D — NAVIGATION AND ERROR CHECKS

Content Uniformity / Sampling Plan 1

PyCuDAL – Content Uniformity & Dissolution Acceptance Limits

File Run View Help

Run Plot OC Curve CSV PDF XLSX Reset About

PyCuDAL Parametric acceptance limits for USP <905> Content Uniformity and USP <711> Dissolution

Content Uniformity -- Plan 1 Content Uniformity -- Plan 2 Dissolution -- Plan 1 Dissolution -- Plan 2

Content Uniformity -- Sampling Plan 1
Single composite sample (USP <905>).

Analysis mode

☒ Acceptance limit table
☐ Probability of passing
☐ Sample probability

Parameters

Number of units (N) 10
Target / label claim (%) 100.0
Lower bound (%) 95.0
Confidence level (%) 95.0
Mean grid low 85.1
Mean grid high 114.9
Mean grid step 0.5

Run Reset 100%

Done -- 61 row(s) computed.

Results 61 rows Plot OC Curve Export CSV Export PDF

MEAN	CV
85.1000	0.3201
85.6000	0.4178
86.1000	0.5143
86.6000	0.6097
87.1000	0.7040
87.6000	0.7973
88.1000	0.8894
88.6000	0.9806
89.1000	1.0707
89.6000	1.1598
90.1000	1.2479
90.6000	1.3350
91.1000	1.4212
91.6000	1.5064
92.1000	1.5907

Created by Moaz El-Essawy

Content Uniformity / Sampling Plan 2

PyCuDAL – Content Uniformity & Dissolution Acceptance Limits

File Run View Help

Run Plot OC Curve CSV PDF XLSX Reset About

PyCuDAL Parametric acceptance limits for USP <905> Content Uniformity and USP <711> Dissolution

Content Uniformity -- Plan 1 **Content Uniformity -- Plan 2** Dissolution -- Plan 1 Dissolution -- Plan 2

Content Uniformity -- Sampling Plan 2
Multiple locations, within/between-location variance components (USP <905>).

Analysis mode

☒ Acceptance limit table
☐ Probability of passing
☐ Sample probability

Parameters

Units per location 6
Number of locations 10
Target / label claim (%) 100.0
Lower bound (%) 95.0
Confidence level (%) 95.0
Within-loc SD -- low 0.5
Within-loc SD -- high 4.0

Run Reset 100%

Done -- 64 row(s) computed.

Results 64 rows Plot OC Curve Export CSV Export PDF

SE	SM	MEANL	MEANU
0.5000	0.5000	86.7203	113.2797
0.5000	1.0000	89.5136	110.4864
0.5000	1.5000	92.4010	107.5990
0.5000	2.0000	95.3147	104.6853
0.5000	2.5000	98.2488	101.7512
0.5000	3.0000		
0.5000	3.5000		
0.5000	4.0000		
1.0000	0.5000	87.5357	112.4643
1.0000	1.0000	89.9398	110.0602
1.0000	1.5000	92.6870	107.3130
1.0000	2.0000	95.5302	104.4698
1.0000	2.5000	98.4227	101.5773
1.0000	3.0000		
1.0000	3.5000		

Created by Moaz El-Essawy

Dissolution / Sampling Plan 1

PyCuDAL – Content Uniformity & Dissolution Acceptance Limits

File Run View Help

Run Plot OC Curve CSV PDF XLSX Reset ? About

PyCuDAL Parametric acceptance limits for USP <905> Content Uniformity and USP <711> Dissolution

Content Uniformity -- Plan 1 Content Uniformity -- Plan 2 **Dissolution -- Plan 1** Dissolution -- Plan 2

Dissolution -- Sampling Plan 1
Single location (USP <711>).

Analysis mode

☒ Acceptance limit table
☐ Probability of passing
☐ Sample probability

Parameters

Number of units (N) 6
Q value (%) 80.0
Lower bound (%) 95.0
Confidence level (%) 95.0
Mean grid step 1.0

Run Reset 100%

Done -- 20 row(s) computed.

Results 20 rows Plot OC Curve Export CSV Export PDF

MEAN	CV
81.0000	0.4453
82.0000	0.8797
83.0000	1.3037
84.0000	1.7176
85.0000	2.1217
86.0000	2.5164
87.0000	2.9012
88.0000	3.2690
89.0000	3.5986
90.0000	3.8700
91.0000	4.0835
92.0000	4.2505
93.0000	4.3832
94.0000	4.4924
95.0000	4.5865

Created by Moaz El-Essawy

Dissolution / Sampling Plan 2

PyCuDAL – Content Uniformity & Dissolution Acceptance Limits

File Run View Help

Run Plot OC Curve CSV PDF XLSX Reset ? About

PyCuDAL Parametric acceptance limits for USP <905> Content Uniformity and USP <711> Dissolution

Content Uniformity -- Plan 1 Content Uniformity -- Plan 2 Dissolution -- Plan 1 **Dissolution -- Plan 2**

Dissolution -- Sampling Plan 2
Multiple locations, within/between-location variance components (USP <711>).

Analysis mode

☒ Acceptance limit table
☐ Probability of passing
☐ Sample probability

Parameters

Units per location 6
Number of locations 5
Q value (%) 80.0
Lower bound (%) 95.0
Confidence level (%) 95.0
Within-loc SD -- low 1.0
Within-loc SD -- high 5.0

Run Reset 100%

Done -- 25 row(s) computed.

Results 25 rows Plot OC Curve Export CSV Export PDF

SE	SM	MEAN
1.0000	1.0000	83.5144
1.0000	2.0000	86.9542
1.0000	3.0000	90.8137
1.0000	4.0000	98.5938
1.0000	5.0000	
2.0000	1.0000	83.6727
2.0000	2.0000	87.0301
2.0000	3.0000	90.9466
2.0000	4.0000	98.8796
2.0000	5.0000	
3.0000	1.0000	83.9425
3.0000	2.0000	87.1629
3.0000	3.0000	91.2006
3.0000	4.0000	99.3641
3.0000	5.0000	

Created by Moaz El-Essawy

APPENDIX E — MATH / LOWER BOUND CALCULATIONS

USP Content Uniformity and Dissolution Tests

Content Uniformity (USP 905)

Stage 1) Test 10 dosage units. Requirements are met if the acceptance value of the first 10 units satisfies

$$AV \leq L_1 = 15$$

Otherwise go to Stage 2.

Stage 2) Test an additional 20 units. Pass if, for all 30 units:

$$AV_{30} \leq 15 \quad \text{and} \quad |x_i - M| \leq 0.25 M \quad \forall i$$

The acceptance value is defined as

$$AV = |M - \bar{X}| + k s, \quad k = \begin{cases} 2.4 & n = 10 \\ 2.0 & n = 30 \end{cases}$$

where $\bar{X}$ is the sample mean and s the sample standard deviation. M is based on the target T (average of the monograph potency limits, % of label claim):

For $T \leq 101.5$:

$$M = \begin{cases} \max(98.5, \bar{X}) & \text{if } \bar{X} \leq 100 \\ \min(101.5, \bar{X}) & \text{if } \bar{X} > 100 \end{cases}$$

For $T > 101.5$:

$$M = \begin{cases} \max(98.5, \bar{X}) & \text{if } \bar{X} \leq 100 \\ \min(T, \bar{X}) & \text{if } \bar{X} > 100 \end{cases}$$

Dissolution (USP 711)

Stage 1) Test 6 units. Pass if

$$x_i \geq Q + 5 \quad \forall i = 1, \dots, 6$$

Stage 2) Test 6 additional units. Pass if

$$\bar{x}_{12} \geq Q \quad \text{and} \quad x_i > Q - 15 \quad \forall i$$

Stage 3) Test 12 additional units. Pass if

$$\bar{x}_{24} \geq Q,$$

with no more than two results $\leq Q - 15$ and no result $\leq Q - 25$. Otherwise Fail.

Statistical Methods

Assume normal dosage-unit results $X_i \sim \mathcal{N}(\mu, \sigma^2)$. Then

$$Z_1 = \bar{X} \sim \mathcal{N}\left(\mu, \frac{\sigma}{\sqrt{n}}\right), \quad Z_2 = \frac{(n-1)s^2}{\sigma^2} \sim \chi_{n-1}^2$$

are independent, and the overall lower bound of passing the USP test is

$$P(\text{pass}) \geq \max\{P(S_1), P(S_2)\}$$

Computation of $P(S_1)$

With $L_1 = 15$ and $k_1 = 2.4$:

$$P(S_1) = (\Phi(t_1) - \Phi(t_2)) P\left(\chi_{n-1}^2 < \frac{(n-1)L_1^2}{k_1^2 \sigma^2}\right) + I_2 + I_3$$

$$t_1 = \frac{101.5 - \mu}{\sigma/\sqrt{n}}, \quad t_2 = \frac{98.5 - \mu}{\sigma/\sqrt{n}}$$

The tail integrals are evaluated numerically with step $h = 0.05$:

$$I_2 = \sum_{i=1}^K [\Phi(z_0 + ih) - \Phi(z_0 + (i-1)h)] P(\chi_{n-1}^2 < g(z_0 + (i - \frac{1}{2})h))$$

$$g(z_1) = \frac{(n-1)(L_1 + 101.5 - z_1)^2}{k_1^2 \sigma^2}$$

$$I_3 = \sum_{i=1}^K [\Phi(z_0 + ih) - \Phi(z_0 + (i-1)h)] P(\chi_{n-1}^2 < q(z_0 + (i - \frac{1}{2})h))$$

$$q(z_1) = \frac{(n-1)(L_1 - 98.5 + z_1)^2}{k_1^2 \sigma^2}$$

Computation of $P(S_2)$

Let $C_{21} = \text{“AV of the 30 units} \leq L_1\text{”}$ and $C_{22} = \text{“no unit deviates from } M \text{ by more than } L_2 = 25\text{”}$. Using $P(A \cap B) \geq P(A) + P(B) - 1$:

$$P(S_2) = P(C_{21} \cap C_{22}) \geq \max\{P(C_{21}) + P(C_{22}) - 1, 0\}$$

$P(C_{21})$ is computed exactly like $P(S_1)$ with $n = 30$, $k_2 = 2.0$, and

$$P(C_{22}) \geq \left[\Phi\left(\frac{98.5 + L_2 - \mu}{\sigma/\sqrt{n}}\right) - \Phi\left(\frac{101.5 - L_2 - \mu}{\sigma/\sqrt{n}}\right) \right]^n$$

(replace 101.5 by T when $T > 101.5$).

Dissolution stage probabilities

Let ℓ be the adjusted lower bound on the mean offset from Q ($\ell = (\text{mean} - Q) - z SE_{\text{mean}}$); units are $x \sim \mathcal{N}(Q + \ell, \sigma)$. The three stage bounds are

$$F_1 = [P(x \geq Q + 5)]^6 = \left[1 - \Phi\left(\frac{5 - \ell}{\sigma}\right)\right]^6$$

$$F_2 = [P(x \geq Q - 15)]^{12} - P(\bar{x}_{12} < Q)$$

$$F_3 = p_3^{24} + 24 p_2 p_3^{23} + 276 p_2^2 p_3^{22} - P(\bar{x}_{24} < Q)$$

with

$$p_2 = \Phi\left(\frac{-15 - \ell}{\sigma}\right) - \Phi\left(\frac{-25 - \ell}{\sigma}\right), \quad p_3 = 1 - \Phi\left(\frac{-15 - \ell}{\sigma}\right)$$

$$P(\bar{x}_n < Q) = \Phi\left(\frac{\sqrt{n}(-\ell)}{\sigma}\right)$$

and the overall bound is $P(\text{pass}) = \max(F_1, F_2, F_3)$.

Plan 2 variance components

With n units per location, L locations and $N = nL$:

$$\chi_{err}^2 = F_{\chi^2}^{-1}(1 - \sqrt{C/100}; L(n-1)), \quad \chi_{loc}^2 = F_{\chi^2}^{-1}(1 - \sqrt{C/100}; L-1)$$

$$h_1 = \frac{L-1}{\chi_{loc}^2} - 1, \quad h_2 = \frac{L(n-1)}{\chi_{err}^2} - 1$$

$$\text{VAR} = (s_m^2 + (1 - \frac{1}{n}) s_e^2) + \sqrt{(h_1 s_m^2)^2 + ((1 - \frac{1}{n}) h_2 s_e^2)^2}$$

$$\text{MVAR} = \frac{(L-1) n s_m^2}{\chi_{loc}^2}$$

The mean limits use the standard error $\sqrt{\text{MVAR}/N}$ with the confidence multipliers

$$z = \Phi^{-1}\left(\frac{1 + \sqrt{C/100}}{2}\right) \quad (\text{CU, two-sided}), \quad z = \Phi^{-1}\left(\sqrt{C/100}\right) \quad (\text{dissolution, one-sided})$$

APPENDIX F — PROGRAM DESCRIPTION

F.1 Overview

PyCuDAL (Content Uniformity and Dissolution Acceptance Limits) is a Python application that calculates parametric, tolerance-interval-based acceptance limits and pass-probability estimates for two United States Pharmacopeia (USP) compendial tests: USP <905> Uniformity of Dosage Units (Content Uniformity) and USP <711> Dissolution. The Program is a translation of a legacy SAS/AF application of the same name, re-implemented in Python using numpy, scipy, and pandas.

F.2 Functional Modules

The Program is organized into five modules:

Module	Contents
cudal.core	Shared statistical primitives and the vectorized root-finding engine used by all four calculation modules.
cudal.cusp1	Content Uniformity (<905>), Sampling Plan 1 — single location / composite sample.
cudal.cusp2	Content Uniformity (<905>), Sampling Plan 2 — multiple locations (within/between-location variance components).
cudal.disp1	Dissolution (<711>), Sampling Plan 1 — single location.
cudal.disp2	Dissolution (<711>), Sampling Plan 2 — multiple locations.

F.3 Analysis Modes

Each of the four calculation modules (cusp1, cusp2, disp1, disp2) supports three analysis modes, implemented as three functions with a consistent naming pattern:

- Acceptance-Limit Table (acceptance_limit_table) — computes the boundary (maximum acceptable variability, or acceptable mean range) for a grid of candidate sample means and/or standard deviations, such that the probability of passing the USP test meets a specified lower bound and confidence level.
- Probability of Passing (probability_of_passing) — given the acceptance-limit table, estimates the probability that a batch with an assumed true population mean and variability will produce a sample within the acceptance region.
- Sample Probability (sample_probability) — given an observed sample mean and variability, directly computes the probability that future samples from that population will pass the USP test.

F.4 cudal.core — Shared Statistical Functions

This module contains the statistical primitives and the vectorized root-finding engine shared by all four calculation modules.

probnorm(x)

Computes the standard normal cumulative distribution function (CDF). Equivalent to the SAS PROBNORM function; implemented via `scipy.special.ndtr` for performance.

Inputs:

- `x` — value or array of values at which to evaluate the standard normal CDF.

Returns: The probability that a standard normal random variable is less than or equal to `x`.

probit(p)

Computes the inverse of the standard normal CDF (quantile function). Equivalent to the SAS PROBIT function.

Inputs:

- `p` — probability (or array of probabilities), between 0 and 1.

Returns: The standard normal quantile (z-value) corresponding to `p`.

probchi(x, df)

Computes the chi-square cumulative distribution function. Equivalent to the SAS PROBCHI function; implemented via the regularized incomplete gamma function for performance.

Inputs:

- `x` — value(s) at which to evaluate the CDF.
- `df` — degrees of freedom.

Returns: The probability that a chi-square random variable with `df` degrees of freedom is less than or equal to `x`.

cinv(p, df)

Computes the inverse chi-square CDF (quantile function). Equivalent to the SAS CINV function.

Inputs:

- `p` — probability.
- `df` — degrees of freedom.

Returns: The chi-square quantile corresponding to `p` at `df` degrees of freedom.

sas_do_range(start, stop, step)

Internal helper that reproduces the sequence of values generated by a SAS “DO start TO stop BY step;” loop, used to build the fixed numerical-integration grid for the content-uniformity probability calculation.

Inputs:

- `start, stop, step` — loop bounds and increment.

Returns: A numpy array of the loop's index values.

content_uniformity_bound(mu, sigma, target)

Computes the estimated probability that a batch with true population mean `mu` and true standard deviation `sigma` passes the USP <905> Content Uniformity test (Stage 1, `n`=10, and Stage 2, `n`=30), for a given label claim/target. This is the core probability engine shared by the CUSP1 and CUSP2 modules. Fully vectorized and broadcastable across arrays.

Inputs:

- `mu` — true population mean (% of label claim).
- `sigma` — true population standard deviation.
- `target` — label claim / target value (usually 100).

Returns: The estimated pass probability (“OVERBD”) — the greater of the Stage 1 and Stage 2 pass probabilities.

dissolution_bound(llu, sigma)

Computes the estimated probability that a batch passes the USP <711> Dissolution test (Stage 1, n=6; Stage 2, n=12; Stage 3, n=24), given a confidence-adjusted lower bound on the population mean and the population standard deviation. This is the core probability engine shared by the DISP1 and DISP2 modules. Fully vectorized and broadcastable.

Inputs:

- llu — confidence-adjusted lower bound on the population mean.
- sigma — population standard deviation.

Returns: The estimated pass probability — the greatest of the three USP <711> stage probabilities.

batched_root_find(func, lo, hi, scan_points, bisect_iters, which)

Solves for the boundary value (root) of a probability function across an entire grid of inputs simultaneously, using a vectorized scan-and-bisect method, in place of the sequential stepped search used by the legacy SAS code. Used by all four modules' acceptance_limit_table functions.

Inputs:

- func — the (vectorized) function whose root is sought.
- lo, hi — search interval bounds.
- scan_points — number of points used to bracket the root.
- bisect_iters — number of bisection refinement iterations.
- which — “first” or “last”, selecting which crossing to report when the function has more than one root in the interval.

Returns: The located root value(s) and a flag indicating whether a valid root was found, for every grid point.

batched_two_sided_bounds(func_lower, func_upper, lo, hi, scan_points, bisect_iters)

Convenience function used by CUSP2 to find both the lower (MEANL) and upper (MEANU) acceptable mean boundaries in a single call, by invoking batched_root_find once per side (“first” crossing for the lower bound, “last” crossing for the upper bound).

Inputs:

- func_lower, func_upper — the lower- and upper-bound probability functions.
- lo, hi — search interval.
- scan_points, bisect_iters — as above.

Returns: The lower bound, its found flag, the upper bound, and its found flag, for every grid point.

F.5 cudal.cuspl — Content Uniformity, Sampling Plan 1

acceptance_limit_table(number, target, lbound, cilevel, mean_low, mean_high, mean_step)

Builds the Content Uniformity Sampling Plan 1 acceptance-limit table: for each candidate sample mean in the specified grid, finds the largest sample %CV for which the probability of passing USP <905> still meets the required lower bound and confidence level.

Inputs:

- number — sample size (N).
- target — label claim.
- lbound — required lower bound on pass probability (%).
- cilevel — confidence level (%).
- mean_low, mean_high, mean_step — range and step size of the candidate mean grid.

Returns: A table (MEAN, CV) of acceptance-limit values.

probability_of_passing(table, number, u_values, cv_values)

Given the acceptance-limit table, estimates the probability that a batch with an assumed true population mean (U) and %CV will produce a sample that falls inside the acceptance region.

Inputs:

- table — the acceptance-limit table.
- number — sample size.
- u_values — grid of assumed true population means.
- cv_values — grid of assumed true %CVs.

Returns: A table of (U, CV, PTRAP) — the estimated probability of passing for each combination.

sample_probability(mean, cv, number, target, lbound, cilevel)

Given an observed sample mean and %CV, directly computes the probability that future samples from that population will pass USP <905>.

Inputs:

- mean — observed sample mean.
- cv — observed sample %CV.
- number — sample size.
- target — label claim.
- lbound, cilevel — reporting context (do not affect the calculation itself).

Returns: The observed sample standard deviation and the estimated pass probability (OVERBD).

F.6 cudal.cusp2 — Content Uniformity, Sampling Plan 2

`_variance_components(se, sm, nn, l, chierr, chiloc)`

Internal helper that computes the total variance (VAR) and the upper-confidence-bound between-location variance (MVAR) from the within-location standard deviation (SE) and between-location standard deviation (SM). Used by all three CUSP2 functions.

Inputs:

- `se` — within-location standard deviation.
- `sm` — between-location standard deviation.
- `nn` — units per location.
- `l` — number of locations.
- `chierr, chiloc` — chi-square quantiles for the error and location variance components.

Returns: VAR (total variance) and MVAR (upper-bound between-location variance).

`acceptance_limit_table(num, loc, target, lbound, cilevel, se_values, sm_values, mean_search_range)`

Builds the Content Uniformity Sampling Plan 2 acceptance-limit table: for each combination of within-location SD (SE) and between-location SD (SM), finds the acceptable range of sample mean [MEANL, MEANU] for which the probability of passing USP <905> meets the required lower bound.

Inputs:

- `num` — units per location.
- `loc` — number of locations.
- `target, lbound, cilevel` — as in CUSP1.
- `se_values, sm_values` — grids of SE/SM to evaluate.
- `mean_search_range` — search bounds for the mean.

Returns: A table (SE, SM, MEANL, MEANU); MEANL/MEANU are blank where no acceptable mean range exists.

`probability_of_passing(table, num, loc, d1, u_values, sigse_values, sigsm_values)`

Given the acceptance-limit table, sums over every (SE, SM) grid cell the probability that a batch with an assumed true mean and true within-/between-location standard deviations falls within the acceptance region.

Inputs:

- `table` — the acceptance-limit table.
- `num, loc` — as above.
- `d1` — bin width used for the SE/SM grid.
- `u_values, sigse_values, sigsm_values` — grids of assumed true mean/standard deviations.

Returns: A table of (U, SIGSE, SIGSM, PSUM) — the estimated probability of passing.

`sample_probability(mean, se, sm, num, loc, target, cilevel)`

Given an observed sample mean, within-location SD, and between-location SD, computes the probability that future samples from that population will pass USP <905>.

Inputs:

- `mean, se, sm` — observed sample statistics.
- `num, loc, target, cilevel` — as above.

Returns: The computed VAR, MVAR, SIGMA, the lower- and upper-bound pass probabilities, and the overall pass probability (OVERBD).

F.7 cudal.disp1 — Dissolution, Sampling Plan 1

`acceptance_limit_table(number, q, lbound, cilevel, meanadj_step)`

Builds the Dissolution Sampling Plan 1 acceptance-limit table: for each candidate sample mean (expressed relative to the USP Q value), finds the largest sample %CV for which the probability of passing USP <711> meets the required lower bound.

Inputs:

- number — sample size.
- q — USP Q value (% dissolved).
- lbound, cilevel — as in CUSP1.
- meanadj_step — step size of the internal mean-adjustment grid.

Returns: A table (MEAN, CV) of acceptance-limit values.

`probability_of_passing(table, number, u_values, cv_values)`

Given the acceptance-limit table, estimates the probability of passing for a grid of assumed true population means and %CVs, including the one-sided upper-tail correction used for means above 99.9% of label claim.

Inputs:

- table, number, u_values, cv_values — as in CUSP1's equivalent function.

Returns: A table of (U, CV, PTRAP).

`sample_probability(mean, cv, number, q, cilevel)`

Given an observed sample mean and %CV, directly computes the probability that future samples will pass USP <711>.

Inputs:

- mean, cv, number, q, cilevel — as above.

Returns: The observed sample standard deviation and the estimated pass probability (OVERBD).

F.8 cudal.disp2 — Dissolution, Sampling Plan 2

acceptance_limit_table(num, loc, q, lbound, cilevel, se_values, sm_values, meanadj_search_range)

Builds the Dissolution Sampling Plan 2 acceptance-limit table: for each (SE, SM) combination, finds the smallest acceptable sample mean for which the probability of passing USP <711> meets the required lower bound. Unlike CUSP2, dissolution has only a lower bound, so a single MEAN boundary is found per grid cell rather than a two-sided range.

Inputs:

- num, loc, q, lbound, cilevel — as above.
- se_values, sm_values — grids of SE/SM.
- meanadj_search_range — search bounds for the mean adjustment.

Returns: A table (SE, SM, MEAN); MEAN is blank where no acceptable mean exists.

probability_of_passing(table, num, loc, dse, dsm, u_values, sigse_values, sigsm_values)

One-sided analogue of CUSP2's probability_of_passing; sums over the (SE, SM) grid the probability that a batch's location mean, within-location SD, and between-location SD together fall within the acceptance region.

Inputs:

- table, num, loc — as above.
- dse, dsm — bin widths for the SE/SM grid.
- u_values, sigse_values, sigsm_values — grids of assumed true values.

Returns: A table of (U, SIGSE, SIGSM, PSUM).

sample_probability(mean, se, sm, num, loc, q, cilevel)

Given an observed sample mean, within-location SD, and between-location SD, computes the probability that future samples will pass USP <711>.

Inputs:

- mean, se, sm, num, loc, q, cilevel — as above.

Returns: The computed VAR, MVAR, SIGMA, and the overall pass probability (OVERBD).

F.9 Technical Implementation

The Program is implemented in Python 3, using numpy and scipy for numerical computation and pandas for tabular output. Acceptance-limit boundaries are located using a vectorized numerical root-finding method (batched bisection, see Section F.4), replacing the sequential, incrementally-stepped search used in the original SAS macros. All underlying probability formulas (content uniformity and dissolution pass-probability calculations) have been verified to be mathematically identical to the original SAS macros.

F.10 Known Behavior

Under one specific, narrow condition — very small within-location and between-location standard deviation combined with a large sample size in the Content Uniformity Sampling Plan 2 (CUSP2) module — the Python Program's acceptance-limit result differs from the original SAS output. This has been investigated and documented; the difference is attributable to a path-dependent artifact of the original SAS/S-PLUS sequential search algorithm, and the Python implementation has been confirmed to produce the mathematically correct result. Refer to the CUSP2 Acceptance-Limit Discrepancy Investigation (Technical Note) for full details.

F.11 Intended Use

The Program is intended for use to establish and evaluate parametric acceptance limits in support of Content Uniformity and Dissolution testing strategies, consistent with USP <905> and USP <711>.

APPENDIX G — PROGRAM TEST DATA SETS

G.1 Content Uniformity — Sampling Plan 1 (CUSP1)

Sample Size	Target	Lower Bound	CI Level	Sample Mean	Program CV Output	R Code CV Output
5	100.0	50.0	50.0	85.1		
				100.0		
				114.9		
2,000	100.0	50.0	50.0	85.1		
				100.0		
				114.9		
5	100.0	50.0	99.0	85.1		
				100.0		
				114.9		
2,000	100.0	50.0	99.0	85.1		
				100.0		
				114.9		
5	100.0	99.0	99.0	85.1		
				100.0		
				114.9		
2,000	100.0	99.0	99.0	85.1		
				100.0		
				114.9		
5	104.5	50.0	50.0	85.1		
				100.0		
				114.9		
2,000	104.5	50.0	50.0	85.1		
				100.0		
				114.9		
5	104.5	50.0	99.0	85.1		
				100.0		
				114.9		
2,000	104.5	50.0	99.0	85.1		
				100.0		
				114.9		
5	104.5	99.0	99.0	85.1		
				100.0		
				114.9		

Sample Size	Target	Lower Bound	CI Level	Sample Mean	Program CV Output	R Code CV Output
2,000	104.5	99.0	99.0	85.1		
				100.0		
				114.9		

G.2 Content Uniformity — Sampling Plan 2 (CUSP2)

#Location	#/Location	Target	Lower Bound	CI Level	SE	SM	Program Output		R Code Output	
							LL	UL	LL	UL
3	2	100.0	50.0	50.0	0.1	0.1				
					0.1	3.0				
					3.0	0.1				
					3.0	3.0				
3	300	100.0	50.0	50.0	0.1	0.1				
					0.1	3.0				
					3.0	0.1				
					3.0	3.0				
300	2	100.0	50.0	50.0	0.1	0.1				
					0.1	3.0				
					3.0	0.1				
					3.0	3.0				
300	300	100.0	50.0	50.0	0.1	0.1				
					0.1	3.0				
					3.0	0.1				
					3.0	3.0				
3	2	100.0	99.0	50.0	0.1	0.1				
					0.1	3.0				
					3.0	0.1				
					3.0	3.0				
3	300	100.0	99.0	50.0	0.1	0.1				
					0.1	3.0				
					3.0	0.1				
					3.0	3.0				
300	2	100.0	99.0	50.0	0.1	0.1				
					0.1	3.0				
					3.0	0.1				
					3.0	3.0				

#Location	#/Location	Target	Lower Bound	CI Level	SE	SM	Program Output		R Code Output	
							LL	UL	LL	UL
300	300	100.0	99.0	50.0	0.1	0.1				
					0.1	3.0				
					3.0	0.1				
					3.0	3.0				
3	2	100.0	99.0	99.0	0.1	0.1				
					3.0	3.0				
3	300	100.0	99.0	99.0	0.1	0.1				
					0.1	3.0				
					3.0	0.1				
					3.0	3.0				
300	2	100.0	99.0	99.0	0.1	0.1				
					0.1	3.0				
					3.0	0.1				
					3.0	3.0				
300	300	100.0	99.0	99.0	0.1	0.1				
					0.1	3.0				
					3.0	0.1				
					3.0	3.0				
3	2	102.5	50.0	50.0	0.1	3.0				
300	300	102.5	50.0	99.0	3.0	3.0				
3	2	102.5	50.0	99.0	0.1	0.1				
3	300	102.5	50.0	99.0	0.1	3.0				

APPENDIX H — CURRICULUM VITAE'S

Attached

FORM 1 — LOAD AND RUN PROGRAM

Program / Module Name	
Version / Build	
Test Date	
Performed By	

Test Steps

Step	Test Step / Action	Expected Result	Pass / Fail	Comments
1	Confirm the Python environment and required packages (numpy, scipy, pandas) are installed.	Environment check completes with no missing dependencies.		
2	Launch the PyCuDAL program.	Program starts without error and displays the main menu.		
3	Confirm the program name and version information are displayed.	Correct program name and version are shown.		
4	Confirm no error messages or warnings appear on startup.	No errors or warnings are displayed.		
5	Confirm the main menu options are displayed (Content Uniformity, Dissolution).	All expected menu options are visible and selectable.		
6	Exit the program using the standard exit option.	Program closes cleanly without error.		

Comments / Deviations

--

Sign-Off

Role	Name (printed)	Signature	Date
Performed By			
Reviewed By			
Approved By			

FORM 2 — PRIMARY WINDOW NAVIGATION

Program / Module Name	
Version / Build	
Test Date	
Performed By	

Test Steps

Step	Test Step / Action	Expected Result	Pass / Fail	Comments
1	Select Content Uniformity Sampling Plan 1 (CUSP1).	CUSP1 analysis-mode screen is displayed (Table / Evaluate / Sample).		
2	Select Content Uniformity Sampling Plan 2 (CUSP2).	CUSP2 analysis-mode screen is displayed (Table / Evaluate / Sample).		
3	Select Dissolution Sampling Plan 1 (DISP1).	DISP1 analysis-mode screen is displayed (Table / Evaluate / Sample).		
4	Select Dissolution Sampling Plan 2 (DISP2).	DISP2 analysis-mode screen is displayed (Table / Evaluate / Sample).		
5	Exit the program from a nested screen.	Program exits cleanly / returns to main menu without error.		

Comments / Deviations

--

Sign-Off

Role	Name (printed)	Signature	Date
Performed By			
Reviewed By			
Approved By			

FORM 3 — PROGRAM STRATEGY / PYTHON CODE VERIFICATION

Program / Module Name	
Version / Build	
Test Date	
Performed By	

Test Steps

Step	Test Step / Action	Expected Result	Pass / Fail	Comments
1	Review cudal.core probability functions (probnorm, probit, probchi, cinv) against Appendix E.	Implementations match the documented USP statistical formulas.		
2	Review content_uniformity_bound() against the USP <905> strategy in Appendix E/F.	Function correctly implements the Stage 1 (n=10) and Stage 2 (n=30) pass-probability formulas.		
3	Review dissolution_bound() against the USP <711> strategy in Appendix E/F.	Function correctly implements the Stage 1/2/3 (n=6/12/24) pass-probability formulas.		
4	Review batched_root_find() and batched_two_sided_bounds() logic.	Root-finding logic correctly locates acceptance-limit boundaries per the documented strategy.		
5	Review cusp1.py functions against the documented CUSP1 strategy.	acceptance_limit_table, probability_of_passing, and sample_probability match the documented strategy.		
6	Review cusp2.py functions against the documented CUSP2 strategy.	Variance-component and acceptance-limit logic match the documented strategy.		
7	Review disp1.py and disp2.py functions against the documented DISP1/DISP2 strategy.	Function logic matches the documented strategy for both modules.		
8	Confirm no undocumented or unauthorized code exists outside the described functional scope.	Code matches Appendix A (Program Listing) with no unexplained additions.		

Comments / Deviations

--

Sign-Off

Role	Name (printed)	Signature	Date
Performed By			
Reviewed By			
Approved By			

FORM 4 — TEST DATA AGREEMENT

Program / Module Name	
Version / Build	
Test Date	
Performed By	

Test Steps

Step	Test Step / Action	Expected Result	Pass / Fail	Comments
1	Execute CUSP1 Acceptance-Limit Table using the Appendix G test data set.	Output agrees with the predefined expected values within tolerance.		
2	Execute CUSP1 Probability of Passing / Sample Probability using the test data set.	Output agrees with the predefined expected values within tolerance.		
3	Execute CUSP2 Acceptance-Limit Table using the test data set.	Output agrees with the predefined expected values within tolerance.		
4	Execute CUSP2 Probability of Passing / Sample Probability using the test data set.	Output agrees with the predefined expected values within tolerance.		
5	Execute DISP1 Acceptance-Limit Table using the test data set.	Output agrees with the predefined expected values within tolerance.		
6	Execute DISP1 Probability of Passing / Sample Probability using the test data set.	Output agrees with the predefined expected values within tolerance.		
7	Execute DISP2 Acceptance-Limit Table using the test data set.	Output agrees with the predefined expected values within tolerance.		
8	Execute DISP2 Probability of Passing / Sample Probability using the test data set.	Output agrees with the predefined expected values within tolerance.		

Comments / Deviations

--

Sign-Off

Role	Name (printed)	Signature	Date
Performed By			
Reviewed By			
Approved By			

FORM 5 — PROBLEM / REQUEST REPORT

Date Raised	
Raised By	
Program / Module	

Description of Problem / Request

--

Severity

☐ Critical ☐ Major ☐ Minor ☐ Enhancement Request

Root Cause

--

Corrective Action / Resolution

--

Status

☐ Open ☐ In Progress ☐ Closed

Sign-Off

Role	Name (printed)	Signature	Date
Performed By			
Reviewed By			
Approved By			

CUSP2 Acceptance-Limit Discrepancy Investigation

Technical Note

Comparison of the Content Uniformity Sampling Plan 2 (multi-location) acceptance-limit calculation between the original SAS CuDAL system and its Python translation

1. Summary

For most parameter combinations, the Python translation of Cusp2.sas reproduces SAS's acceptance-limit table (MEANL / MEANU) to within a small fraction of a percent. However, at very small within-location and between-location standard deviations (SE, SM), a visible gap appears between the two outputs.

Example case

Parameter	Value
Locations (LOC)	3
Units per location (NUM)	300
Lower bound	99%
Confidence level	50%
Target	100
SE	0.1
SM	0.1

Source	MEANL	MEANU
SAS PyCuDAL	84.8	115.2
Python (independent solve)	84.479	115.520

Root cause: this is not a numerical-accuracy problem in either implementation. It is a search-algorithm artifact in the original SAS code, which only becomes visible when the underlying probability curve is extremely steep — which happens precisely when SE and SM are very small. The Python version finds the mathematically correct root of the exact same formula; SAS's sequential, warm-started search finds a nearby point instead, because at this steepness almost any nearby starting guess already looks like it “passes.”

2. Two Hypotheses That Were Ruled Out

Before identifying the real cause, two commonly-suspected sources of error were checked and eliminated.

2.1 The $h = 0.05$ numerical integration step

The content-uniformity probability formula (c1calc / cullu / cuulu in SAS) approximates two integrals with a manual Riemann sum using step size $h = 0.05$ (about 300 terms). This step size, the integration grid, and the formula are byte-for-byte identical between the SAS code and the Python translation.

Because both implementations use the exact same integration scheme, this cannot be the source of a difference between them — it would affect both equally, not create a gap.

2.2 SAS's 0.1% rounding

SAS's mean search steps in increments of 0.1 (DO MEAN = ... BY D; with $D = 0.10$), so its reported boundary is always rounded to the nearest 0.1. This does introduce a small, expected discrepancy — but only on the order of 0.1 units. The observed gap here (84.8 vs. 84.479, a difference of 0.321) is roughly three times larger than rounding alone can explain. Something else is stacking on top of the rounding.

3. What Is Actually Happening

3.1 The probability curve becomes a near step-function

With $SE = SM = 0.1$ and a large sample ($NUM = 300$, $LOC = 3$), the resulting population standard deviation fed into the USP <905> probability formula (sigma) is very small (≈ 0.197). USP <905>'s own internal test always uses a fixed sample size ($n = 10$ for stage 1, $n = 30$ for stage 2) — this is a fixed part of the USP method, unrelated to NUM/LOC . At such a small sigma, that fixed-size test becomes almost deterministic: the pass probability jumps from ~0% to ~100% within about one unit of mean, instead of changing gradually.

This was confirmed by evaluating the probability function directly at this exact sigma:

Mean (mu)	Pass probability
83.0	$\approx 3 \times 10^{-26}$ (essentially zero)
84.0	0.953
84.14	0.9999
84.5 and above	1.0000 (fully saturated)

The required probability (99%) sits inside that narrow, steep transition — right around $\mu \approx 84.1$ –84.2. This is the true root of the formula, and it is what the Python implementation finds.

3.2 Why SAS lands on a different point

SAS's search is not an independent solve for each grid cell. It is a sequential, warm-started search:

```
DO MEAN = MEANL - D TO 115.5 BY D;
    %cullu;
    IF OVERBDL > LBOUND/100 THEN DO;
        MEANL = MEAN;
        GOTO UPPER;
    END;
END;
```

MEANL here is inherited from the previous SE/SM grid cell and the loop only scans upward from $MEANL_{previous} - D$, stopping at the very first point that passes.

Because the true curve is so steep in this regime, almost any inherited starting point above ~84.5 will already register as “passing” on the very first check. SAS's loop therefore never actually walks back down toward the true crossing near 84.1 — it simply reports wherever its inherited starting guess happened to be. The result (84.8) is a by-product of the search path, not a solve of the equation.

The Python implementation instead performs an independent, from-scratch root search for every grid cell, so it always converges to the true crossing of the identical formula — which is why it comes out consistently wider (more permissive) than SAS specifically in this regime.

3.3 Why this only shows up at very small SE/SM

At more typical variability levels (SE, SM on the order of 1–3), the probability curve is much gentler and changes gradually over many units of mean rather than snapping within one unit. SAS's warm-started guess lands close enough to the true root in those cases that the two implementations agree closely (differences on the order of the expected 0.1 rounding only). The discrepancy is specific to the near-zero-variability corner of the parameter space.

4. Conclusion

- This is not a bug in the Python translation's formulas or numerical integration — those match SAS exactly.
- This is not primarily a rounding issue — rounding alone cannot produce a gap this large.
- The gap is a path-dependent artifact of SAS's sequential search algorithm, exposed only when the underlying probability curve is extremely steep (i.e., at very small SE/SM). In that regime, SAS's answer depends on which nearby grid cell it happened to start from, rather than on the true root of the formula.
- The Python implementation's result (84.479 / 115.520) is the mathematically correct root of the shared formula. SAS's result (84.8 / 115.2) is a valid output of its particular search procedure, but not a more accurate answer to the underlying equation.